

POST-OCR TEXT CORRECTION FOR OTTOMAN TURKISH
TRANSCRIPTION

by
ALİZE SEVGİ YALÇINKAYA

Submitted to the Graduate School of Engineering and Natural Sciences
in partial fulfilment of
the requirements for the degree of Master of Science
in Computer Science and Engineering

Sabancı University
May 2026

POST-OCR TEXT CORRECTION FOR OTTOMAN TURKISH
TRANSCRIPTION

Approved by:

Prof. BERRİN YANIKOĞLU
(Thesis Supervisor)

Asst. Prof. DİLARA KEKÜLLÜOĞLU
(Thesis Co-Supervisor)

Prof. SELİM BALCISOY

Assoc. Prof. ÖZNUR TAŞTAN

Asst. Prof. ESMA BİLGİN TAŞDEMİR

Date of Approval: May 20, 2026



ALİZE SEVGİ YALÇINKAYA 2026 ©

All Rights Reserved

ABSTRACT

POST-OCR TEXT CORRECTION FOR OTTOMAN TURKISH TRANSCRIPTION

ALİZE SEVGİ YALÇINKAYA

Computer Science and Engineering, M.Sc. Thesis, May 2026

Thesis Supervisor: Prof. BERRİN YANIKOĞLU

Thesis Co-Supervisor: Asst. Prof. DİLARA KEKÜLLÜOĞLU

Keywords: ocr post-correction, ottoman turkish, confidence calibration, error detection, low-resource nlp

Historical document digitization requires effective post-OCR error correction, particularly for low-resource scripts like Ottoman Turkish that lack modern native speakers and linguistic tools. We develop and evaluate fully automatic post-processing methods for Ottoman texts digitized via a frozen TrOCR-based recognizer, AKİS. Our corpus comprises 102,000 tokens across four books spanning 1732–1920, with token error rates of 17.3%–34.8%. We evaluate a single-stage ByT5 byte-level encoder-decoder for direct correction, and three error detection methods, to test whether detection could guide correction in a two-stage approach. The corrector is trained on synthetically augmented OCR–reference pairs, with one variant weighting the training objective toward low-confidence sequences. Both ByT5 variants achieve approximately 9.6% CER on the test set, an 8.07–8.55% relative reduction from the 10.51% OCR baseline. An error-type decomposition of the 3,092 test-set errors shows that the model copies 85% of errors unchanged and corrects circumflex substitutions at 42%, the only subtype with clear net benefit. Confidence-weighted training does not improve corpus-level CER but reduces overcorrections by 19%. In the two-stage approach, calibrated confidence thresholding achieves token-level $F1 = 0.577$, BERT-based classification reaches $F1 = 0.640$, and confidence-fused BERT reaches $F1 = 0.647$ (beta-calibrated concatenation fusion; 0.643 with raw confidence). All three produce well-calibrated probabilities (ECE 0.022–0.067). A quality-estimation experiment is negative: detector scores do not predict whether a correction will help or harm a sequence. These findings show that OCR confidence is most useful as a training-time risk-control signal rather than as an inference-time selector.

ÖZET

OSMANLI TÜRKÇESİ TRANSKRİPSİYONU İÇİN OCR SONRASI METİN DÜZELTME

ALİZE SEVGİ YALÇINKAYA

Bilgisayar Bilimi ve Mühendisliği, Yüksek Lisans Tezi, Mayıs 2026

Tez Danışmanı: Prof. Dr. BERRİN YANIKOĞLU

İkinci Tez Danışmanı: Dr. Öğr. Üyesi DİLARA KEKÜLLÜOĞLU

Anahtar Kelimeler: ocr sonrası düzeltme, osmanlı türkçesi, güven kalibrasyonu, hata tespiti, düşük kaynaklı doğal dil işleme

Tarihi belgelerin sayısallaştırılması, özellikle modern anadil konuşurlarından ve dil araçlarından yoksun olan Osmanlı Türkçesi gibi düşük kaynaklı diller için etkili OCR sonrası hata düzeltme gerektirmektedir. Donmuş TrOCR tabanlı AKİS tanıyıcısıyla sayısallaştırılan Osmanlı Türkçesi metinleri için tamamen otomatik OCR sonrası işleme yöntemleri geliştirilip değerlendirilmektedir. Derlemimiz 1732–1920 yılları arasındaki dört kitaba ait 102.000 token içermekte olup token hata oranları %17,3–%34,8 aralığındadır. Doğrudan düzeltme için tek aşamalı ByT5 bayt düzeyinde kodlayıcı-kod çözücü ve tespit sinyallerinin iki aşamalı bir yaklaşımda düzeltmeye yön verip veremeyeceğini incelemek üzere üç hata tespiti yöntemi değerlendirilmektedir. Tek aşamalı düzeltici, sentetik olarak artırılmış OCR–referans çiftleriyle eğitilmekte; bir varyant eğitim hedefini düşük güvenli diziler lehine ağırlıklandırmaktadır. Her iki ByT5 varyantı da test kümesinde yaklaşık %9,6 CER elde etmekte, bu da %10,51 OCR temel değerinden %8,07–%8,55 görece azalmadır. Test kümesindeki 3.092 hatanın türlerine göre ayrıştırılması, modelin hataların %85’ini değiştirmeden kopyaladığını ve net fayda sağlanan tek alt tür olan inceltme işareti değişimlerini %42 oranında düzelttiğini göstermektedir. Güven ağırlıklı eğitim bütünsel CER’i iyileştirmemekte ancak aşırı düzeltmeleri %19 oranında azaltmaktadır. İki aşamalı yaklaşımda, kalibre edilmiş güven eşikleme token düzeyinde $F1 = 0,577$, BERT tabanlı sınıflandırma $F1 = 0,640$ ve güven entegre edilmiş BERT $F1 = 0,647$ değerine ulaşmaktadır (beta kalibreli birleştirme; ham güvenle 0,643). Üç yöntem de iyi kalibre edilmiş olasılıklar üretmektedir (ECE 0,022–0,067). Kalite kestirimi deneyi olumsuz sonuç vermektedir: tespit skorları, bir düzeltmenin diziye yarar mı zarar mı vereceğini öngörememektedir. OCR güveni, çıkarım zamanında bir seçici olmaktan çok eğitim zamanında bir risk kontrol sinyali olarak yararlıdır.

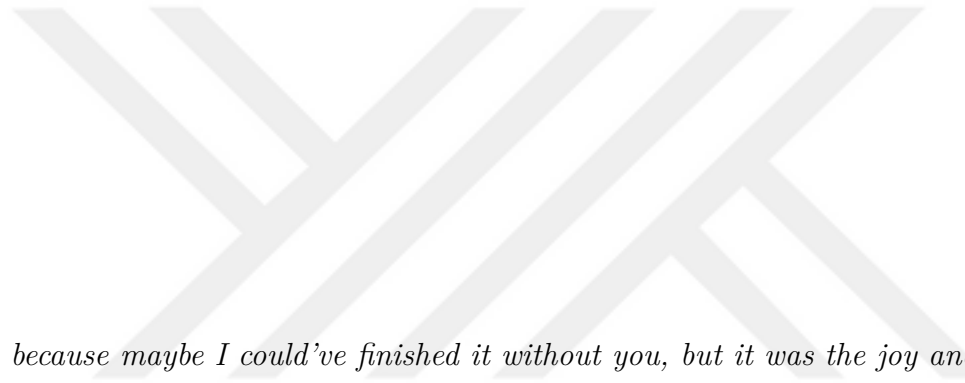
ACKNOWLEDGEMENTS

I want to thank my advisor, Prof. Berrin Yanıkoğlu, for her endless patience, perseverance against my stubbornness, and her inherent discipline. I also thank my co-advisor, Asst. Prof. Dilara Keküllüoğlu, for all her efforts and detailed feedback, but most of all, for making everything easier and more pleasant.

This thesis builds directly on AKİS, the Ottoman Turkish OCR system that produced every line and confidence score used in the experiments. I am grateful to the AKİS team for providing access to the trained recognizer, its curated corpus of transliterated lines, and the per-token confidence outputs that the methods in this thesis depend on. I particularly thank Umut Şen, who always returned my emails and never made me feel bad for asking for his help. AKİS itself was developed with TÜBİTAK support, and I am grateful to TÜBİTAK for the project scholarship (bursiyer) I held under Project 122E399 during part of my master's. Special thanks to jury member Asst. Prof. Esmâ Bilgin Taşdemir for her role in the AKİS project and for going the extra mile even after the project's completion. I also thank the other members of the thesis jury, Prof. Selim Balcısoy and Assoc. Prof. Öznur Taştan, for teaching me most things that laid the groundwork for this thesis, and for being lovely overall.

My family: my mother, for instilling and supporting my ambitions and for all the potatoes; my father, for all that he does incessantly and is willing to do; my brother, for being so loving and kind despite everything; and my sister, for her humour and witchcraft even though she is far away. I thank Müge, who makes life better for everyone but especially me, for being on my team since we were children and always remaining interesting and interested. I thank Sevil Helva for all her contributions to my graduation and for a whole lot of crepes.

And for Mako, who is eternally warm and missed, whom I thought about all the time while working.



*For Oğuz,
because maybe I could've finished it without you, but it was the joy and privilege of
my life doing it by your side.*

TABLE OF CONTENTS

ABSTRACT	iv
ÖZET	v
LIST OF TABLES	xi
LIST OF FIGURES	xii
LIST OF SYMBOLS	xiii
LIST OF ABBREVIATIONS	xv
1. INTRODUCTION	1
2. RELATED WORK	5
3. BASELINE SYSTEM (AKİS)	8
3.1. AKİS Overview	8
3.2. Corpus	9
3.3. Confidence Extraction	11
4. DATASET AND PREPROCESSING	14
4.1. Data Cleaning And Token-Confidence Realignment	14
4.2. Error Tagging And Class Distribution	16
4.3. Confidence Calibration	17
4.4. Corpus Statistics And Baseline Error Rates	20
4.5. Empirical Error Landscape	20
4.5.1. Subtype Frequency	23
4.5.2. Per-Book Distribution	24
4.5.3. Confidence $\times$ Subtype	25
4.5.4. Position And Span Length	26
4.6. Train/Dev/Test Split	27
5. EVALUATION FRAMEWORK	29
5.1. Detection Metrics	29
5.2. Correction Metrics	30

5.3. Statistical Reporting	31
6. SINGLE-STAGE CORRECTION: A BYT5 MODEL.....	32
6.1. Approach	32
6.2. Synthetic Data Generation	32
6.3. Experimental Settings & Data	37
6.4. Experiments & Results	38
6.5. Discussion	40
7. ERROR DETECTION METHODS	43
7.1. Overview	43
7.2. Method 1: Calibrated Confidence Thresholding.....	44
7.2.1. Approach.....	44
7.2.2. Experiments & Results.....	49
7.2.3. Discussion	50
7.3. Method 2: Error Detection With Contextualized BERT	51
7.3.1. Approach.....	51
7.3.2. Experimental Settings & Data	52
7.3.3. Experiments & Results.....	53
7.3.4. Discussion	53
7.4. Method 3: Error Detection With Confidence-Fused BERT	54
7.4.1. Approach.....	54
7.4.1.1. Variant A: Additive fusion	55
7.4.1.2. Variant B: Concatenation fusion.....	56
7.4.2. Experimental Settings & Data	57
7.4.3. Experiments & Results.....	57
7.4.4. Discussion	58
7.5. Comparative Analysis	59
8. CORRECTION ANALYSIS	63
8.1. Error-Type Correction Analysis	63
8.1.1. Substitution And Omission Patterns.....	67
8.1.2. Introduced Errors And Confidence Effects	68
8.1.3. Implications	69
8.2. Quality Estimation For Selective Correction	70

8.2.1. Experiments & Results.....	72
8.2.2. Discussion.....	73
8.3. Qualitative Comparative Error Analysis	74
8.4. Comparison With External Tools	76
8.4.1. Modern-Turkish Spell Checkers And Morphology	76
8.4.2. Cross-Recognizer Generalization (<i>Transkribus</i>)	78
8.4.3. Large Language Model (LLM)	80
9. SUMMARY	82
10. LIMITATIONS	84
11. CONCLUSION	88
BIBLIOGRAPHY.....	90
APPENDIX A - SINGLE-STAGE CORRECTION DETAILS	94
APPENDIX B - ERROR DETECTION DETAILS.....	97
APPENDIX C - CORRECTION ANALYSIS DETAILS	102
APPENDIX D - REPRODUCIBILITY	110
APPENDIX E - EXTERNAL-TOOL COMPARISON DETAILS	114

LIST OF TABLES

Table 3.1. Characteristics of the four books in the evaluation corpus (102,000 tokens, 8,294 sequences)	10
Table 4.1. Per-book baseline (AKIS) error rates on the 828-sequence test set. .	20
Table 4.2. Subtype taxonomy.	22
Table 4.3. Subtype frequency, corpus and test.....	23
Table 4.4. Span length distribution by subtype (corpus scope).....	27
Table 6.1. CER and WER, ByT5 variants	38
Table 6.2. Sequence-level correction outcomes, ByT5 variants	38
Table 7.1. Detection results, confidence thresholding.....	49
Table 7.2. Calibration and operating-point diagnostics for Method 1 variants.	50
Table 7.3. Standardized coefficients of the multi-feature logistic regression, ranked by absolute magnitude.	50
Table 7.4. Detection results, BERT-only baseline	53
Table 7.5. Calibration diagnostics for the BERT-only detector (Method 2). ...	53
Table 7.6. Token-level detection results, confidence-fused BERT	58
Table 7.7. Calibration diagnostics, confidence-fused BERT	58
Table 7.8. Sequence-level paired bootstrap on OR-aggregated detection	61
Table 7.9. Token-level paired bootstrap on detection F_1	61
Table 8.1. Character-level correction outcomes, substitution errors	66
Table 8.2. Character-level correction outcomes for omission errors by the un-weighted ByT5 model (1,072 error characters).....	67
Table 8.3. Quality estimation results for selective correction.	73
Table 8.4. Character- and word-error rates (raw and format-normalized) and sequence-level outcomes on the 828-line test set.	76
Table 8.5. <i>Transkribus</i> probe, per-page character error rate.	79
Table 8.6. Comparison on the 85 test lines of the four probe pages.	80

LIST OF FIGURES

Figure 1.1. Overview of the methods evaluated	4
Figure 3.1. Source material for the running example. (a) A page from Hayat-tan Sahifeler (1919). (b) Cropped line images from AKİS. (c) Ground-truth transliterations paired with OCR predictions.	8
Figure 3.2. The AKİS recognizer pipeline	9
Figure 3.3. Distribution of minimum token confidence across training sequences ($n = 6,614$).	13
Figure 4.1. Observed token error rates versus naïve and beta-calibrated predic-tions, binned by OCR confidence.	19
Figure 4.2. Subtype $\times$ book distribution of error characters across the corpus ($n = 28,198$). Cell colour encodes log count; per-book totals are 12,939 / 2,460 / 5,144 / 7,655. The numerical companion tables are in Appendix C (Tables C.1 and C.2 for corpus and test scope).	24
Figure 4.3. Per-subtype distribution of error characters across OCR confidence bins (corpus scope, $n = 28,198$). Each row sums to 100%; segment widths show the share of that subtype’s errors in each confidence bin. Confidence values for omission-style subtypes use sequence-mean fallback (see footnote).	25
Figure 6.1. Degradation rate by book for unweighted and confidence-weighted ByT5.	39
Figure 8.1. Gold-reference parse rate, circumflex vs plain tokens	78

LIST OF SYMBOLS

- β F-measure weighting parameter; also denotes the post-hoc beta-calibration variant of Method 1.
- $\mathbf{c}_t$ Token-level decoder confidence; the posterior probability assigned to token t by the OCR decoder. Defined in Chapter 3.
- Δ Mean character error rate (CER) reduction relative to the OCR baseline; positive values indicate improvement.
- $\Delta \mathbf{F}_1$ Change in $\mathbf{F}_1$ between two detector configurations; bootstrap-resampled for paired statistical inference.
- ℓ Sequence-level score of a decoded line: the sum of decoder log-probabilities over the non-padding, non-special token positions. Defined in Chapter 3.
- $\bar{\ell}$ Length-normalized sequence-level score; ℓ divided by the number of scored token positions, used when comparing lines of different lengths.
- $\mathbf{F}_1$ F-measure with $\beta = 1$ (harmonic mean of precision and recall); primary detection metric.
- $\mathbf{F}_2$ F-measure with $\beta = 2$ (recall-weighted variant); secondary detection metric for the recall-prioritising cost regime.
- $\mathcal{I}$ Index set of non-padding, non-special token positions in a decoded line, over which sequence-level scores are accumulated.
- L Length, in tokens, of a decoded OCR line $y_{1:L}$.
- λ Learnable scalar parameter governing the magnitude of the confidence-dependent perturbation in the additive-fusion method; clamped to $[0, 1]$.
- $\hat{p}$ Calibrated estimate of token error probability, output of the post-hoc calibration mapping (temperature scaling or beta calibration).

- p_t** Decoder posterior probability assigned to the emitted token at step t along the selected beam path; the token-level confidence is $c_t = p_t$. Defined in Chapter 3.
- T** Temperature parameter in temperature scaling, optimized to minimize negative log-likelihood on the development set.
- θ** Decision threshold on calibrated probability; fitted on the development set for the F_1 -optimal operating point of the error detector.



LIST OF ABBREVIATIONS

AdamW Adam optimizer with decoupled Weight decay.

AKİS TrOCR-based Ottoman Turkish recognizer used as the baseline (proper name).

AUC Area Under the Curve.

BERT Bidirectional Encoder Representations from Transformers.

BERTurk cased Turkish BERT model (dbmdz/bert-base-turkish-cased).

Bi-LSTM Bidirectional Long Short-Term Memory.

ByT5 Byte-level T5 (text-to-text transformer operating on raw UTF-8 bytes).

CER Character Error Rate.

CI Confidence Interval.

ConfBERT Confidence-enhanced BERT detector of Hemmer et al. (2024).

CTC Connectionist Temporal Classification.

CW Confidence-Weighted (training variant of ByT5).

ECE Expected Calibration Error.

EOS End-of-Sequence (decoder special token).

FN False Negative.

FP False Positive.

GPT Generative Pre-trained Transformer.

GroupKFold Group-aware K-fold cross-validation procedure.

GT Ground Truth.

L-BFGS-B Limited-memory Broyden-Fletcher-Goldfarb-Shanno algorithm with Box constraints.

LGB LightGBM (gradient-boosted decision tree library).

LLM Large Language Model.

LR Logistic Regression.

m-BERT Multilingual BERT.

M1 Method 1 (calibrated confidence thresholding detector).

M2 Method 2 (BERT-only contextual detector).

M3 Method 3 (confidence-fused BERT detector).

MLP Multilayer Perceptron.

NLP Natural Language Processing.

NMT Neural Machine Translation.

OCR Optical Character Recognition.

OOF Out-Of-Fold.

PR-AUC Precision-Recall Area Under the Curve.

QE Quality Estimation.

ROC-AUC Receiver Operating Characteristic Area Under the Curve.

TN True Negative.

TP True Positive.

TrOCR Transformer-based Optical Character Recognition (Microsoft, 2021).

UW Unweighted (training variant of ByT5).

ViT Vision Transformer.

WER Word Error Rate.

WordPiece subword tokenization scheme used by BERT.

1. INTRODUCTION

Digitization has broadened access to historical collections, but Optical Character Recognition (OCR) on such documents can still yield sizable character and word error rates (CER/WER, i.e., normalized edit distance at character/word level). Despite advances in deep-learning-based computer vision models, script and language-specific factors introduce systematic errors beyond generic vision errors. Retraining or adapting recognizers for historical collections is often impractical due to limited ground truth, model access constraints, and cost, especially for non-Latin or low-resource scripts (Akcan et al., 2022; Şen et al., 2025). Even with domain-specific OCR, scarce ground truth, script-specific rules, orthographic variability, and data scarcity necessitate a downstream post-processing stage.

Ottoman Turkish presents a particularly difficult case for post-OCR correction. The Ottoman alphabet contains 28 letters from Arabic and 5 letters from Persian, with right-to-left script, positional letter forms, and diacritics inherited from all three source languages (Persian, Arabic and Turkish). The language has no native speakers and limited digital resources, lacking tools such as grammar checkers, spell-checkers, or large labelled corpora that automatic correction methods in high-resource languages routinely depend on (Akcan et al., 2022; Şen et al., 2025). Character-level confusions specific to this script—circumflex substitution ($\hat{a} \leftrightarrow a$), dotted/dotless i substitution ($\dot{i} \leftrightarrow \ddot{i}$), and Turkish diacritic substitution ($\text{ç} \leftrightarrow c$, $\text{ş} \leftrightarrow s$)—alter word identity and meaning, yet arise from visual similarities that the recognizer cannot reliably distinguish. These properties motivate our focus on automatic post-OCR processing in a setting where neither retraining the recognizer nor relying on external linguistic resources is feasible.

Recent post-processing methods follow two paradigms. In the single-stage correction approach, a sequence-to-sequence model directly maps OCR output lines to their corrected counterparts. Such models struggle with non-erroneous spans that they may alter unnecessarily, and they can amplify inconsistencies in the supervised correction labels (i.e., errors and noise in the ground-truth correction targets). They also require substantial paired training data. In contrast, two-stage post-OCR systems first flag

likely errors (via model posteriors, language-model scores, or consistency checks) and then apply targeted edits. Here, missed detections leave errors unfixed, while false positives risk introducing new errors into previously correct text. The two paradigms differ not only in architecture but in what they reveal: single-stage models provide aggregate correction, while two-stage systems expose per-token error signals whose relationship to actual correction quality can be studied independently. Figure 1.1 summarizes the methods evaluated in this thesis.

OCR systems expose token and sequence-level confidence scores: internal probabilities produced at inference by common architectures (Connectionist Temporal Classification, CTC, and encoder–decoder transformers). However, these scores are often miscalibrated, meaning that a confidence of 0.9 does not correspond to a 90% probability of correctness. This limits their direct use for post-hoc detection and correction. Calibrating these scores and evaluating their use across detection, training-time weighting, and inference-time selective correction is a central concern of this work.

This thesis examines and compares fully automatic post-processing methods for transliterated Ottoman Turkish produced by a frozen TrOCR-based recognizer (AKİS). We evaluate single-stage sequence correction alongside the detection component of two-stage post-OCR systems, testing whether detection signals can decide when to apply the single-stage corrector at inference time, rather than building a dedicated second-stage correction module. The focus is on the practical reduction of residual recognition errors in a low-resource, class-imbalanced setting without modifying the recognizer. Our primary methods are fine-tuned encoder–decoder and encoder-only transformers rather than prompt-based large language models (LLMs); we additionally report a bounded zero-shot LLM pilot in Section 8.4.3. This emphasis reflects the finding that fine-tuned, task-specific models remain competitive with or superior to LLMs on low-resource scripts where large models lack sufficient pre-training exposure (Kanerva et al., 2025). Ottoman Turkish, with no native speakers and minimal digital presence, represents exactly such a setting. We also benchmark the corrector against external systems—two modern-Turkish toolkits, a second OCR recognizer, and a large language model—in Section 8.4.

We extract token-level confidence scores from the OCR decoder and apply post-hoc calibration so that they become comparable across texts. These calibrated confidences serve as a signal at multiple points: as the sole feature for a confidence-thresholding detector, as an input fused into BERT-based contextual detectors, as per-sequence weights during ByT5 training, and as candidate features for inference-time quality estimation. For single-stage correction, we evaluate a ByT5 character-level encoder–

decoder in both unweighted and confidence-weighted training variants. For error detection, we assess (i) calibrated confidence thresholding at token and character granularity, (ii) a contextual BERT classifier, and (iii) four confidence-fused BERT variants that integrate decoder confidence into BERT’s contextual representations.

Beyond aggregate metrics, we investigate which error types are actually corrected and whether detection signals can predict correction quality. An error-type decomposition of the test-set errors reveals that the corrector intervenes on only a small fraction of error characters, and that only one error subtype, circumflex substitutions, is corrected at a rate that makes targeted application clearly beneficial. A quality estimation experiment shows that detector-side scores do not predict, on this corpus, whether applying a correction will reduce or increase the sequence-level error rate, establishing that detection and correction address complementary aspects of post-OCR processing. Confidence-weighted training does not improve corpus-level CER but produces fewer overcorrections as shown in Chapter 6.

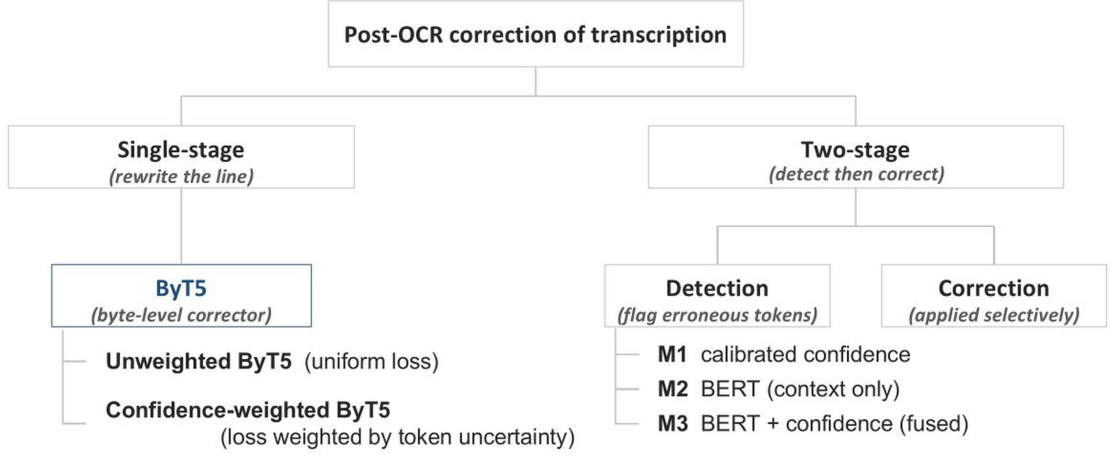


Figure 1.1 Overview of the post-OCR correction methods evaluated in this thesis: a single-stage byte-level corrector (unweighted and confidence-weighted ByT5) and a two-stage pipeline pairing token-level error detection (Methods 1–3) with selective correction.

The remainder of this thesis is organized as follows. Chapter 2 reviews related work on post-OCR processing. Chapter 3 introduces the baseline system, covering the AKIS recognizer, corpus characteristics, and confidence extraction. Chapter 4 describes the shared data pipeline (cleaning, error tagging, confidence calibration, corpus statistics, and the canonical train/dev/test split) used by all subsequent methods. Chapter 5 establishes our evaluation framework, defining metrics and statistical reporting procedures used throughout. Chapter 6 presents single-stage correction via a ByT5 character-level encoder–decoder, and Chapter 7 presents three error detection strategies (calibrated confidence thresholding, contextual BERT, and confidence-fused BERT). Chapter 8 analyzes correction outcomes by error type, evaluates detection-guided selective correction, traces detection and correction predictions on individual lines, and compares the corrector against external systems (two modern-Turkish toolkits, a second OCR recognizer, and a large language model). Chapter 9 summarizes the findings. Chapter 10 and Chapter 11 discuss limitations and conclude.

2. RELATED WORK

Post-OCR correction has been surveyed comprehensively by Nguyen et al. (2021), who classify approaches into isolated-word methods, context-dependent methods, and sequence-to-sequence models, and document that most work targets high-resource Latin-script languages. The ICDAR 2017 and 2019 shared tasks (Chiron et al., 2017; Rigaud et al., 2019) established standard benchmarks for error detection and correction, with the best-performing systems in 2019 using BERT (Devlin et al., 2019), a Transformer-based masked language model, fine-tuned separately for each subtask (Rigaud et al., 2019). Our work differs from these benchmarks in two respects: we operate on a non-Latin, low-resource script where no pre-existing detection or correction models exist, and we evaluate OCR confidence across multiple post-processing stages rather than treating detection and correction signals as entirely independent.

Sequence-to-sequence correction treats OCR output as noisy text and trains a model to “translate” it into clean text. Nguyen et al. (2020) combined a fine-tuned multilingual BERT (m-BERT) error detector with a character-level Neural Machine Translation (NMT) corrector initialized from BERT, achieving a 35% relative CER reduction on English and French historical books. Ramírez-Orta et al. (2022) corrected full documents by sliding a character-level transformer over n -gram windows and aggregating overlapping outputs via majority voting, reporting a 24% CER reduction on German newspapers. Both approaches require paired corpora substantially larger than what is available for Ottoman Turkish.

Byte-level models have been increasingly adopted for post-OCR correction because they avoid tokenizer-dependent failure modes. Xue et al. (2022) showed that a standard transformer operating on raw UTF-8 bytes (ByT5) matches or exceeds subword models on noise-sensitive tasks while handling any script without vocabulary modification. Löfgren and Dannélls (2024) fine-tuned ByT5 on Swedish historical newspapers, achieving a 36% CER reduction with a straightforward noisy-to-clean training setup. Guan and Greene (2024) compared synthetic data generation strategies for ByT5 across multiple languages, including several low-resource ones, and

found that glyph-similarity-based noise injection outperforms random character substitution. In follow-up work, Guan et al. (2024) introduced test-time adaptation for ByT5 and reported up to 68% CER reduction on the ICDAR 2019 benchmarks. Stankevičius and Lukoševičius (2022) applied ByT5 to simultaneous diacritics restoration and typo correction across 12 languages, a task closely related to our setting where circumflex substitution and Turkish diacritic errors are the dominant Turkish-specific error types. Our work extends this line by adding confidence-weighted training and difficulty-aware synthetic augmentation, and by decomposing correction outcomes at the error-subtype level.

Synthetic data strategies for OCR correction have focused primarily on the noise-injection side: generating artificial errors to expand training sets. Guan and Greene (2024) showed that glyph-similarity-based corruption outperforms random substitution, and Rijhwani et al. (2020) used rule-based noise models for endangered languages. A complementary approach is to vary the noise level of existing pairs instead of generating new corruptions. Curriculum learning, which orders training examples by increasing difficulty, has improved convergence in machine translation (Platanios et al., 2019) and other sequence tasks, though it has not been applied to post-OCR correction. Our synthetic augmentation takes a related but distinct approach: rather than ordering examples temporally, we produce multiple versions of each training pair at different noise levels by selectively removing subsets of errors, weighted by their estimated difficulty (Section 6.1).

Low-resource and non-Latin scripts pose particular difficulties. Rijhwani et al. (2020) trained a joint Bidirectional Long Short-Term Memory (Bi-LSTM) detector and sequence-to-sequence corrector with synthetic noise on five endangered languages, reporting an average 20% CER reduction from Tesseract output using fewer than one million tokens of ground truth. Their multilingual subword sharing relies on script similarities that do not hold for Ottoman’s Arabic-derived alphabet. More recently, Kanerva et al. (2025) evaluated open-weight LLMs (Llama, Poro, Gemma, Mistral) on historical English and Finnish post-OCR correction and found that LLMs reduced CER for English but failed to reach useful performance on Finnish, a morphologically complex language with limited LLM pre-training data. Their conclusion, that fine-tuned task-specific models remain competitive and that LLM gains do not transfer reliably to under-resourced languages, aligns with our decision to use fine-tuned ByT5 and BERT rather than LLM prompting, as discussed in Chapter 1. We report a bounded zero-shot pilot of a large language model on our own setting in Section 8.4.3; its results are consistent with this pattern.

OCR confidence scores are a built-in but underused signal. Most post-OCR detectors

ignore them entirely (Nguyen et al., 2021). Hemmer et al. (2024) proposed ConfBERT, which incorporates a scalar confidence into every token embedding via an additive perturbation, and demonstrated improved detection F_1 over a text-only BERT baseline on English and German documents. Our additive fusion variant (Section 7.4) follows their architecture. When confidence is used for detection, the raw scores must first be calibrated: Kull et al. (2017) derived beta calibration from first principles as a two-parameter alternative to Platt scaling that corrects asymmetric score distributions without losing rank ordering. Their restricted form is the calibration function we apply to the AKIS decoder’s token-level confidences (Eq. 4.2). To our knowledge, no prior work has evaluated calibrated OCR confidence across detection, training-time loss weighting, and inference-time quality estimation within a single post-processing study.

The question of whether detection can usefully guide correction has received limited direct attention. Quality estimation in machine translation predicts output quality without reference translations (Specia et al., 2018), but analogous work in post-OCR correction is scarce. Our quality estimation experiment (Section 8.2) tests whether token-level detection scores—from both confidence-based and BERT-based detectors—can predict whether applying a ByT5 correction will improve or degrade a given sequence. Similarly, while many correction studies report corpus-level CER, few decompose results by error type to identify which errors a model actually fixes. Our error-type analysis (Section 8.1) addresses this gap by classifying all 3,092 test-set error characters into thirteen outer subtypes (whole spans; Section 4.5) and ten inner subtypes (the individual character edits inside those spans; Section 8.1) and tracking their correction fates individually.

Comparison against non-neural correctors is reported in Section 8.4, against two openly available modern-Turkish toolkits — a spell checker and a deasciifier — and against a second OCR recognizer whose output we also post-correct. Modern-Turkish tools are an imperfect proxy for Ottoman: they treat much Arabic- and Persian-origin vocabulary as out-of-vocabulary. The comparison is informative because the mismatch quantifies what a domain-adapted corrector provides over a general-purpose one. Prior post-OCR work on under-resourced and historical languages (Löfgren & Dannélls, 2024; Rijhwani et al., 2020; Stankevičius & Lukoševičius, 2022) reports that fine-tuned neural sequence-to-sequence models outperform edit-distance and lexicon-lookup correctors when paired data is available. We do not build a bespoke Ottoman lexicon from this corpus, because a lexicon derived from the test data would not constitute an independent baseline.

3. BASELINE SYSTEM (AKİS)

3.1 AKİS Overview

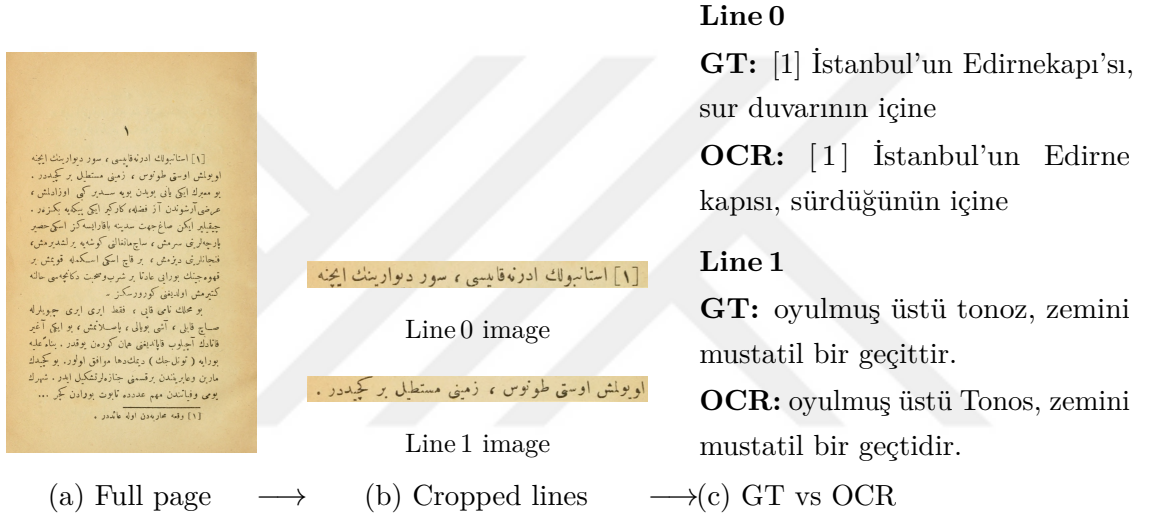


Figure 3.1 Source material for the running example. (a) A page from Hayattan Sahifeler (1919). (b) Cropped line images from AKİS. (c) Ground-truth transliterations paired with OCR predictions.

We use the AKİS variant of TrOCR (Li et al., 2023) as our baseline recognizer. AKİS is a transformer-based OCR system with a Vision Transformer encoder (Dosovitskiy et al., 2021) and a transformer decoder that takes an Ottoman line image and directly outputs its modern Turkish transcription in a single step. We use the terms *transcription* and *transliteration* interchangeably throughout this thesis for this Ottoman-script-to-Latin-script output. In this study, we use a fixed trained checkpoint at epoch 483 (4 encoder / 4 decoder layers) and we treat the recognizer as frozen. AKİS was used strictly for inference, and all improvements are made via post-OCR processing. Figure 3.2 shows the recognizer pipeline and the per-token confidence it produces.

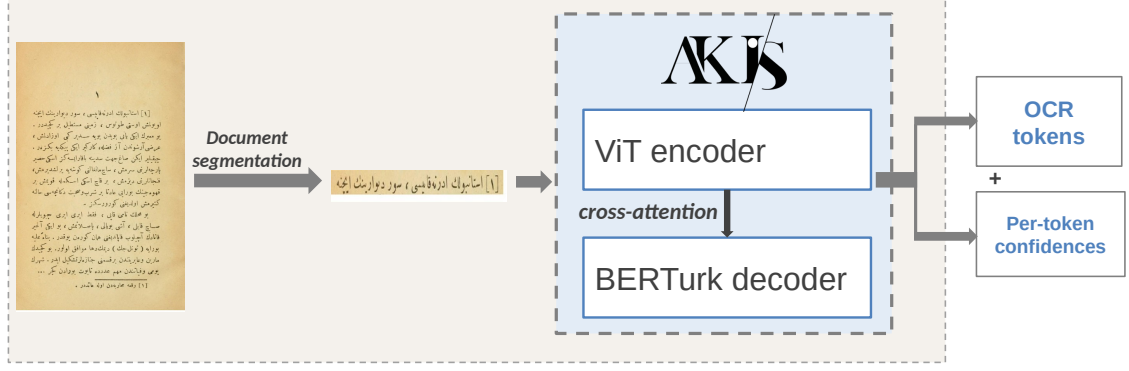


Figure 3.2 The AKİS recognizer pipeline. A ViT encoder and Transformer decoder transcribe each line image; the decoder also emits a per-token confidence score, which is the signal used for training-time weighting and for error detection throughout this thesis.

Throughout this thesis we trace a running example consisting of two consecutive lines from page 3 of *Hayattan Sahifeler* (Fig. 3.1); the OCR predictions on these two lines exhibit both correct and erroneous tokens and are revisited in subsequent sections to illustrate confidence behaviour, error tagging, calibration, and correction.

The TrOCR-base encoder-decoder architecture for text generation is used and paired with the cased Turkish BERT tokenizer to handle modern Turkish orthography better. The decoder’s start, end-of-sequence and padding markers are aligned to the tokenizer’s special tokens. Inputs are converted to luminance and replicated to three channels for the Vision Transformer (ViT) encoder; a horizontal mirror is applied during preprocessing. At inference, AKİS uses beam search (4 beams) with early stopping, a 3-gram repeat constraint, a length penalty of 2.0, and a maximum output of 250 tokens; other settings follow the reference implementation.

Our baseline loads TrOCR as the model variant even though it specializes in handwriting recognition: AKİS used this checkpoint for printed OCR because it handled connected Arabic script shapes and ligatures more robustly. We report quality using AKİS’s metrics: CER (Character Error Rate) and WER (Word Error Rate), defined in Section 5.2.

3.2 Corpus

Our dataset comprises the held-out books of the AKİS OCR corpus: 8,294 sequences from four Ottoman Turkish books. In chronological order these are: *Usûlü’l-Hikem fi*

Nizâmi'l-Ümem (“Principles of Wisdom in the Order of Nations”, 1732) by İbrahim Müteferrika (c. 1674–1745); *Arnavutlar Solyotlar: Tarihe Müstenit Bir Roman* (“Albanians and Souliotes: A Novel Based on History”, 1888) by Ahmet Mithat Efendi (1844–1912); *Hayattan Sahifeler* (“Pages from Life”, 1919) by Hüseyin Rahmi Gürpınar (1864–1944); and *Tarihçe-i Erzurum yâhud Hemşehrîlere Armağan* (“A Brief History of Erzurum, or A Gift to Fellow Townsmen”, 1920) by Mehmed Nusret (Erzurumlu). We refer to them throughout by the short forms *Ususül Hikem*, *Arnavutlar Solyotlar*, *Hayattan Sahifeler*, and *Tarihçe-i Erzurum*, respectively. We use the term *sequence* throughout to refer to a single OCR line as produced by AKİS, i.e., one row of input–output text in our corpus; sequences correspond to printed lines on the source page rather than to grammatical sentences. These printed texts (matbu, naskh script) span 1732–1920. They were never seen during AKİS recognizer training: in the broader AKİS corpus of 13 books (80,000+ lines) with its own train/dev/test splits, these four evaluation books appear only in the validation and test partitions, which makes them an independent basis for evaluating post-OCR methods. Within these four books we then construct our own page-stratified post-OCR train/dev/test split (Chapter 4); no sequence used to fit any post-OCR parameter appears in our post-OCR test partition.

The four books have substantial variation in publication era (188 years), genre (political treatise, novels, history), and OCR difficulty. Error rates range 2-fold from 17.3% to 34.8% (Table 3.1), with the oldest text (1732) and the history volume (1920) showing the highest rates due to connected letterforms, position-dependent glyph shapes, and variable print quality that complicate recognition. This domain heterogeneity motivates a page-level splitting strategy in which sequences from the same source page are assigned to the same train/dev/test partition. This prevents within-page leakage, preserves book proportions across partitions, and allows us to assess whether methods generalize across domains or exploit book-specific patterns. All metrics are reported both per-book and as overall test-set averages.

Table 3.1 Characteristics of the four books in the evaluation corpus (102,000 tokens, 8,294 sequences)

Book	Tokens	Error Rate	Genre	Year
Ususül Hikem	19,588	34.8%	political treatise	1732
Arnavutlar Solyotlar	54,419	22.4%	novel	1888
Hayattan Sahifeler	12,412	17.3%	novel	1919
Tarihçe-i Erzurum	15,581	32.8%	history	1920
Total	102,000	25.7%		

All methods operate on cleaned text with aligned confidence scores; the cleaning and alignment pipeline is described in Chapter 4. Detailed corpus statistics and baseline error rates are reported in Section 4.4.

3.3 Confidence Extraction

In our confidence-aware post-processing setup, the first step is to estimate token confidences from the OCR decoder. We decode with beam search, compute per-step log-probabilities for the chosen prediction path, exponentiate to obtain per-token confidences, and retain the token identifiers plus a sequence-level score computed from the sum of log-probabilities over non-padding steps.

Let $y_{1:L}$ be the decoded prediction for a line of length L . At each step t , the decoder assigns a posterior p_t to the emitted token y_t along the selected beam path. We define the token-level confidence as $c_t = p_t$ and the sequence-level score as

$$\ell = \sum_{t \in \mathcal{I}} \log p_t, \quad \bar{\ell} = \frac{1}{|\mathcal{I}|} \sum_{t \in \mathcal{I}} \log p_t,$$

where $\mathcal{I}$ indexes non-padding, non-special tokens; $\bar{\ell}$ is the length-normalized variant used when comparing lines of different lengths. We stop accumulation at the end-of-sequence token (EOS) and exclude non-lexical specials (e.g., padding and control markers). Confidence extraction uses the same tokenizer as the recognizer to ensure consistent subword segmentation.

We use $\{c_t\}$ as a primary signal for error detection. The specific detectors and acceptance rules are detailed in Chapter 7.

To illustrate how raw token confidences behave in practice, we return to the two lines from Fig. 3.1 where the OCR prediction contains both correct and erroneous tokens. In each case, we highlight the model’s token confidences for correct and incorrect spans. These examples motivate calibrating confidences and combining them with additional signals instead of relying on raw scores alone. Two representative examples from the *Hayattan Sahifeler* volume are shown in Example 1 and Example 2 (digits are shortened for legibility):

Example 1 (Confidence behaviour on line 0, *Hayattan Sahifeler*, page 3).

Reference:	[1] İstanbul’un Edirnekapı’sı, sur duvarının içine
Prediction:	[1] İstanbul’un Edirne kapısı, sürdüğünün içine
Prediction Tokens:	[, 1,], İstanbul, [,], un, Edirne, kapısı, [,], , sürdüğünü, ##n, içine
Token Confidences:	1.0, 1.0, 1.0, 0.9872, 1.0, 1.0, 0.9997, 0.8207 , 0.9999, 0.3339 , 1.0 , 0.9999

Example 2 (Confidence behaviour on line 1, *Hayattan Sahifeler*, page 3).

Reference:	oyulmuş üstü tonoz, zemini mustatıl bir geçittir.
Prediction:	oyulmuş üstü Tonos, zemini mustatıl bir geçtidir.
Prediction Tokens:	oyu, ##lm, ##uş, üstü, Ton, ##os, [,], zemini, must, ##ati, ##l, bir, geçti, ##dir, [,]
Token Confidences:	0.7698, 0.9999, 0.9999, 0.9999, 0.6619 , 0.9263 , 1.0, 0.9996, 0.9324, 0.9758, 0.9999, 1.0, 0.8026 , 0.9988 , 1.0

Several observations emerge from these examples. In Example 1, the correct token "İstanbul" receives high confidence (0.9872), while the erroneous word "sürdüğünün"—tokenized into *sürdüğünü* + *##n*—shows the low-confidence signal on the main subword (*sürdüğünü* with 0.3339) whereas the continuation *##n* inherits ceiling confidence (1.0); this subword-level pattern is examined in detail in Example 7. In Example 2, the error "Tonos" shows moderate confidence on the first piece (0.6619) but high confidence on the continuation (0.9263). These patterns demonstrate that decoder confidence provides a noisy but informative signal: many errors exhibit reduced confidence, but some incorrect predictions receive high scores, and some correct tokens show moderate confidence. The training-set distribution of minimum token confidence per sequence (Fig. 3.3) confirms this variability: the distribution is broadly spread, with approximately one-third of sequences falling below 0.5. This motivates calibration to improve the reliability of confidence as an error indicator. These confidences are retained across the subsequent processing stages: cleaning, error tagging, calibration, and correction analysis.

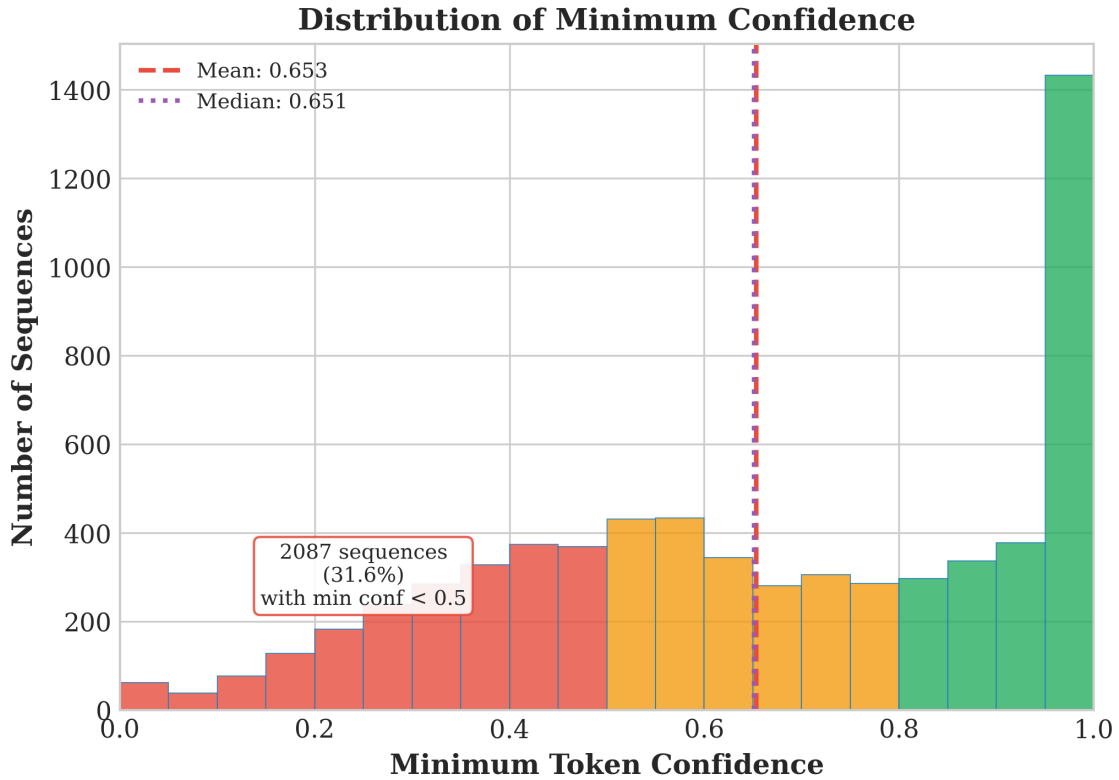


Figure 3.3 Distribution of minimum token confidence across training sequences ($n = 6,614$).

4. DATASET AND PREPROCESSING

A fair comparison of correction methods requires that each method operates on identical input. All methods evaluated in this thesis share a single data pipeline derived from the AKIS OCR outputs described in Chapter 3. This preprocessing workflow takes raw OCR outputs and transforms them into clean, tagged, calibrated tokens ready for detection and correction experiments. This workflow is laid out across six sections. Sections 4.1–4.4 cover the technical pipeline (cleaning, tagging, calibration, baseline rates) that every downstream method depends on. Section 4.5 then steps back and presents the empirical landscape underneath the baseline aggregates: which error subtypes dominate, how they distribute across books, and how they align with OCR confidence. The design choices in Chapters 6–8 (synthetic-data weighting, detection-feature fusion, selective correction) are easier to justify against this explicit empirical picture than against aggregate rates alone. Section 4.6 closes with the canonical train/dev/test split used by every experiment in the thesis.

4.1 Data Cleaning And Token-Confidence Realignment

OCR output often contains surface noise such as inconsistent punctuation, encoding artifacts and minor normalization issues that are not meaningful for evaluation, but can create spurious token differences and distort error statistics. Cleaning must remove this noise without disturbing the signal we rely on later: the decoder’s token-level confidences must remain comparable across tokens and lines. We therefore normalize surface inconsistencies while preserving lexical content and the attached confidence scores, so that downstream methods see differences that reflect actual OCR failures instead of formatting artifacts.

Each OCR line consists of subword tokens with associated confidence scores. Preprocessing transforms these into normalized tokens with preserved confidences, ready for binary error tagging. Cleaning removes 328 of the original 8,622 OCR sequences

(3.8%), primarily citation prefixes and encoding artifacts, producing the canonical dataset of 8,294 sequences used in all subsequent experiments.

WordPiece subword markers (##) are preserved in all tokens throughout preprocessing. These markers distinguish word-initial tokens from continuation tokens (e.g., "İstanbul" vs. "##un"), which is structural information that detection methods can exploit. The markers are only removed temporarily during alignment when we check whether two differently-tokenized sequences represent the same underlying text, a check referred to as "post-join equality" throughout. The normalization used to create split tokens is identical to the policy used for alignment, post-join equality, and scoring for both prediction and reference. This ensures that all operations are defined on a single, coherent tokenization scheme. Across the corpus, every sequence remains well within the 512-position input limit used by BERT, so the WordPiece grid is consumed end-to-end without truncation by any downstream detection method.

To illustrate the effect of cleaning on real OCR output, we concatenate two consecutive lines from the earlier examples (Hayattan Sahifeler, page 3, lines 1–2) into a single example, shown in Example 3.

Example 3 (Surface-noise cleaning on lines 0–1). *Line 0: the citation prefix is removed from both prediction and reference. Line 1 passes through unchanged. Bold marks removed text.*

Line 0	
RawPred	[1] İstanbul'un Edirne kapısı, sürdüğünün içine
Cleaned	İstanbul'un Edirne kapısı, sürdüğünün içine
Changes	Citation prefix [1] deleted
Line 1	
RawPred	oyulmuş üstü Tonos, zemini mustatil bir geçtidir.
Cleaned	oyulmuş üstü Tonos, zemini mustatil bir geçtidir.
Changes	None (no surface artifacts)

The cleaning step is selective: line 0's citation marker (a common artifact of page-initial lines) is removed because it is not part of the literary text, while line 1 passes through unchanged because it contains no surface noise. The OCR errors in both lines ("Edirne kapısı" for "Edirnekapı'sı" in line 0; "Tonos" for "tonoz" in line 1) are preserved, because these are genuine recognition errors to be handled by downstream detection and correction, not formatting artifacts. The cleaned tokens, with their

associated confidence scores intact, proceed to error tagging (Section 4.2).

4.2 Error Tagging And Class Distribution

Because cleaning can alter tokenization boundaries, error tags must be derived on the cleaned tokens: we must determine which cleaned prediction tokens are correct and which contain errors. We align cleaned prediction tokens to cleaned reference tokens using sequence matching that identifies corresponding spans and operations. The alignment groups tokens into spans labelled as equal (tokens match), replace (tokens differ), insert (prediction has extra tokens), or delete (prediction missing tokens). This span-level (outer) alignment serves as the basis for token-level error tagging.

The alignment uses the Ratcliff/Obershelp longest-common-subsequence algorithm, which identifies matching blocks greedily and produces a minimal sequence of edit operations. At the token level used here, the algorithm operates on token lists; for the character-level error-type analysis in Section 8.1, it operates on character lists. When the token-level alignment produces multi-character replace spans where several characters on both sides differ simultaneously, a secondary alignment pass is applied within each span to decompose it into individual character-level substitutions, insertions, and deletions. This inner alignment recovers error pairs such as circumflex substitutions that would otherwise be hidden inside longer multi-character groups.

Each token receives a binary error tag (1 = error, 0 = correct) based on its alignment span and post-join equality result. This creates a class-imbalanced token classification task: 25.7% of tokens contain errors overall, though rates vary substantially (17.3%-34.8%) across books. The error tags represent character-level content differences after normalizing apostrophes/hyphens and accounting for benign tokenization variation. These labels serve as supervision for detection models and ground truth for evaluating detection performance; together with the confidence scores and cleaned tokens, they form the shared input to all detection methods. This aggregate token-error rate is the high-level class-imbalance figure detection methods need to know up front; Section 4.5 unpacks what kinds of errors make up this 25.7% and how they are distributed across the corpus.

To illustrate the alignment and tagging process, Example 4 shows the token-level error tags produced for line 0 of the running example (Fig. 3.1) after cleaning (Example 3).

Example 4 (Token-level error tagging for line 0). *Cleaned tokens from Example 3 are aligned to reference tokens; three of nine prediction tokens are tagged as errors.*

Pred Token(s)	Ref Token(s)	Tag(s)
İstanbul	İstanbul	0
,	,	0
un	un	0
Edirne	Edirne	0
kapısı	##kapı, ', sı	1
,	,	0
sürdüğünü, ##n	sur, duvarı, ##nın	1,1
içine	içine	0
Error rate: 3/9 pred tokens = 33.3%		

The span-based view reveals how tokenization boundaries interact with error structure. In span 5, the OCR output “Edirne kapısı” corresponds to the reference compound “Edirnekapı’sı”: the recognizer split one word into two tokens, and the reference further decomposes into three subwords (**##kapı, ', sı**), of which **##kapı** and **'** have no prediction counterpart (insert operations). Only the prediction-side token **kapısı** receives an error tag. In span 7, the entirely incorrect “sürdüğünün” (2 tokens) replaces “sur duvarının” (3 tokens); the insert token **sur** is again visible only on the reference side. These error tags serve as ground truth for all detection methods: Method 1 (calibrated confidence thresholding) uses them to fit calibration parameters (Example 7), and Methods 2 (BERT-only contextual classifier) and 3 (confidence-fused BERT) use them as supervision labels for fine-tuning. The precise token-level labelling rule is given in Appendix B.

4.3 Confidence Calibration

Raw OCR confidence scores are systematically miscalibrated relative to their underlying error rates, but the direction of the miscalibration depends on the granularity at which we evaluate it. At the sequence level (Fig. 4.1), high-confidence predictions are

markedly less reliable than the raw scores suggest: at minimum-token-confidence ≥ 0.9 the observed sequence-level error rate (the fraction of sequences containing at least one erroneous character) is 40.0%, far above the naive $1 - c = 5.0\%$ that the raw scores would predict. This sequence-level inflation is partly an aggregation artifact (a single stray character flips the indicator), but its magnitude exceeds what aggregation alone would explain. At the character level the picture is reversed: the mean per-sequence character error rate in the same high-confidence bin is approximately 3.2%, modestly below the naive predictor. Both readings are consistent: high-confidence predictions copy most characters correctly, but the remaining character errors concentrate enough that few sequences are entirely error-free. This direction-flipped pattern is a corpus-wide aggregate; Section 4.5.3 disaggregates the remaining high-confidence character errors by subtype and shows that the concentration is uneven. Correcting either distortion requires a calibration function that maps raw confidences to probabilities aligned with empirical rates. We evaluate two such functions, chosen because they represent the standard minimal and flexible options in the calibration literature.

Temperature scaling uses a single parameter to uniformly rescale all scores, offering the smallest possible intervention. It divides the logit z by a scalar $T > 0$ before applying the softmax:

$$\hat{p} = \sigma(z/T), \quad (4.1)$$

where T is optimized to minimize negative log-likelihood on the development set. A single parameter controls the sharpness of the predictive distribution without changing the ranking of predictions.

Beta calibration uses two parameters, allowing it to correct over-confident and under-confident regions independently. This matters because the AKIS decoder’s confidence distribution is asymmetrically distorted, with most tokens clustered near 1.0 and a long tail of uncertain cases (Fig. 4.1).

We implement a restricted form of beta calibration (Kull et al., 2017) that applies Platt scaling to the logit of the error proxy $(1 - s)$:

$$\hat{p} = \sigma(a \cdot \text{logit}(1 - s) + b), \quad (4.2)$$

where $s \equiv c_t$ is the raw confidence score (Section 3.3), $\text{logit}(x) = \log \frac{x}{1-x}$, σ is the sigmoid function, and a (scaling) and b (bias) are fitted on training data by minimizing negative log-likelihood via L-BFGS-B. This is equivalent to the general three-parameter beta calibration of Kull et al. with the constraint that the coefficients on $\log s$ and $\log(1 - s)$ are equal in magnitude but opposite in sign; the

restriction reduces the model to two free parameters while retaining the ability to correct asymmetric distortions.

Both methods preserve the rank ordering of scores when $a > 0$ and $T > 0$: if token A has higher raw confidence than token B, it retains higher calibrated confidence. Rank preservation ensures that calibration corrects the probability scale without changing which tokens are considered more or less uncertain. Both methods are fitted by minimizing negative log-likelihood on the training split via L-BFGS-B. Evaluating both allows us to determine whether the additional flexibility of beta calibration translates into meaningful improvement, or whether the simpler single-parameter temperature scaling suffices. Fig. 4.1 illustrates the effect on our data: observed error rates substantially exceed the naïve 1–confidence baseline at every confidence level, and beta calibration tracks the empirical rates more closely (per-partition calibration bins in Appendix B, Tables B.7–B.9).

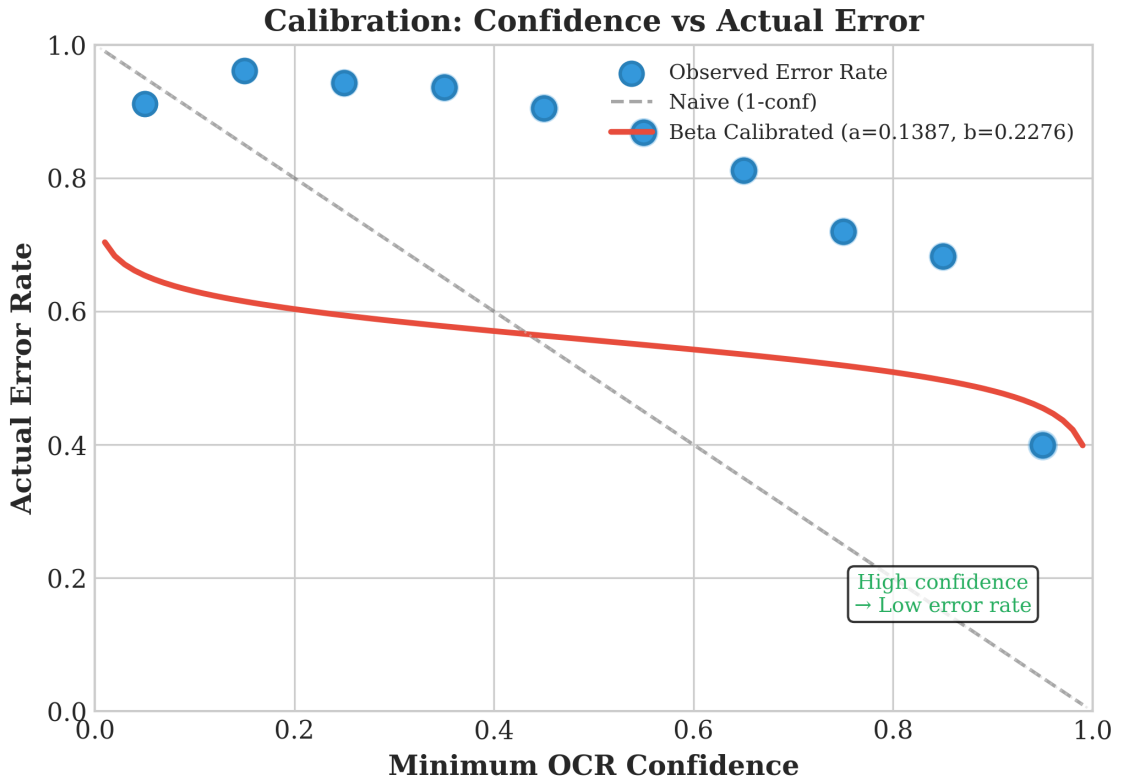


Figure 4.1 Observed token error rates versus naïve and beta-calibrated predictions, binned by OCR confidence.

4.4 Corpus Statistics And Baseline Error Rates

The corpus comprises 8,294 original sequences containing 102,000 tokens and 368,364 characters (average 44.41 characters per sequence) from four Ottoman Turkish historical texts spanning 1732–1920. Table 4.1 breaks down baseline error rates by book on the 828-sequence test set. Per-book rates here differ slightly from the corpus-level rates in Table 3.1 because the test partition is a page-stratified subset of each book rather than a proportional sample. The oldest text (1732) exhibits more than three times the CER of the cleanest text (Hayattan Sahifeler, 1919), reflecting differences in print quality, script complexity, and recognizer performance across historical periods. Character-level error analysis reveals 8.64% of characters are tagged as errors, with 75.75% of sequences containing at least one error. Per-book CER is the headline aggregate but does not report on composition: two books with the same CER could carry entirely different error profiles, with very different implications for post-correction. Section 4.5 disaggregates by subtype, book, and confidence to make these compositional differences visible.

Table 4.1 Per-book baseline (AKİS) error rates on the 828-sequence test set.

Book	Seq	CER	WER	Token Err.
Ususül Hikem	86	19.95%	52.85%	34.4%
Arnavutlar Solyotlar	555	9.53%	30.62%	26.2%
Hayattan Sahifeler	112	6.06%	22.42%	14.9%
Tarihçe-i Erzurum	75	13.51%	44.79%	34.1%
Overall	828	10.51%	33.10%	27.4%

4.5 Empirical Error Landscape

Per-book CERs (Section 4.4) report how often AKİS errs on each text but say nothing about what kinds of errors are involved. Chapters 6 and 7 need this finer picture: the synthetic-data weighting in Section 6.2 and the confidence-fused detection in Section 7.4 both depend on which error subtypes are common, how they distribute across books, and how they align with OCR confidence.

This section provides the finer picture. We disaggregate the corpus error population at the character level along three axes: subtype, book, and OCR confidence.

Two views of the same error characters recur in this thesis. The outer view aligns each OCR error against the reference as a single span (the outer alignment of Section 4.2) and labels it with one of thirteen outer subtypes, leaving multi-character corruptions intact and binning them by length. The inner view runs a second alignment inside each multi-character span (the inner alignment of Section 4.2) to recover the individual character edits it contains, producing ten inner subtypes. The outer and inner views coincide on single-character errors and differ only on multi-character spans: the outer view keeps the whole span as one unit, whereas the inner view re-aligns inside it to label each character edit separately. (The token-level rate of Section 4.2 only locates which words are wrong; both views describe the character errors inside those words.) This section reports the outer view (Table 4.2); the per-character fix-rate analysis in Section 8.1 reports the inner view (Tables 8.1 and 8.2).

Counts are reported in parallel for the full 8,294-sequence corpus (28,198 error characters) and for the 828-sequence test partition (3,092 error characters). Corpus-wide counts include the test partition; this is descriptive reporting only and does not affect any quantity used for model selection or evaluation in Chapter 6 or 7 (see Section 4.6). Position-within-sequence and span-length distributions are reported at the end of the section as secondary findings.

The 25.7% token-error rate of Section 4.2 averages over very different error families: single Turkish-specific character confusions like â ↔ a alongside multi-character corruption where OCR has produced several wrong characters in a row. We distinguish twelve such families that matter for post-correction. Two of them (Turkish diacritic and Punctuation) combine substitution and omission operations at the outer level, and three (short, medium, and long multi-character substitution) are spans of several character edits binned by length.

A thirteenth row holds the unresolvable spans: multi-character spans where the OCR and reference sides differ in length, so the inner alignment cannot map them character by character and cannot identify which individual edits occurred (Example 8 shows one from each book). We keep these separate for two reasons: counting them as substitutions would inflate the substitution totals, and dropping them would remove those positions from the analysis. Table 4.2 lists all thirteen rows with descriptions and examples. Section 8.1 re-runs the inner alignment to separate substitution from omission for the other twelve families; the unresolvable spans are the residue where that re-decomposition cannot apply.

Table 4.2 Subtype taxonomy. Twelve decomposable subtypes plus an unresolvable-span row; Turkish diacritic and Punctuation combine substitution and omission operations at the outer level (see Section 8.1).

Subtype	Description	Example (OCR → GT)
<i>Turkish-specific substitution</i>		
Circumflex substitution	OCR confuses a plain vowel (a/i/u) with its circumflex counterpart (â/î/û), or vice versa.	kaza → kazâ
Dotted/dotless i substitution	OCR confuses dotted i and dotless ı.	müte'allik → müte'allık
Turkish diacritic	OCR misses or substitutes one of the Turkish diacritics ç, ş, ğ, ö, ü.	gecti → geçti
Vowel substitution	OCR substitutes one vowel for a different vowel.	silahşörluktan → silahşörlükten
<i>Other single-character substitution</i>		
Other character substitution	Single-character substitution not covered by the four Turkish-specific subtypes above.	kitab → kitap
<i>Multi-character substitution and omission</i>		
Short multi-character substitution (2–3)	Contiguous 2–3 character substitution whose inner alignment decomposes into discrete edits.	adedini akde → hedesini akde
Medium multi-character substitution (4–6)	Contiguous 4–6 character substitution that inner alignment decomposes.	Papaz mı tutuştu?.. → paçası mı tutuştu?..
Long multi-character substitution (7+)	Contiguous 7+ character substitution that inner alignment decomposes.	masrûğdâd ve bölük-i Rûm → Mısır ve Bağdad...
Character omission	Ground truth contains characters that OCR did not produce.	müstağni 'ani'l-beyâtdır → müstağniyyun 'ani'l-beyândır
<i>Formatting</i>		
Case substitution	OCR differs from ground truth only in capitalisation.	Devlet-i Aliye'nin → devlet-i aliyenin
Punctuation	Punctuation added, missing, or substituted by OCR.	Katerinanın → Katerina'nın
Spacing omission	Whitespace missing in OCR output.	Hazret-i Vâlide bu → Hazreti Vâli de bu
<i>Unresolvable</i>		
Unresolvable span	Multi-character span where OCR and reference differ in length, so inner alignment cannot map them character by character (see Example 8, Section 8.1).	küffâr-ı ârâm bu cevâha... → küffâr ârâm bulup ve alay...

4.5.1 Subtype Frequency

Table 4.3 Subtype frequency in the corpus and test partition, grouped by family; the Rank column gives each decomposable subtype’s position by corpus count.

Subtype	Corpus N	Corpus %	Test N	Test %	Rank
<i>Turkish-specific substitution</i>					
Circumflex substitution	1,694	6.9	218	8.2	5
Dotted/dotless i substitution	547	2.2	60	2.3	10
Vowel substitution	1,305	5.3	118	4.4	8
Turkish diacritic	2,486	10.2	226	8.5	4
<i>Other single-character substitution</i>					
Other character substitution	3,116	12.7	349	13.1	3
<i>Multi-character substitution and omission</i>					
Short multi-character substitution (2–3)	6,338	25.9	645	24.3	1
Medium multi-character substitution (4–6)	1,562	6.4	232	8.7	7
Long multi-character substitution (7+)	543	2.2	59	2.2	11
Character omission	3,992	16.3	450	16.9	2
<i>Formatting</i>					
Case substitution	1,582	6.5	166	6.2	6
Punctuation	954	3.9	97	3.7	9
Spacing omission	361	1.5	34	1.3	12
Subtotal (12 decomposed subtypes)	24,480	100.0	2,654	100.0	—

Unresolvable multi-character spans. A further 3,718 error characters (13.2% of the corpus) sit inside multi-character spans where the OCR and reference sides differ in length, so inner alignment cannot map them character by character. We tally these separately because they do not belong to any one of the twelve subtypes above and would distort per-subtype fix-rate analysis (Section 8.1) if folded into the short, medium, or long multi-character substitution rows.

Table 4.3 anchors three readings used throughout the rest of the thesis. First, half the corpus error mass lives in multi-character corruption and omissions: error families where the model must reconstruct material it did not see, not just correct a single visible character. Second, Turkish-specific substitutions (circumflex, dotted/dotless i, Turkish diacritic, vowel) account for roughly a quarter of the corpus, distributed across ranks 4, 5, 8, and 10. They are substantial but not dominant. Third, formatting errors (case, punctuation, spacing) are the long tail rather than the headline, occupying ranks 6, 9, and 12.

The test partition reproduces the corpus error distribution faithfully: ten of twelve subtype ranks match between corpus and test, the two exceptions are one-step swaps among adjacent ranks, and Spearman rank correlation is 0.97. This is the empirical basis for treating test-set findings in later chapters as representative of the corpus rather than as artifacts of a particular page sample. Two caveats apply. First, the test partition is page-stratified by book (Section 4.6), so rank stability between corpus and test does not imply rank stability between train and test under arbitrary re-splits. Second, the unresolvable-span row in the callout is reported against the full 28,198-character denominator rather than the 24,480-character decomposed subtotal, so its rank reflects total volume rather than position among decomposable subtypes.

4.5.2 Per-Book Distribution

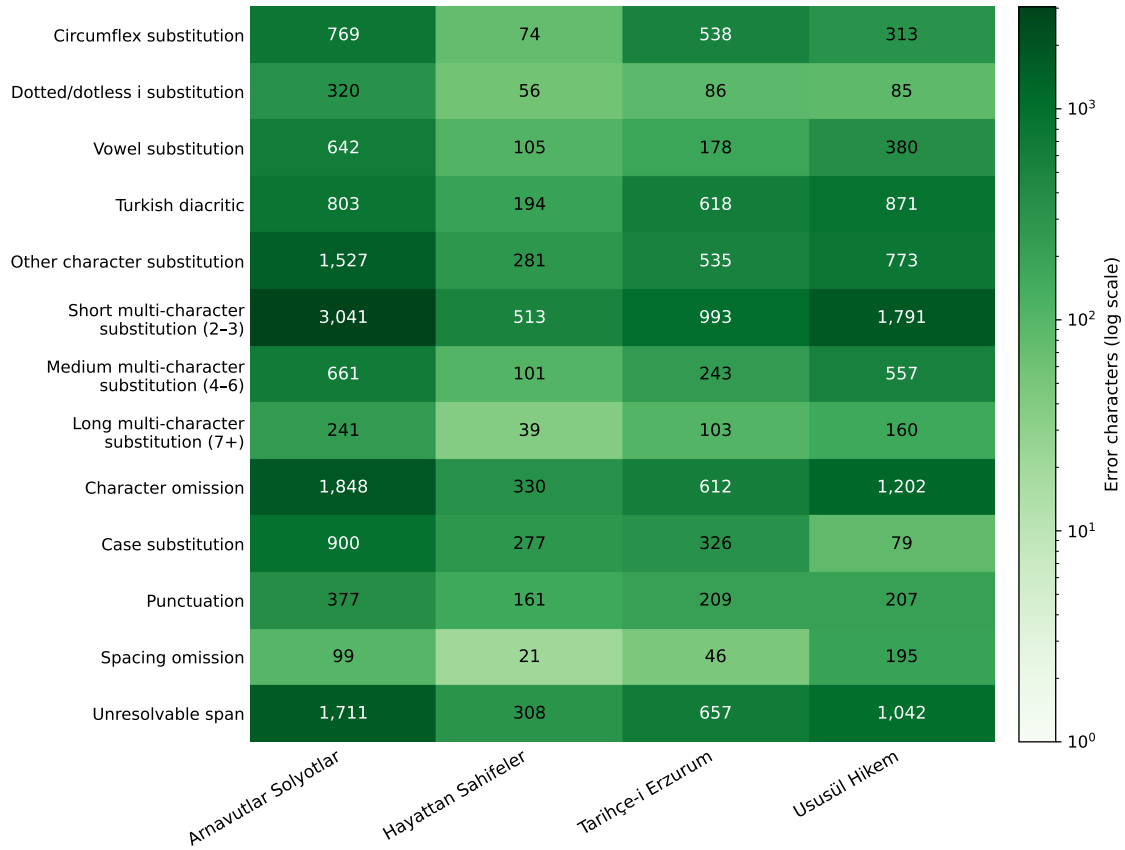


Figure 4.2 Subtype $\times$ book distribution of error characters across the corpus ($n = 28,198$). Cell colour encodes log count; per-book totals are 12,939 / 2,460 / 5,144 / 7,655. The numerical companion tables are in Appendix C (Tables C.1 and C.2 for corpus and test scope).

Figure 4.2 breaks the per-book error mass down by subtype, showing that composition—not just volume—differs qualitatively across the four books. Ar-

navutlar Solyotlar concentrates the corpus’s circumflex and case errors (45% and 57% of those totals respectively). Ususül Hikem has the highest CER but a different profile, dominated by Turkish diacritic errors and multi-character corruption rather than case. Tarihçe-i Erzurum sits between the two in volume but shares Ususül’s bias toward circumflex and Turkish diacritic. Hayattan Sahifeler is small but proportionally heavier on case. No single global per-subtype weighting can fit all four books equally well.

4.5.3 Confidence × Subtype

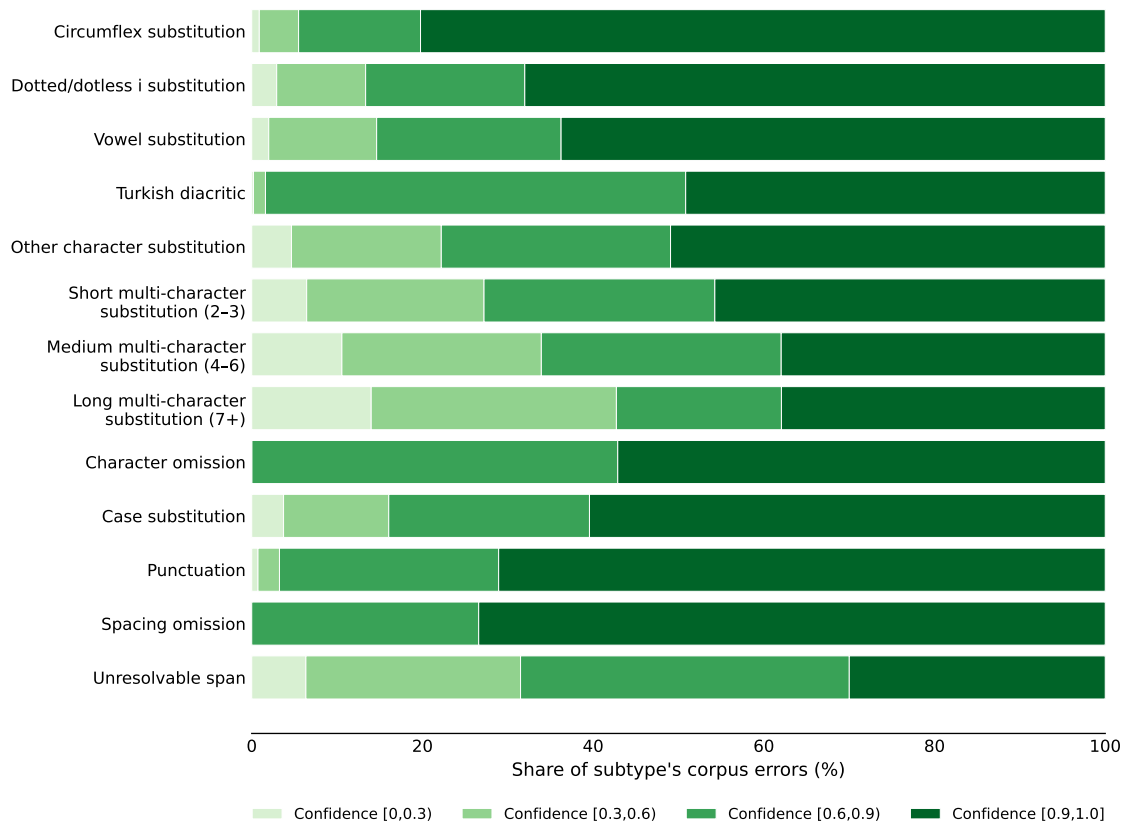


Figure 4.3 Per-subtype distribution of error characters across OCR confidence bins (corpus scope, $n = 28,198$). Each row sums to 100%; segment widths show the share of that subtype’s errors in each confidence bin. Confidence values for omission-style subtypes use sequence-mean fallback (see footnote).

Per-subtype confidence distributions are heavily skewed toward the high-confidence bin. Circumflex substitution places 80.2% of its corpus errors in $[0.9, 1.0]$; spacing omission 73.4%; punctuation 71.1%; dotted/dotless i substitution 68.0%; vowel substitution 63.8%; case substitution 60.4%. The multi-character substitution rows and the unresolvable-span row sit lower (38.0%, 37.9%, 30.0% in $[0.9, 1.0]$ for medium,

long, and unresolvable respectively), reflecting that OCR confidence drops modestly on longer corruption. The overall Spearman correlation between OCR-side character confidence and edit-span length is $\rho = -0.27$ ($n = 28,198$, $p \approx 0$): real but weak as a per-character discriminator.¹

Figure 4.3 reveals a structural alignment between “fixable” and “OCR believes it got the answer right”: the substitution subtypes a corrector could plausibly correct concentrate at high confidence, while the multi-character corruption that confidence does drop on is precisely the family where the corrector has no usable per-character pattern. This is not a probability-scale problem. Confidence calibration (Section 4.3) adjusts probability values to match observed error rates but does not redistribute the underlying error population across confidence bins. The two diagnoses are independent.

4.5.4 Position And Span Length

Error positions are distributed approximately uniformly across the sequence for most subtypes (Spearman $\rho \approx 0$ between error position and normalized sequence position for nine of thirteen rows). Three subtypes depart from uniformity: case substitution concentrates at the start of sequences ($\rho = -0.115$, reflecting capitalization of the first word), long multi-character substitutions concentrate later in the sequence ($\rho = +0.301$), and short multi-character substitutions tilt mildly toward the sequence end ($\rho = +0.075$). These three exceptions are real but localized; the rest of the thesis does not pursue position-aware detection or correction because the broader signal is absent.

Table 4.4 reports span lengths per subtype. Six subtypes are essentially single-character, sitting at length 1 by construction or near-uniformly: circumflex, dotted/dotless i, vowel, and other-character substitution, together with case substitution and spacing omission. The three multi-character substitution rows (short, medium,

1. Confidence values for roughly half the test error characters (1,556 of 3,092) use a principled fallback. Direct per-character OCR confidence is available only for single-character substitutions; multi-character substitution spans fall back to span-mean confidence (798 cases), and omissions fall back to the host sequence’s mean confidence (747 cases, with 11 corner cases using sequence mean). This reflects an operational asymmetry: substitution errors have an OCR-side character whose confidence is meaningful, but OCR omissions have no character at the omitted position, and asking what OCR was “confident about” at a position where it produced no output has no operational meaning. Per-subtype bars in Figure 4.3 therefore combine two confidence sources: substitution-style subtypes (circumflex, dotted/dotless i, vowel, case, other character substitution) carry per-character signal; omission-style subtypes (spacing, character omission) inherit the host sequence’s confidence profile.

long) are length-binned by definition. The remaining subtypes spread across more than one bin: Turkish diacritic errors and character omissions populate all four bins, punctuation errors fall in lengths 1 and 2–3, and unresolvable spans occupy lengths 2–3 and above with none at length 1. This length structure means that aggregated per-subtype counts can be misleading without length context: a single “Turkish diacritic” row averages over very different cases, ranging from one missing diacritic to a seven-character missed diacritic chain.

Table 4.4 Span length distribution by subtype (corpus scope).

Subtype	Length 1	Length 2–3	Length 4–6	Length 7+	Total
Circumflex substitution	1,694	—	—	—	1,694
Dotted/dotless i substitution	547	—	—	—	547
Turkish diacritic	358	875	525	728	2,486
Vowel substitution	1,305	—	—	—	1,305
Other character substitution	3,116	—	—	—	3,116
Short multi-character substitution (2–3)	—	6,338	—	—	6,338
Medium multi-character substitution (4–6)	—	—	1,562	—	1,562
Long multi-character substitution (7+)	—	—	—	543	543
Character omission	895	2,136	638	323	3,992
Case substitution	1,568	2	12	—	1,582
Punctuation	853	101	—	—	954
Spacing omission	361	—	—	—	361
Unresolvable span	—	1,141	2,048	529	3,718

4.6 Train/Dev/Test Split

Our corpus comprises four books from different eras and genres (see Table 3.1). All experiments use a single canonical train/dev/test split that is fixed across all methods.² The split allocates 6,614 sequences (79.7%) to training, 852 (10.3%) to development, and 828 (10.0%) to test, stratified by book to preserve the corpus-level class distribution in each partition. No tuning decision uses the test set: the F1-optimal decision threshold θ for each detection method, the beta-calibration parameters a and b used throughout (Eq. 4.2), the temperature scaling parameter T for both Method 1 and the BERT-based detectors, and all BERT learning-rate, focal-loss, and early-stopping decisions are fitted or selected on the training and development partitions alone. No test-set sequence informs any parameter choice at any stage of the pipeline, so no test-set leakage enters threshold selection, calibration fitting, or checkpoint selection.

² The exact index assignments are stored as a configuration file shipped alongside the codebase to ensure reproducibility.

Concretely: threshold selection for the detection methods of Chapter 7 uses development F1 only; calibration fitting (Section 4.3) uses training data only; model selection by epoch in Chapters 6 and 7 uses development metrics only; the test partition is consulted only once per experiment for the per-book and overall metrics reported in later chapters. No quantity that affects a reported metric is fit on or selected against the test set. The empirical landscape of Section 4.5 reports counts over the full corpus including the test partition; those counts are descriptive only and do not enter any model-selection, threshold-fitting, or calibration step.



5. EVALUATION FRAMEWORK

5.1 Detection Metrics

We assess error detectors along two complementary axes: *rank quality*, which measures how well a model separates erroneous tokens from correct ones across all thresholds, and *operating-point performance*, which evaluates binary classification at a single chosen decision threshold (we use the F_1 -optimal threshold selected on the development set).

For rank quality, we report the area under the precision–recall curve (PR-AUC) and the area under the receiver operating characteristic curve (ROC-AUC). Because our dataset is class-imbalanced (correct tokens outnumber erroneous ones roughly 3:1 on average), PR-AUC is more informative than ROC-AUC, as it focuses on the minority (error) class.

At a chosen operating threshold θ , we report precision, recall, F_1 , and F_2 on the error class. We select θ by maximizing the F_1 score on the development set, which equally balances precision and recall. F_1 provides a single summary of the precision–recall trade-off at the chosen operating point; we additionally report F_2 (which weights recall more heavily) to characterize how each detector performs under an alternative cost assumption that penalizes missed errors more than false alarms.

To quantify calibration, i.e., how well predicted probabilities match empirical error rates, we report Expected Calibration Error (ECE), Brier score, and log-loss. ECE partitions predictions into equal-width bins and computes the weighted average of the absolute difference between mean predicted probability and observed error fraction in each bin:

$$\text{ECE} = \sum_{b=1}^B \frac{|S_b|}{N} |\text{acc}(S_b) - \text{conf}(S_b)|, \quad (5.1)$$

where S_b is the set of tokens in bin b , $\text{acc}(S_b)$ is the observed error rate, and $\text{conf}(S_b)$ is the mean predicted probability. Intuitively, if a well-calibrated detector assigns

error probability 0.7 to a group of tokens, roughly 70% of those tokens should actually be erroneous; ECE measures how far observed frequencies diverge from this match across all probability bins. Aggregate ECE compresses bin-level miscalibration patterns into a single number, so two detectors with identical ECE can still differ in how over- and under-confident they are at specific probability ranges. ECE takes values in $[0, 1]$: 0 indicates perfect calibration (predicted probabilities exactly match observed error frequencies in every bin), while larger values indicate progressively poorer agreement between predicted and observed error rates. Brier score is the mean squared error between predicted probabilities and binary labels; log-loss is the negative log-likelihood under the predicted distribution.

5.2 Correction Metrics

Correction quality is measured at two granularities. Character Error Rate (CER) is the normalized edit distance between the predicted and ground-truth character sequences:

$$\text{CER} = \frac{S + D + I}{N_{\text{ref}}}, \quad (5.2)$$

where S , D , and I are the number of character-level substitutions, deletions, and insertions respectively, and N_{ref} is the length of the reference sequence. Word Error Rate (WER) is computed analogously at the whitespace-delimited token level. Both metrics are computed per sequence and then macro-averaged across sequences.

We additionally report sequence-level outcome counts: the number of test sequences that are *improved* (CER decreased), *degraded* (CER increased), or *unchanged* by each correction method. A sequence is classified as unchanged when its absolute CER change is below 0.001, the threshold used consistently across all experiments. The degradation rate, the fraction of sequences whose CER increases after correction, quantifies the risk of a method introducing new errors, complementing corpus-level CER/WER, which can mask per-sequence regressions.

5.3 Statistical Reporting

To quantify uncertainty, we report 95% bootstrap confidence intervals for all primary metrics. For each metric, we resample the 828-sequence test set with replacement 1,000 times, compute the metric on each bootstrap sample, and report the 2.5th and 97.5th percentiles as the confidence interval bounds.

All detection experiments report the full metric suite (PR-AUC, ROC-AUC, ECE, Brier, log-loss, F_1 , F_2 , precision, recall, accuracy, coverage) to enable comparison across methods. Correction experiments report CER, WER, and sequence-level outcome counts (improved/degraded/unchanged). Full notes on each statistical test, including the multiple-testing stance and the bootstrap seeds, are given in Appendix D. With the evaluation framework established, we now present the methods evaluated in this thesis, beginning with single-stage correction.

6. SINGLE-STAGE CORRECTION: UTILIZING A BYT5 MODEL

6.1 Approach

In the single-stage correction setting, the model directly maps each noisy OCR line to its corrected form without a separate error-detection stage. We use ByT5-small (Xue et al., 2022), a byte-level encoder-decoder transformer with approximately 300 million parameters, for OCR post-correction. The encoder reads erroneous OCR input and builds contextualized representations, while the decoder generates corrected output character-by-character using cross-attention. The pre-training of the model provides language understanding for distinguishing errors from valid text. Unlike subword tokenizers that can mishandle rare characters and fragment words unpredictably, ByT5 processes text as raw UTF-8 bytes, operating on a fixed vocabulary of 256 byte values plus special tokens. This naturally handles any character without vocabulary constraints and aligns with character-level OCR errors.

We first fine-tune ByT5 directly on original OCR-to-ground-truth pairs. While this produces a working corrector, it preferentially learns easy, high-frequency fixes and underweights harder Turkish-specific errors. The following synthetic augmentation strategy addresses this imbalance.

6.2 Synthetic Data Generation

OCR correction models trained on raw noisy-to-clean pairs preferentially learn high-frequency, low-impact corrections (case normalization, punctuation restoration, spacing adjustments) while underweighting semantically important Turkish-specific errors such as circumflex substitution ($\hat{a} \leftrightarrow a$), dotted/dotless i substitution ($\dot{i} \leftrightarrow \iota$, which represents different phonemes in Turkish), and vowel substitution that alters word meaning. Turkish orthography exacerbates this problem: diacritics on ç , ş , ğ , ö ,

ü are frequently lost by OCR systems despite being essential for correct Turkish text. The core difficulty is that in a standard correction pair containing both easy and hard errors, the easy patterns dominate the loss gradient and the model’s capacity is allocated accordingly.

We mitigate this through synthetic augmentation that modifies only the input side of each training pair. Starting from the original noisy OCR text, we selectively apply a subset of the ground-truth corrections (replacing OCR characters with the corresponding ground-truth characters at chosen error positions) to produce a partially corrected input. The target remains the full, unchanged ground truth. The effect is that the synthetic variant presents the same correction task as the original pair but with fewer easy distractions: the hard errors that remain are proportionally more prominent in the loss, so the model allocates more capacity to learning them. Only sequences with at least two classifiable error spans are eligible, since each variant must fix at least one error and leave at least one for the model to learn from; roughly half the training sequences meet this threshold. Each eligible sequence produces up to two such variants independently, giving the model two or three views—one original and one or two synthetic—of each correction task at different noise levels.

Each error span identified by character-level alignment receives a removal weight that controls how often it is selected for correction and thus removed from the synthetic input. The four Turkish-specific substitution subtypes (circumflex, dotted/dotless i, Turkish diacritic, vowel) receive removal weight 0.3, rarely removed from the synthetic input and frequently retained as a training signal. Formatting subtypes (case substitution, punctuation, spacing omission) receive removal weight 3.0, frequently removed. The remaining subtypes (other character substitution, the three multi-character span lengths) receive removal weight 1.0. The assignment is grounded in four converging considerations. First, the asymmetric semantic cost of leaving each kind of error uncorrected: a missed dotted/dotless i confusion produces a different Turkish word, a missed capitalization does not. Second, the empirical structure of the corpus reported in Section 4.5.3: Turkish-specific substitution subtypes concentrate at high OCR confidence (68–80% of their corpus errors fall in the $[0.9, 1.0]$ bin), exactly where the default cross-entropy loss would treat them as already correct and gain little signal from them; up-weighting these subtypes compensates for that loss-landscape blind spot. Third, the loss-dominance pressure of uniform sampling, under which multi-character and omission errors would contribute disproportionately to the gradient and push the model toward copy-through behaviour. Fourth, the pattern in prior post-OCR augmentation literature, where targeted corruption focused on phonologically or structurally critical errors has consistently outperformed uniform

schemes (Guan et al., 2024; Rijhwani et al., 2020; Stankevičius & Lukoševičius, 2022).

The 10:1 ratio between formatting and Turkish-specific weights operationalizes these priors directly but was set by hand rather than swept; a refined scheme that derives weights from the confidence distribution of Section 4.5.3 or from per-subtype fix rates measured on held-out data is a natural empirical extension. The empirical fix rates reported in Section 8.1 are partially consistent with the scheme: case substitution (32.5%) and spacing omission (22.0%) are corrected at rates higher than every Turkish-specific subtype except circumflex despite being down-weighted, and among the up-weighted subtypes circumflex reaches 42.0% while dotted/dotless i, vowel, and Turkish diacritic substitution remain in the 5–8% range typical of generic single-character errors. The weighting scheme is therefore most effective where the high-confidence concentration of Section 4.5.3 coincides with a subtype the corrector has a reliable per-character pattern for; where the corrector lacks such a pattern, up-weighting in training does not produce a proportional fix-rate gain at inference time.

For each eligible sequence, the number of errors to correct per variant is drawn uniformly at random, ensuring at least one error is corrected and at least one remains. Errors are selected via weighted sampling without replacement and applied in reverse position order to avoid offset shifts. All synthetic pairs are generated once, offline, with a fixed random seed (42). Example 5 illustrates how the original pair and its two synthetic variants present the model with three views of the same correction task at different noise levels.

Example 5 (Three training views of one correction task from *Ususül Hikem*, 1732).
Errors: $e \rightarrow i$ (vowel), $Bek \rightarrow yiğ$ (short multi-character), $' \rightarrow m$ (punctuation), $a \rightarrow â$ (circumflex).

Original pair—all 4 errors present

Input etmişler iken Bekir'i veya otuz ...

Target itmişler iken yiğirmi veyâ otuz ...

Variant 1—fixes punctuation only; two Turkish-specific plus short multi-character remain

Input etmişler iken Bekirmi veya otuz ...

Variant 2—fixes two Turkish-specific plus punctuation; short multi-character remains

Input itmişler iken Bekirmi veyâ otuz ...

Target for all three: itmişler iken yiğirmi veyâ otuz ...

Variant 1 removes only the punctuation error; both Turkish-specific substitutions and the short multi-character substitution remain, and the model trains on an input nearly identical to the original but with one easy distraction removed. Variant 2 removes both Turkish-specific substitutions and the formatting error, leaving only the short multi-character substitution—the model practises correcting a single remaining error from a nearly-clean input. The three pairs thus provide a training curriculum of increasing focus on the structurally harder error patterns.

Training uses three pair types. *Original correction pairs* map the OCR text to the ground truth, presenting the model with the full error load. *Synthetic correction pairs* map a partially corrected OCR text to the same ground truth, presenting a reduced error load that emphasizes harder errors as described above. *Identity pairs* map clean text to itself (input equals target) for original sequences with Character Error Rate below 5%, teaching the model when not to make corrections; without identity pairs, the model might develop an unconditional tendency to alter input text. The 5% threshold is applied to the source row's OCR-vs-ground-truth CER, not to the pair itself: the input and target of an identity pair are both the ground-truth reference, so the pair contains no errors regardless of small residual differences in the underlying OCR row. Identity pairs use only original (non-synthetic) sequences to ensure the model learns to preserve authentic text rather than artificially generated examples. In total, the training set comprises 14,868 pairs: 6,614 original correction pairs

presenting the full error load, 5,463 synthetic correction pairs presenting a reduced error load, and 2,791 identity pairs for the 42.2% of original training sequences whose Character Error Rate falls below 5% (per-book identity-pair counts in Appendix A, Table A.2).

The unweighted baseline trains ByT5 without incorporating OCR confidence information: all training examples contribute equally to the loss function via standard cross-entropy, regardless of whether the OCR system assigned high or low confidence to the characters. Each of the three pair types receives identical gradient contribution.

The confidence-weighted variant uses OCR confidence scores at training time to weight each example’s contribution to the loss, focusing the model’s capacity on cases most likely to need correction. For each sequence, we take the minimum confidence across all tokens, a more informative summary than the mean, which could mask a single low-confidence error among many high-confidence tokens. Raw confidence is poorly calibrated (Section 4.3); this minimum is therefore transformed into a calibrated error probability via beta calibration (Eq. 4.2), using parameters fitted on the training split (Section 4.6), and clipped to $[0.2, 1.0]$. The weight floor of 0.2 ensures that even high-confidence examples contribute some gradient, while the ceiling normalizes the range, providing $5\times$ differentiation across the distribution. Weight assignment varies by pair type: original correction pairs use the full confidence-weighted scheme, synthetic correction pairs receive fixed weight 0.5 because they are artificially generated and lack meaningful per-sequence confidence, and identity pairs receive fixed weight 1.0 because preservation is always important to learn. Confidence can also be used at inference time for selective application; we explore this in Section 8.2.

We select the best model by development set Character Error Rate (CER) rather than training loss. After each epoch, a callback generates corrections for every development sequence using beam search, computes per-sequence CER (Levenshtein distance normalized by reference length), and tracks aggregate statistics including mean CER, percentage of sequences improved, and percentage degraded.

The decision to augment training pairs through controlled noise variation rather than train solely on original correction pairs is grounded in prior post-OCR work on under-resourced and historical languages, where related strategies have consistently yielded measurable gains: Rijhwani et al. (2020) reported that rule-based noise injection improves CER reduction across five endangered languages; Guan and Greene (2024) showed that glyph-similarity-based corruption outperforms random substitution for ByT5 training across multiple languages; and Stankevičius and Lukoševičius

(2022) found that controlled augmentation aids ByT5 on Turkish and eleven other languages for diacritic restoration and typo correction. The synthetic augmentation introduced here adapts these strategies to the Ottoman Turkish error profile by varying noise level rather than corruption type. The full generation procedure is given as pseudocode in Appendix A. A matched-conditions run that removes synthetic augmentation while holding all other training settings fixed reaches 9.84% test CER, against 9.61% for the augmented model (Table 6.1); the augmentation therefore contributes to the reported CER reduction, consistent with the cited prior literature.

6.3 Experimental Settings & Data

Corpus statistics and baseline error rates are reported in Section 4.4. Following page-level splitting to prevent data leakage (Section 4.6)—every sequence from a given source page is assigned to a single partition, so no page is shared across training, development, and test—the 8,294 original sequences divide into training (6,614), development (852), and test (828). Synthetic augmentation, applied only to the training split, expands it from 6,614 to 12,077 correction pairs (14,868 total training pairs including 2,791 identity pairs; Section 6.1). Of the 6,614 training sequences, 50.8% have fewer than two classifiable error spans and therefore receive no synthetic variants; among those that do, 68.0% produce two variants and 32.0% produce one. Retention analysis confirms the intended distributional shift: synthetic pairs retain an average of 0.59 up-weighted Turkish-specific errors per sequence, compared to 0.42 in original pairs, representing a $1.40\times$ increase in up-weighted Turkish-specific error frequency. Development and test sets contain exclusively original OCR/ground-truth pairs with no synthetic augmentation, ensuring evaluation reflects real-world correction performance. The same split is used throughout.

Both ByT5 variants are fine-tuned from the google/byt5-small checkpoint (approximately 300M parameters) using the AdamW optimizer (Loshchilov & Hutter, 2019) with learning rate 5×10^{-5} , weight decay 0.01, and linear warmup over the first 10% of training steps. The effective batch size is 8 sequences (batch size 2 with gradient accumulation over 4 steps). Input and target sequences are capped at 128 bytes. Training runs for up to 10 epochs, monitored by development-set CER; the checkpoint with the lowest development CER is retained, and training halts if CER does not improve for 3 consecutive epochs. At inference, beam search with 4 beams is used; repetition-penalty constraints are disabled because Ottoman text contains valid lexical and morphological repetitions that such constraints would suppress.

All settings are identical between the two variants, and all experiments use a fixed random seed (42) for reproducibility (compute and runtime per model in Appendix D, Table D.2).

6.4 Experiments & Results

Table 6.1 Comparison of CER and WER for unweighted (UW) and confidence-weighted (CW) ByT5 models.

Metric	Model	Baseline	Corrected	Change / Reduction
CER	Unweighted	10.51%	9.61%	↓0.90pp / 8.55%
CER	Confidence-weighted	10.51%	9.66%	↓0.85pp / 8.07%
WER	Unweighted	33.10%	28.72%	↓4.38pp / 13.24%
WER	Confidence-weighted	33.10%	29.00%	↓4.10pp / 12.40%

Table 6.1 presents the overall test set performance for both correction models. The unweighted baseline achieves 9.61% CER, representing an 8.55% relative reduction from the 10.51% baseline OCR output. Word Error Rate shows larger gains, improving from 33.10% to 28.72%, a 13.24% relative reduction, suggesting that character-level corrections frequently resolve entire word-level errors. The confidence-weighted model achieves slightly higher overall CER of 9.66%. While confidence-weighted training does not improve corpus-level CER, the models differ in their error profiles: confidence weighting reduces overcorrections (sequences the model makes worse than the OCR input) at the cost of more conservative behaviour.

Table 6.2 Comparison of unweighted (UW) and confidence-weighted (CW) ByT5 on sequence-level correction outcomes.

Model	Improved (n, %)	Degraded (n, %)	Unchanged (n, %)	Net (Impr.–Degr.)
Unweighted	244 (29.47%)	79 (9.54%)	505 (60.99%)	+165
Confidence-weighted	238 (28.74%)	64 (7.73%)	526 (63.53%)	+174
Δ	−6 (−0.73pp)	−15 (−1.81pp)	+21 (+2.54pp)	+9

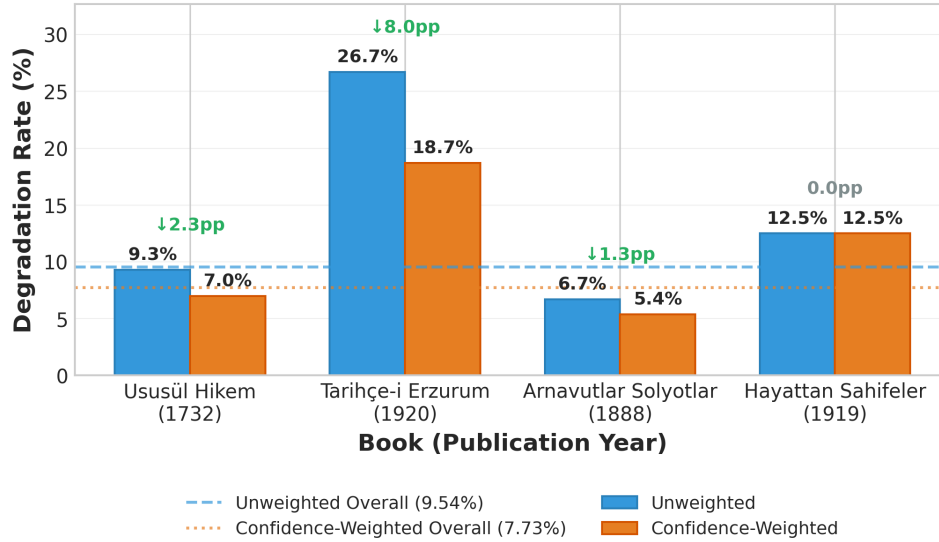


Figure 6.1 Degradation rate by book for unweighted and confidence-weighted ByT5.

Confidence weighting’s primary benefit is degradation reduction (Table 6.2, Fig. 6.1): 19% fewer sequences experience overcorrections, dropping from 79 to 64 degraded sequences (McNemar’s exact test, $p = 0.017$, significant at $\alpha = 0.05$), while improvement rates remain similar at 29% for both variants. The more conservative behaviour (64% unchanged vs 61%) means the confidence-weighted model attempts fewer edits overall, trading some potential improvements for a lower degradation rate. This pattern is most pronounced on *Tarihçe-i Erzurum*, where confidence weighting reduces degradations from 26.7% to 18.7%, enabling overall improvement where unweighted degrades. *Hayattan Sahifeler*, the already-clean text, shows identical degradation rates of 12.5% under both models, suggesting that confidence weighting’s benefit concentrates on higher-error texts where the weighting signal is most informative (per-book outcomes for both variants in Appendix A, Table A.1).

Example 6 (ByT5 correction outcomes on three test lines from Hayattan Sahifeler).¹

Row	Key Span	CER	Outcome
<i>Line 154 (degraded: ByT5 capitalises but misses word boundary)</i>			
OCR	... tekmi Çifti	9.6%	
ByT5	... <u>T</u> ekmi Çifti	11.5%	Degraded
Ref	... tek mi çift mi		
<i>Line 155 (improved: ByT5 fixes diacritic but misses morphology)</i>			
OCR	... Şerîfe ablam	14.0%	
ByT5	... Şerife ablam	12.0%	Improved
Ref	... Şerife Abla'm		
<i>Line 156 (perfect: ByT5 fixes vowel harmony)</i>			
OCR	çirkefiyla hemen koştı.	8.7%	
ByT5	çirkefiyle hemen koştı.	0.0%	Perfect
Ref	çirkefiyle hemen koştı.		

Line 154 exemplifies the degradation risk: ByT5 capitalizes “t→T” (introducing a new error) without resolving the critical word-boundary confusion “tekmi Çifti” → “tek mi çift mi”, increasing CER from 9.6% to 11.5%. Line 155 shows partial success: the diacritic correction “î→i” reduces CER, but the model fails to recover the possessive “Abla’m” (and to capitalize “Abla”). Line 156 achieves perfect correction by fixing the vowel harmony violation “ı→i” and the suffix “yla→yle”. The degradation case motivates the quality-estimation experiment in Section 8.2: if detection signals could predict which sequences will be harmed, corrections could be selectively suppressed.

6.5 Discussion

ByT5 reduced CER by 8.55% using textual context alone, without access to OCR confidence information. This confirms that byte-level processing combined with encoder-decoder architecture can learn to distinguish errors from valid text and apply character-level corrections. Beyond the corpus-level CER reduction, a smaller fraction of sequences is degraded by correction — an expected outcome for a general-

1. The *Perfect* outcome label is a special case of *Improved*: a sequence is labelled *Perfect* when the corrected CER drops to zero. Such sequences are counted under the *Improved* column of Table 6.2; *Perfect* is used here only to highlight a fully-resolved line in the worked example.

purpose corrector trained on a limited corpus, with per-sequence counts reported in Table 6.2.

The larger relative WER gain (13.24% compared to 8.55% for CER) indicates that single-character corrections often resolve entire word-level errors. In Turkish agglutinative morphology, a single diacritic or suffix error renders the whole word incorrect, so fixing one character can restore it entirely.

The oldest text, “Ususül Hikem” (1732), showed both the highest baseline error rate at 19.95% and the highest proportion of sequences improved at 50%, suggesting that heavily corrupted older texts provide richer correction opportunities where the model’s learned patterns can add substantial value. The error patterns in this book were apparently well-represented in the training distribution, enabling effective correction. Conversely, “Hayattan Sahifeler” (1919), with the lowest baseline error rate at 6.06%, showed minimal improvement at 17% of sequences, indicating either that high-quality OCR offers limited correction opportunities or that the model’s correction strategies become counterproductive when baseline quality is already high. “Tarihçe-i Erzurum” (1920) showed aggregate degradation despite 29.3% of sequences improving; this is consistent with the training-share imbalance across books: *Arnavutlar Solyotlar* contributes the majority of training sequences, whereas *Tarihçe-i Erzurum* contributes only a small fraction, so the model’s per-book correction quality correlates with how much each book is represented in training (Section 8.1 also reports a strongly book-skewed correction profile, with 216 of 346 corrections falling on *Arnavutlar Solyotlar*).

The unweighted baseline at 9.61% test CER establishes a performance floor for controlled comparison. By excluding confidence information, it isolates textual context’s contribution from potential confidence signal. Matching all other factors between variants as described in Section 6.3 enables attribution: any confidence-weighted improvement derives specifically from confidence incorporation instead of implementation details. The 61% unchanged rate suggests substantial conservatism, as intended. The degradation observed on approximately 10% of sequences highlights the opposite failure mode: cases where correction patterns learned from the training distribution are applied to contexts where they do not hold. The *Tarihçe-i Erzurum* results illustrate this: the model helps many sequences but harms a subset severely enough that the corpus-level CER worsens.

Effect of confidence weighting. The confidence-weighted variant addresses a limitation of the unweighted baseline: equal treatment of all training examples regardless of OCR certainty. As described in Section 6.1, it uses minimum confidence

per sequence transformed via beta calibration to weight each training example, focusing capacity on uncertain examples while preserving some gradient contribution from high-confidence ones.

The CER difference between the two variants is negligible (around 0.05 pp), but their sequence-level risk profiles diverge: the confidence-weighted model leaves more sequences unchanged and degrades fewer (Table 6.2). This indicates that the confidence signal helps the model identify sequences where intervention carries higher risk, leading it to preserve them instead of attempting modifications. On Tarihçe-i Erzurum, the confidence-weighted variant avoids some of the harm through more conservative behaviour. Whether the synthetic-data weighting hypothesis succeeds for the up-weighted subtypes and fails for the down-weighted ones is the empirical question taken up in Section 8.1.



7. ERROR DETECTION METHODS

The preceding chapter demonstrated that single-stage correction achieves meaningful CER reduction but is limited by its inability to target specific error types or suppress overcorrections at inference time. An alternative paradigm separates detection from correction, potentially enabling more controlled intervention. This chapter evaluates three detection strategies (calibrated confidence thresholding, contextual BERT, and confidence-fused BERT). Detection performance is reported here in F_1 , ECE, and rank-quality metrics (PR-AUC, ROC-AUC); the corresponding sequence-level correction quality under detection-guided application of the ByT5 corrector is reported in Section 8.2 alongside the selective-correction analysis.

7.1 Overview

Given the frozen AKIS recognizer, our goal is to flag likely OCR errors as a step toward targeted correction without retraining the model. This section presents three detection methods that assign error likelihoods to tokens using confidence-derived and contextual signals; we use *detection signals* as a shorthand for the per-token features produced by these methods (calibrated OCR confidence, contextual BERT logits, or their fusion). All methods operate on the shared preprocessed data pipeline established in Chapter 4: cleaned tokens with aligned confidences, binary error tags, and calibrated probability scores. These methods are described in Sections 7.2–7.4. Whether these detection signals can guide the single-stage corrector at inference time is evaluated in Section 8.2.

7.2 Method 1: Error Detection With Calibrated Confidence Thresholding

7.2.1 Approach

Method 1 serves as a detection baseline that uses only the OCR decoder’s token-level confidence scores. Each token is classified independently: a calibration function transforms its raw confidence into an estimated error probability, and a single threshold converts that probability into a binary prediction. We use the term *selective application* to mean applying a downstream operation, here a correction, only when an upstream score crosses a chosen threshold. This use of threshold-based selection is unrelated to multiplicative units in recurrent architectures. No textual context or neighbouring-token information is used for Method 1. This means the detector cannot identify high-confidence errors that are only recognizable from surrounding words, a limitation that the contextual methods in Sections 7.3–7.4 are designed to address.

Method 1 has three variants distinguished by the detection unit and feature set: a token-level beta-calibrated confidence threshold (the primary variant), a character-level variant that aggregates confidence within the WordPiece sub-tokens, and a multi-feature logistic-regression variant that combines token confidence with simple positional and aggregate features. The remainder of this section introduces and evaluates each in turn.

Method 1 quantifies how much of the detection problem can be solved by the recognizer’s own uncertainty signal alone. The gap between its performance and that of contextual methods measures the added value of linguistic context for error detection.

Method 1 operates on the preprocessed tokens and confidence scores described in Chapter 4. Each token is a WordPiece subword produced by the BERTurk tokenizer (Schweter, 2020) (`dbmdz/bert-base-turkish-cased`) applied to the OCR prediction text, with punctuation, case, and diacritical marks preserved. Of these inputs, Method 1 uses only the transferred confidence scores as features; the error labels serve exclusively as supervision for fitting calibration parameters and as ground truth for evaluation.

A non-trivial alignment problem arises because the detector operates on cleaned tokens (Section 4.1), while the decoder produces confidences for the original, uncleaned token sequence. If confidences are naively dropped or misaligned during

cleaning, the detection signal degrades. We address this by computing Levenshtein edit operations between the original and cleaned token sequences, using the edit operations to transfer each original token’s confidence to its corresponding cleaned token. For the small number of cleaned tokens with no original counterpart (0.04% of all tokens, arising from punctuation restoration during cleaning), we assign the sequence-level mean confidence as a conservative fallback. The 99.96% direct-transfer success rate validates that cleaning and confidence extraction are compatible: the preprocessing that enables fair method comparison does not destroy the detection signal. A more principled approach would propagate confidence from adjacent aligned tokens or assign a uniform prior, but the rarity of fallback cases does not justify the added complexity.

We apply the calibration framework defined in Section 4.3 to transform each token’s raw confidence into an estimated error probability. Both temperature scaling (Eq. 4.1) and beta calibration (Eq. 4.2) are evaluated; parameters are fitted on the training split by minimizing negative log-likelihood via L-BFGS-B. We include an L_2 regularization term $\lambda_{\text{reg}}(a^2 + b^2)$ and search over six candidate strengths $\lambda_{\text{reg}} \in \{0.0, 0.001, 0.005, 0.01, 0.05, 0.1\}$, selected by development-set F_1 . The optimal $\lambda_{\text{reg}} = 0.0$ indicates that with 81,160 training tokens and only two parameters, the model does not benefit from explicit shrinkage. Multi-start optimization from five initial points converges to the same solution ($a = 0.139$, $b = 0.228$), confirming a well-behaved loss surface.

Because both calibration functions preserve rank ordering (Section 4.3), the detector’s discrimination (ROC-AUC, PR-AUC) is fixed by the raw confidence ranking and cannot be improved by calibration. Calibration affects only the quality of predicted probabilities and, indirectly, the optimal threshold location.

Calibration transforms raw confidence into an estimated error probability, but a practical detector must make binary decisions. The choice of decision threshold θ determines the balance between flagging genuine errors (recall) and avoiding false alarms on correct tokens (precision). We follow the evaluation protocol defined in Section 5.1: the threshold is selected on the development set by maximizing F_1 , which equally weights precision and recall. This threshold is then frozen and applied to the test set without further adjustment. Each Method 1 variant receives its own optimized θ , since the three variants produce different score distributions despite operating on identical input data.

Token-level detection classifies entire WordPiece subwords, but not all tokens contribute equally to correction: a single incorrect character within a multi-character

token represents a small edit distance, while an entirely wrong token represents a larger one. Character-level detection provides a finer-grained view by evaluating each character position independently, potentially enabling more targeted correction that preserves correct characters within partially-erroneous tokens. The char-level variant uses the same beta calibration framework as the token-level approach, with one key difference in how confidence is assigned: since the decoder produces confidences at the subword level, each character within a subword inherits the same confidence score. The shift from token to character granularity changes the class distribution substantially: the character-level error rate on the test set is 9.4% compared to 27.4% at token level (corpus-wide rates are 8.6% and 25.7% respectively; Section 4.4). This is because most tokens are multi-character: a single-character error marks only that one character at the character level but the entire token at the token level. This more extreme class imbalance makes character-level detection a harder classification problem, as reflected in the different operating characteristics of the two variants.

The practical consequence is that if a 5-character token has low confidence, all five characters receive the same high error probability, even though perhaps only one character is actually wrong. This resolution mismatch between the confidence granularity and the detection granularity explains why character-level detection is expected to achieve lower classification F_1 than token-level detection, despite the two variants having access to the same underlying information. Character-level evaluation does, however, provide a different perspective on calibration quality: the more extreme class imbalance at character level (approximately 9.4% errors on test set) tests whether the calibrated probabilities generalize across very different base rates.

The third variant extends confidence-based detection beyond a single scalar input. Instead of beta-calibrating the raw confidence, we construct a feature vector of nine confidence-derived attributes for each token and train a logistic regression classifier with balanced class weights. The features capture three types of information. First, the token’s own confidence and its length in characters, which provide the same base signal as the single-feature variant. Second, contextual confidence features: the confidence of the immediately preceding and following tokens, the mean confidence over a three-token window centered on the current token, the sequence-level mean confidence, and the token’s confidence rank (percentile) within its sequence. Third, structural features: whether the token is a WordPiece continuation (##) and its normalized position within the sequence. All features are standardized to zero mean and unit variance before classification. The logistic regression model is trained with balanced class weighting to account for the 3:1 correct-to-error ratio, using the

L-BFGS-B solver with a maximum of 1,000 iterations.¹

The threshold for the multi-feature variant is selected by the same development-set F_1 maximization procedure used for all Method 1 variants (Section 5.1), ensuring that the comparison between single-feature beta calibration and multi-feature logistic regression reflects genuine differences in detection ability instead of threshold selection artifacts. Logistic regression produces calibrated probabilities by construction when fit with maximum likelihood, so no additional calibration step is needed, though this means its ECE cannot be directly compared to the post-hoc-calibrated beta variants. This contrasts with the single-feature variants, where post-hoc calibration (beta or temperature) is the primary transformation.

The nine features are all derived from confidence scores and token structure; no textual content is used. This means the model can detect patterns in confidence distributions (e.g., consistently low confidence in a region) but cannot detect errors that require linguistic knowledge (e.g., a morphologically impossible suffix). Logistic regression was chosen over more expressive classifiers such as gradient boosting or neural networks because with only nine features and approximately 81,000 training tokens, a linear model provides a reasonable bias-variance trade-off while remaining interpretable through its feature coefficients. The balanced class weighting addresses the 3:1 class imbalance but may not be optimal for all books, given that per-book error rates range from 15% to 34% in the test set.

1. The scikit-learn implementation exposes this as `solver='lbfgs'` but invokes the same SciPy bounded-memory routine internally; we use the algorithm name L-BFGS-B throughout for consistency.

To illustrate how calibration operates at the token level, Example 7 traces beta calibration on the training-set line introduced in Fig. 3.1 (line 0, “İstanbul’un Edirne kapısı, sürdüğünün içine”). Using the parameters $a=0.139$, $b=0.228$ from Eq. 4.2 and threshold $\theta=0.341$, each token’s raw confidence is mapped to an estimated error probability.

Example 7 (Method 1 calibration walkthrough on line 0).² *Beta calibration maps raw confidence to error probability; thresholding yields binary predictions.*

Token	Raw	$\hat{p}$	$>\theta?$	True	Result
İstanbul	.987	.407	✓	0	FP
,	1.00	.049		0	TN
un	1.00	.049		0	TN
Edirne	1.00	.286		0	TN
kapısı	.821	.504	✓	1	TP
,	1.00	.197		0	TN
sürdüğünü	.334	.580	✓	1	TP
##n	1.00	.049		1	FN
içine	1.00	.121		0	TN
Confusion: TP = 2, FP = 1, FN = 1, TN = 5. P = 0.667, R = 0.667.					

Two classification errors expose the strengths and limitations of confidence-only detection. The **false positive** on “İstanbul” ($c=0.987$, $\hat{p}=0.407$) arises because beta calibration assigns an above-threshold error probability to almost every token whose raw confidence is below the ceiling—a consequence of the heavily skewed confidence distribution where even slightly-below-ceiling scores are rare and correlated with errors in the training data. A contextual model would recognize that “İstanbul” is a common proper noun and suppress this alarm. The **false negative** on “##n” ($c=1.0$, $\hat{p}=0.049$) is a fundamental limitation: the continuation subword inherits ceiling confidence from the decoder despite being part of the erroneous reconstruction “sürdüğünün” $\rightarrow$ “sur duvarının”. Because Method 1 treats each token independently, it cannot detect that “##n” belongs to an erroneous word group. More broadly, all tokens with ceiling confidence ($c=1.0$) receive $\hat{p}=0.049$ under the canonical clipping boundary ($\epsilon=10^{-10}$), well below the decision threshold; their binary classifications are therefore robust to the exact choice of ϵ . These two different types of failures, false

2. Raw confidences are displayed to two decimals; tokens shown as 1.00 represent values in the range $[0.99, 1.00)$. The probability column $\hat{p}$ is computed from the full-precision values, which differ across these visually-identical entries.

alarm on correct high-frequency tokens and missed detection of confident-but-wrong continuations, directly motivate the contextual methods in Sections 7.3–7.4.

7.2.2 Experiments & Results

Method 1 tests how much detection signal resides in the OCR decoder’s confidence scores alone, without textual context. Table 7.1 presents the test-set detection results for all three Method 1 variants, evaluated on 10,393 tokens across 828 sequences.

Table 7.1 Token-level error detection results for three calibrated confidence thresholding variants.

Metric	Token- β	Char- β [†]	Multi-Feature
F ₁	0.577 [0.559, 0.594]	0.347 [0.327, 0.364]	0.588 [0.570, 0.605]
Precision	0.521 [0.500, 0.542]	0.268 [0.250, 0.285]	0.528 [0.507, 0.549]
Recall	0.647 [0.625, 0.666]	0.492 [0.469, 0.516]	0.663 [0.639, 0.685]
F ₂	0.617	0.421	0.631

[†] Char- β is evaluated at character granularity (32,655 test characters); the other two variants are evaluated at token granularity (10,393 test tokens).

The token-level beta calibration variant achieves $F_1 = 0.577$ [0.559, 0.594] with precision 0.521 and recall 0.647: it correctly identifies nearly two-thirds of erroneous tokens while accepting substantial false positives.

The character-level variant, evaluated on 32,655 characters, achieves substantially lower $F_1 = 0.347$ [0.327, 0.364], indicating that three out of four flagged characters are actually correct. This reflects the more severe class imbalance (9.4% error rate versus 27.4%) and the resolution mismatch between subword-level confidence and character-level evaluation. This variant demonstrates that inherited subword confidence is a poor proxy for character-level error status, establishing a lower bound on what confidence alone can achieve at fine granularity.

The multi-feature logistic regression achieves the highest Method 1 $F_1 = 0.588$ [0.570, 0.605]: contextual confidence patterns (relative rank within a sequence, neighbouring token confidences) add 1.1 percentage points over the single-feature baseline (Table 7.3). The gain is modest. These results collectively define the confidence-only detection ceiling against which contextual models (Methods 2–3) are compared.

Table 7.2 Calibration and operating-point diagnostics for Method 1 variants.

Metric	Token- β	Char- β	Multi-Feature	Temp. Scaling
ECE@10	0.058	0.011	0.171	0.063
Brier	0.165	0.077	0.183	0.167
Log Loss	0.509	0.276	0.552	0.511
ROC-AUC	0.759	0.767	0.796	0.759
PR-AUC	0.572	0.279	0.615	0.572
Parameters	$a=0.139, b=0.228$	$a=0.164, b=-1.172$	balanced LR	$T=8.27$
Threshold (θ)	0.341	0.173	0.512	0.316

Table 7.3 Standardized coefficients of the multi-feature logistic regression, ranked by absolute magnitude.

Rank	Feature	Coefficient
1	confidence rank within sequence	-0.688
2	sequence-level mean confidence	-0.423
3	character count excl. ##	-0.391
4	WordPiece ## token	+0.301
5	raw token confidence	-0.278
6	3-token window mean	-0.221
7	previous token confidence	-0.122
8	next token confidence	-0.054
9	normalized sequence position	-0.008

7.2.3 Discussion

We evaluate both calibration functions on the same data and splits. On this dataset, temperature scaling’s optimized parameter is pushed high within the search range ($T = 8.27$; range $[0.01, 10.0]$), indicating that the single-parameter model cannot adequately capture the distortion in the raw confidence distribution. Despite this, the two calibration methods produce identical detection outcomes at their respective F_1 -optimal thresholds: the confusion matrices coincide exactly (1,845 true positives, 1,694 false positives, 5,847 true negatives, 1,007 false negatives). This equivalence occurs because both methods preserve rank ordering, and F_1 optimization selects thresholds that yield the same binary partition. The methods differ only in calibration quality (Table 7.2): beta calibration achieves $ECE = 0.058$ versus 0.063 for temperature scaling, and Brier score 0.165 versus 0.167. The practical implication is that for Method 1, the choice of calibration function matters only for the quality of predicted

probabilities, not for detection accuracy. Since beta calibration achieves marginally better probability calibration ($\text{ECE} = 0.058$ versus 0.063 for temperature), we select it as the primary variant.

Both calibration methods share a fundamental constraint: they are global transformations applied identically to every token regardless of its textual context, sequence position, or book of origin. Temperature scaling applies a single rescaling parameter to the entire confidence distribution, which assumes that the calibration distortion is uniform across all error types and text domains. Beta calibration relaxes this uniformity assumption by allowing asymmetric corrections — it can, for example, shrink over-confident scores more aggressively than it expands under-confident ones, but it still applies a single pair of parameters across the entire dataset. Neither method can capture book-specific or context-dependent calibration patterns, such as the possibility that confidence scores are better calibrated on the 1919 text than on the 1732 text. The formal definitions of both functions are provided in Section 4.3; here we focus on their detection performance.

7.3 Method 2: Error Detection With Contextualized BERT

7.3.1 Approach

Method 1 (calibrated confidence thresholding) captures the OCR decoder’s own uncertainty about its output as a confidence-only baseline. However, OCR errors arise from two distinct sources: visual ambiguity in the scanned image, and contextual implausibility in the recognized text. A high-confidence token may still be erroneous if it is semantically inconsistent with its neighbours. By fine-tuning a pre-trained Turkish BERT model for binary token classification, we test whether self-attention over the surrounding context provides complementary signal to the decoder’s visual uncertainty, establishing a context-only baseline against which Method 3 (confidence-fused BERT) will be evaluated.

We adopt a standard fine-tuning approach for token classification: the full BERT encoder processes the input sequence, and each token’s contextual representation is passed through a classification head that predicts whether the token contains an OCR error. Unlike frozen-embedding approaches, all 12 transformer layers are trainable, enabling the model to specialize its contextual representations for error detection rather than relying on general-purpose language features. The classification

head uses a bottleneck architecture ($768 \rightarrow 384 \rightarrow 1$) with ReLU activation and dropout regularization, reducing the hidden dimension by half before the binary prediction. The goal is to measure what contextual information alone contributes, not to engineer a complex classifier.

The architecture processes each OCR line independently, with no cross-line context. This means errors arising from page-level coherence (e.g., a word split across two lines) cannot be detected.

The classifier consumes the recognizer’s WordPiece sequence directly, with one BERT input position per WordPiece and no further re-tokenization. Error labels are produced by Levenshtein alignment of the prediction sequence to the reference: prediction tokens involved in replace or delete operations receive label 1, all others receive label 0. Because labelling and evaluation share the WordPiece grid, every input position carries exactly one binary label and no cross-position aggregation is needed. The model outputs uncalibrated logits that are corrected post hoc by temperature scaling (Section 4.3). This protocol is shared by all BERT-based methods.

7.3.2 Experimental Settings & Data

We address the class imbalance (roughly 25% of WordPieces are erroneous) with focal loss (Lin et al., 2017), which down-weights easy examples so that training concentrates on hard-to-classify positions. The full encoder is fine-tuned with AdamW under a linear warmup-and-decay schedule, with early stopping on development-set F_1 . The training settings shared across all BERT-based methods are listed in Table D.1.

After training, we calibrate the model’s output probabilities via post-hoc temperature scaling (Eq. 4.1, Section 4.3), applied here to the trained BERT model’s output logits rather than raw OCR confidence. A scalar temperature T is fitted on development-set logits by minimizing negative log-likelihood via L-BFGS-B, bounded to $[0.01, 10.0]$. The calibrated probability for each token is then $\sigma(\text{logit}/T)$, where σ is the sigmoid function. On the calibrated development probabilities, we select the decision threshold θ that maximizes token-level F_1 , scanning along the full precision-recall curve. Both T and θ are then applied unchanged to the test set. This procedure is shared by all BERT-based methods; for BERT-only, the learned temperature is $T = 1.207$ and the selected threshold is $\theta = 0.382$.

The single-split evaluation means that results depend on the specific random partition.

Sequence-level bootstrap resampling ($n = 1,000$) provides confidence intervals that account for test-set variability. The error rates differ somewhat across splits (25.7% train, 24.7% dev, 27.4% test), reflecting the natural heterogeneity of book-level error distributions before any model is applied. All three detection methods use identical train, dev, and test splits and are evaluated on the same WordPiece grid, so their results are directly comparable.

7.3.3 Experiments & Results

Table 7.4 Token-level error detection results for the BERT-only baseline (Method 2).

Metric	Value	95% CI
F ₁	0.640	[0.623, 0.658]
Precision	0.591	[0.569, 0.613]
Recall	0.697	[0.675, 0.720]
F ₂	0.673	—

Table 7.5 Calibration diagnostics for the BERT-only detector (Method 2).

Metric	Value
ECE@10	0.022
Brier	0.140
Temperature (T)	1.207
Threshold (θ)	0.382

The rank-quality PR-AUC of 0.665 (ROC-AUC: 0.834) confirms that the model’s ranking ability exceeds the class prior, though the gap between the two area metrics reflects the 3:1 class imbalance.

7.3.4 Discussion

BERT’s operating profile is moderately balanced, leaning slightly toward recall: recall reaches 0.697 while precision is 0.591 (Table 7.4). The gain is driven by contextual attention: BERT catches tokens that are linguistically implausible in their surrounding context despite receiving high decoder confidence. A clear example is the continuation subword “##n” in Example 7, where ceiling confidence masks

membership in a completely wrong word. These are errors where the confidence value itself is unremarkable — the error is only visible when the token is evaluated in context.

The precision-recall trade-off has practical consequences: roughly four in ten flagged tokens are correct rather than erroneous, so a downstream corrector that acts on every flagged token will encounter some non-error inputs. Whether this is acceptable depends on the corrector’s conservatism — a conservative corrector that tends to preserve input tolerates these false positives, while an aggressive corrector risks turning them into new errors.

The single-split evaluation limits how much can be concluded from these results. Per-book error rates range from 14.9% to 34.4% on the test set, and the model may over-flag tokens in cleaner books while under-flagging in harder ones. Bootstrap confidence intervals address test-set variability but not this distributional concern.

Post-hoc temperature scaling yields $\text{ECE} = 0.022$ and $\text{Brier} = 0.140$ (Table 7.5), indicating tight alignment between predicted error probabilities and observed error rates. The learned temperature $T = 1.207$ is above 1.0, meaning the raw logits are slightly overconfident and need modest softening to match the empirical distribution.

Method 2’s structural limitation is that it processes only the recognized text, with no access to the decoder’s internal confidence. Two tokens equally plausible in context, one recognized confidently and the other uncertainly, appear identical to BERT. In a low-resource setting where token-level confidence may be the only per-token quality signal available, ignoring it leaves out a potentially useful signal.

7.4 Method 3: Error Detection With Confidence-Fused BERT

7.4.1 Approach

Method 3 tests whether combining contextual coherence from BERT with visual certainty from the recognizer yields a more accurate detector. We instantiate four variants spanning two orthogonal design choices. The first axis is the fusion architecture: an additive variant that perturbs BERT’s output embeddings, and a concatenation variant that feeds confidence through a small projection and concatenates the result before classification. Both architectures operate after the encoder, so the fusion modulates the contextual representation that the classification head receives rather

than the input the transformer processes. The second axis is the confidence input: each architecture is trained once on the raw per-WordPiece confidence and once on its beta-calibrated counterpart (Section 4.3). Training on beta-calibrated probabilities tests whether the reliability map fitted in Section 4.3 transfers usefully into a learned classifier; training on raw confidence asks the fusion module to absorb any calibration mismatch on its own.

The fusion hypothesis assumes that confidence and context carry non-redundant information about errors. Method 2’s ROC-AUC of 0.834 sets a ceiling on what context alone provides, while Method 1’s ROC-AUC of 0.759 sets a floor on what confidence alone provides; the 0.075-point gap quantifies the non-overlapping signal that fusion is positioned to capture.

Method 1 and the Method 3 fusion variants both consume per-WordPiece confidence. The recognizer stores its confidences at character granularity, so we aggregate them to WordPieces identically across methods: for each WordPiece, the character-level confidences of its constituent characters (excluding the leading continuation marker) are mean-pooled. The aggregation succeeds for every WordPiece in the corpus. The raw-confidence variants of Method 3 feed this pooled vector unchanged into the fusion module. The beta-calibrated variants pass the pooled vector through the same calibration map used by Method 1 before fusion, so each WordPiece enters the fusion module carrying exactly Method 1’s per-token error probability. The beta-calibrated fusion variants can therefore be read as supplying BERT with Method 1’s full output as an auxiliary feature, rather than with a generic transformed confidence.

7.4.1.1 Variant A: Additive fusion

Additive fusion is inspired by the ConfBERT architecture of Hemmer et al. (2024), which integrates OCR confidence directly into token embeddings for error detection. It integrates confidence via a minimal perturbation of the BERT embeddings. Given BERT’s output embedding $\mathbf{e}_i \in \mathbb{R}^{768}$ for token i and the token’s aggregated confidence c_i , the fused representation is

$$\mathbf{e}'_i = \mathbf{e}_i + \lambda \cdot c_i \cdot \mathbf{d}, \quad (7.1)$$

where $\lambda \in \mathbb{R}$ is a learnable scalar (initialized to 0.1, clamped to $[0, 1]$) and $\mathbf{d} \in \mathbb{R}^{768}$ is a learnable *error direction* vector. This requires only 769 additional parameters

(1 scalar + 768 vector). The model learns both the magnitude (λ) and the direction ($\mathbf{d}$) of the confidence-dependent perturbation in embedding space. The scalar λ converged from its initial value of 0.1 to 0.145 under raw confidence and to 0.134 under beta-calibrated confidence, indicating that the fusion uses the confidence signal modestly in both formulations.

The additive architecture constrains confidence integration to a single direction in embedding space: all tokens are shifted along the same vector $\mathbf{d}$, scaled only by their confidence score. Consequently, the model cannot learn token-specific confidence-error relationships (e.g., content words vs function words, or word-initial vs continuation subwords). Clamping λ to $[0, 1]$ also prevents the model from inverting the confidence signal or amplifying it beyond a unit perturbation. The convergence to $\lambda \approx 0.13\text{--}0.15$ (well below the upper bound of 1.0) suggests that larger perturbations are harmful, likely because they distort the pretrained BERT representations too aggressively. This conservative behaviour contrasts with the concatenation variant, which has a greater capacity to modulate how it uses confidence.

7.4.1.2 Variant B: Concatenation fusion

While additive fusion constrains confidence to a single direction in embedding space, concatenation fusion allows the model to learn arbitrary nonlinear relationships. It uses a richer integration: the scalar confidence is first projected into a learned 64-dimensional embedding, then concatenated with BERT’s 768-dimensional output and projected back to the original hidden size. Formally, for token i with BERT embedding $\mathbf{e}_i \in \mathbb{R}^{768}$ and aggregated confidence c_i :

$$\mathbf{c}_i = \text{ReLU}(\mathbf{W}_1 c_i + \mathbf{b}_1), \quad (7.2)$$

$$\mathbf{e}'_i = \mathbf{W}_2 [\mathbf{e}_i; \mathbf{c}_i] + \mathbf{b}_2, \quad (7.3)$$

where $\mathbf{W}_1 \in \mathbb{R}^{64 \times 1}$, $\mathbf{W}_2 \in \mathbb{R}^{768 \times 832}$, and dropout (rate 0.3) is applied after the ReLU. This architecture requires 639,872 trainable fusion parameters: 128 for the confidence projection (64 weights + 64 biases) and 639,744 for the back-projection layer ($768 \times 832 = 638,976$ weights + 768 biases). The model can learn arbitrary nonlinear mappings from confidence to embedding adjustments, at the cost of a substantially larger fusion module.

The concatenation architecture introduces 639,872 parameters (832 $\times$ more than

additive fusion), raising the risk of overfitting on a dataset of 81,160 training tokens. The dropout (0.3) on the confidence projection mitigates this risk, and early stopping prevents the model from memorizing training patterns. The best epoch for concatenation (epoch 4)³ is earlier than for additive (epoch 9) or BERT-only (epoch 9), suggesting that the additional parameters accelerate convergence but also approach overfitting earlier. The 64-dimensional confidence projection was not hyperparameter-tuned; this dimension was chosen to be small enough to limit parameter growth while large enough to represent nonlinear confidence patterns.

To isolate the effect of confidence integration, both fusion architectures inherit every design choice from Method 2’s BERT-only classifier. The backbone, the classifier head, the supervision strategy, the loss function, the optimizer, the learning rate schedule, and the evaluation protocol are held constant. This controlled design means that any performance difference between Method 2 and Method 3 is attributable solely to the fusion mechanism and its learned parameters. The fusion module is inserted at a single point in the architecture (between BERT’s final hidden states and the classification head) and does not modify any other component.

7.4.2 Experimental Settings & Data

All four fusion configurations follow the training protocol described for Method 2; the training settings are listed in Table D.1, and the only method-specific differences are the fusion parameters described above and the resulting learned temperatures and thresholds, reported in Table 7.7.

7.4.3 Experiments & Results

Tables 7.6 and 7.7 report token-level detection metrics and calibration diagnostics for the four fusion variants (two architectures crossed with two confidence-input formulations). Concatenation fusion with beta-calibrated confidence achieves the best F_1 (0.647), with precision 0.584 and recall 0.726; raw-confidence concatenation reaches $F_1 = 0.643$ (precision 0.603, recall 0.688); the additive variants reach $F_1 = 0.638$ (raw confidence) and 0.641 (beta-calibrated). Compared with the Method 2 BERT-only baseline ($F_1 = 0.640$, precision 0.591, recall 0.697; Tables 7.4 and 7.5),

3. Epochs are 1-indexed. Training ran for 10 epochs in every BERT-based configuration; the best-epoch column reports the epoch at which development-set F_1 was highest and from which the test-set evaluation checkpoint was taken.

no single fusion variant dominates on every metric: concatenation fusion leads on token-level F_1 , ROC-AUC, and PR-AUC, while the additive variants and the BERT-only baseline achieve substantially lower calibration error. On rank-quality metrics, concatenation also leads (ROC-AUC = 0.844, PR-AUC = 0.677 with beta-calibrated confidence), confirming that the F_1 gain holds when controlling for operating point.

Table 7.6 Token-level error detection results for confidence-fused BERT variants. Bold F_1 marks the canonical Method 3 variant.

Metric	Additive fusion		Concatenation fusion	
	Raw	β -cal	Raw	β -cal
F_1	0.638 [0.619, 0.657]	0.641 [0.622, 0.660]	0.643 [0.622, 0.663]	0.647 [0.628, 0.667]
Precision	0.563 [0.540, 0.585]	0.580 [0.557, 0.603]	0.603 [0.578, 0.626]	0.584 [0.560, 0.606]
Recall	0.737 [0.714, 0.760]	0.717 [0.695, 0.740]	0.688 [0.664, 0.714]	0.726 [0.704, 0.750]
F_2	0.694	0.685	0.669	0.692

Table 7.7 Calibration and rank-quality diagnostics for the confidence-fused BERT variants.

Metric	Additive fusion		Concatenation fusion	
	Raw	β -cal	Raw	β -cal
ECE@10	0.025	0.025	0.067	0.066
Brier	0.140	0.140	0.144	0.143
Log Loss	0.437	0.437	0.443	0.441
ROC-AUC	0.833	0.833	0.842	0.844
PR-AUC	0.663	0.662	0.677	0.677
Temperature (T)	1.209	1.211	0.630	0.623
Threshold (θ)	0.332	0.361	0.489	0.445
Best epoch	9	9	4	4
Fusion params	769		639,872	

Concatenation fusion also leads on rank quality, with the additive variants only slightly behind; the ROC-AUC and PR-AUC figures for all four variants are given in Table 7.7.

7.4.4 Discussion

The two fusion architectures exhibit contrasting behaviours that show how design choices affect confidence integration. The opposite temperature scaling directions

are informative here. Additive fusion requires $T \approx 1.21$ (both confidence inputs): the model is overconfident and needs softening. Concatenation requires $T \approx 0.63$ (both confidence inputs): the model is underconfident and needs sharpening. Additive fusion adds only 769 parameters to the BERT backbone, so the pretrained representations dominate prediction; the small confidence perturbation slightly concentrates the already-sharp logits, producing overconfidence. Concatenation adds 639,872 parameters that transform the representation before classification, introducing additional variance that softens the output distribution. Temperature scaling corrects these opposing tendencies.

Within the fusion variants, a calibration-discrimination tension emerges. The concatenation variants rank error tokens best, while the additive variants produce better-calibrated probabilities (Table 7.7). Among all detectors reported in this thesis, the lowest expected calibration error is attained by the BERT-only baseline ($\text{ECE} = 0.022$; Section 7.3), with the additive fusion variants close behind. This presents a practical trade-off: better probability estimates from the simpler architecture versus better ranking of error tokens from the richer one. Most post-OCR detection work focuses on F_1 or AUC and ignores whether predicted probabilities are trustworthy. When detection scores are used to decide whether to apply a correction model, calibration matters: a detector that reports $P(\text{error}) = 0.8$ for tokens that are actually erroneous 80% of the time enables more principled threshold selection, while a poorly calibrated detector requires ad-hoc tuning.

7.5 Comparative Analysis

At the token level, F_1 rises from Method 1 (calibrated confidence thresholding, $F_1 = 0.577$) to Method 2 (BERT-only contextual classifier, $F_1 = 0.640$) to Method 3 (confidence-fused BERT, beta-calibrated concatenation variant, $F_1 = 0.647$). Almost all of this comes from one step, the move to contextual modelling in Method 2. As the sequence-level bootstrap below shows, only the Method 1 to Method 2 step is statistically reliable. This ratio suggests that context and confidence respond to many of the same token-level anomalies, and their non-overlapping information is limited (per-book detection breakdowns for all methods in Appendix B, Tables B.1–B.6).

The statistical significance of these differences varies. The bootstrap confidence intervals for Method 1 (Table 7.1) and Method 2 (Table 7.4) do not overlap, establishing that contextual modelling provides genuine added signal beyond confidence alone.

The intervals for Method 2 and the canonical Method 3 variant (concatenation with beta-calibrated confidence) [0.628, 0.667] (Table 7.6) overlap, with the Method 3 lower bound only marginally above the BERT-only point estimate. To test these comparisons formally, we run a paired bootstrap on token-level F_1 across the three methods. Each replicate resamples the 828 test sequences with replacement ($B = 10,000$ replicates, seed 20260430), and each method’s F_1 is computed on the resampled tokens; the per-replicate difference ΔF_1 is recorded and a percentile 95% confidence interval is taken on the empirical distribution. The bootstrap confirms what the intervals suggest (full figures in Table 7.9): the gain from adding context (Method 1 to Method 2) is large and significant, as is Method 3 over Method 1, but the further gain of fusion over BERT-only context (Method 3 over Method 2) is not significant.

The three models operate at different F_1 -optimal thresholds: 0.341 for Method 1, 0.382 for Method 2, and 0.445 for the canonical Method 3 variant (concatenation, beta-calibrated confidence). This means their precision-recall profiles are not directly comparable at face value. Method 2 operates with a slight recall lean (recall 0.697, precision 0.591); the canonical Method 3 variant trades a small amount of precision for a small amount of additional recall (recall 0.726, precision 0.584). The token-level rank-quality metrics confirm that Method 3’s advantage persists after controlling for the per-token operating point: concatenation leads by ROC-AUC (0.844) and PR-AUC (0.677) as well as F_1 , although the size of the ROC-AUC advantage over Method 2 (0.834) is small.

In practice a corrector is applied to a whole line, not to individual tokens, so the question that matters is per-sequence: should this line be sent for correction at all? We answer yes whenever the detector flags at least one token in the line (OR-aggregation). Re-scoring each method under this rule, with the same dev-tuned thresholds, gives sequence-level F_1 of 0.897 (Method 1), 0.892 (Method 2), and 0.892 (Method 3, canonical variant); 657 of the 828 test sequences contain at least one ground-truth error.

At the sequence level the token-level ranking disappears. A paired bootstrap ($B = 10,000$, same resampled sequences for all three methods; Berg-Kirkpatrick et al., 2012; Koehn, 2004) finds no significant difference between any pair (all $p > 0.5$; full figures in Table 7.8). The ranking even reverses: Method 1’s lower token-level threshold ($\theta = 0.341$, against 0.382 for Method 2 and 0.445 for Method 3) makes it flag more sequences, and on this high-error-rate test set the higher flag rate is enough to offset its lower per-sequence precision. The fusion advantage seen at the per-token level therefore does not carry over to the sequence-level decision a corrector actually

makes.

Table 7.8 Sequence-level paired bootstrap on OR-aggregated detection ($B = 10,000$; 828 test sequences). Δ is the F_1 of the later, more elaborate method minus that of the earlier one.

Comparison	ΔF_1	95% CI	p
M2 – M1	−0.006	[−0.022, +0.011]	0.516
M3 – M1	−0.005	[−0.021, +0.011]	0.533
M3 – M2	+0.0005	[−0.012, +0.012]	0.943

Table 7.9 Token-level paired bootstrap on detection F_1 ($B = 10,000$; 828 test sequences). Δ is the F_1 of the later, more elaborate method minus that of the earlier one.

Comparison	ΔF_1	95% CI	p
M2 – M1	+0.063	[+0.042, +0.082]	<0.001
M3 – M1	+0.069	[+0.050, +0.089]	<0.001
M3 – M2	+0.007	[−0.002, +0.017]	0.151

Calibration quality varies across the methods as follows. Method 1 beta calibration achieves $ECE = 0.058$ via a two-parameter transformation of raw confidence. Method 2 achieves $ECE = 0.022$ via post-hoc temperature scaling of the BERT classifier — the lowest expected calibration error of any detector reported in this thesis. Within Method 3, the additive variants closely approach the Method 2 baseline at $ECE = 0.025$, while the concatenation variants remain markedly less well calibrated at $ECE = 0.067$ (raw) / 0.066 (beta-calibrated). Method 2’s temperature scaling acts on the high-dimensional logit space of a fine-tuned transformer, whereas Method 1’s beta calibration learns only two scalar parameters from a single raw confidence score. The calibration gap of 0.058 against 0.022 is consistent with that difference in expressive capacity.

The methods differ not only in aggregate performance but in the types of errors they can detect. Method 1 relies entirely on the decoder’s uncertainty signal and cannot detect high-confidence errors that are only recognizable from context. The continuation subword “##n” in Example 7 illustrates this: it inherits ceiling confidence from the decoder despite being part of a completely wrong word. Method 1 has no way to flag such tokens. Method 2 catches them because they sit in linguistically implausible contexts, but it cannot distinguish between two contextually plausible tokens that differ in decoder confidence. Method 3 addresses this gap modestly: the

fusion of both signals yields the highest discrimination, but the gain is small because context already captures most of the detectable signal.

The practical implication for downstream correction depends on the corrector’s behaviour. Method 2’s slightly recall-leaning profile means a corrector receives a modest number of correct tokens alongside the genuine errors. If the corrector is conservative (as the ByT5 corrector in Chapter 6 is, leaving 61% of sequences unchanged), those false positives are tolerable because the corrector will not act on most of them. If the corrector were more aggressive, they would more often translate into introduced errors. The canonical Method 3 variant has a near-identical operating point, so it does not materially alter this picture.

This thesis does not build a separate second-stage corrector; it only tests whether these detection scores can decide when to apply the existing single-stage model. The quality-estimation results in Section 8.2 show that they cannot do so reliably on this data, which is the main limit on detection-guided selective correction here. In practical terms, which detector to use depends on the goal. For ranking individual error tokens, for example token-level routing or specialized downstream correctors that operate on single tokens, the canonical Method 3 variant (concatenation with beta-calibrated confidence) is best. When the probability values themselves must be trustworthy rather than merely well-ordered, Method 2 (BERT-only) is preferable, since it attains the lowest expected calibration error in this study. For the sequence-level decision of whether to apply a corrector to a whole line, no method is significantly better than the others under token-tuned thresholds, so Method 1, which needs no model training beyond calibration fitting, is a defensible default.

8. CORRECTION ANALYSIS

The preceding chapters evaluated detection and correction methods individually: Chapter 6 established that ByT5 achieves an 8.55% relative CER reduction with 30% of sequences improving and 10% degrading, while Chapter 7 showed that detection methods reach token-level F_1 up to 0.647, with most of the signal attributable to contextual modelling. These aggregate metrics, however, cannot reveal which error types are actually corrected, whether inference-time selective correction can suppress overcorrections, or how detection and correction interact on individual lines. This chapter takes up each question in turn.

8.1 Error-Type Correction Analysis

How much ByT5 helps depends heavily on the error subtype: some are corrected reliably, most are not. This section decomposes the 3,092 character-level errors in the test set by the thirteen subtypes of Section 4.5 and reports fix rates, untouched rates, and newly introduced errors per subtype, identifying which families a general-purpose corrector can reach and which lie outside its operating range. The Section 6.2 weighting was designed to push the model toward Turkish-specific substitutions; whether that succeeded is one of the empirical questions answered here.

For each test sequence, the OCR text and the model output are independently aligned against the ground truth using Ratcliff/Obershelp on character lists. Both alignments are indexed in ground-truth coordinate space, so every ground-truth position can be directly compared across the two outputs without a separate cross-alignment step. Two classes of characters fall outside this per-position analysis: 875 hallucinated characters (present in OCR but absent from ground truth) are excluded because they have no anchor position, and 438 positions within multi-character spans that inner alignment could not individually resolve are retained as a single Unresolvable span row rather than discarded (per-book distribution and joint per-span outcomes in Appendix C, Tables C.3–C.4).

Example 8 (representative unresolvable spans drawn from each book in the test set).

These spans illustrate why 438 ground-truth character positions cannot be individually classified: in each case the outer alignment yields a multi-character replace whose inner secondary alignment again produces a multi-character replace with unequal OCR and GT lengths, leaving no one-to-one character correspondence. The model copies the OCR span verbatim in five of the six cases shown (fate: untouched); the Hayattan Sahifeler example (**fey**→**Fi**) is the exception, where the model altered the surrounding context. Either way, these positions remain in the Unresolvable span row of Table 8.1.

Book	OCR span	GT span	OCR/GT chars	Inner unresolvable
Arnavutlar Solyotlar	id-i	et	4/2	2
Arnavutlar Solyotlar	fethi	kı	5/2	2
Hayattan Sahifeler	fey	Fi	3/2	2
Tarihçe-i Erzurum	kadı	gâ	5/2	2
Ususül Hikem	cid	çb	3/2	2
Tarihçe-i Erzurum	güveyi	Göksu	6/5	5

Each of the 3,092 error character positions is assigned one of five outcomes, which we refer to throughout as the character’s *fate*. “Corrected” means the model produced the right character where OCR had a wrong one. “Untouched” means the model’s output is identical to the OCR error—the model copied it verbatim (this should be read as output-identical-to-OCR rather than deliberately conservative). “Failed attempt” means the model did change the character, but to something still wrong. “New error” applies to positions where OCR was correct but the model introduced a

mistake. “Preserved” means both OCR and the model are correct. The distinction between untouched and failed attempt separates two very different failure modes: a conservative model that leaves errors alone versus an active model that tries and gets it wrong.

The thirteen outer subtypes of Section 4.5 (the whole-span taxonomy of Table 4.2) give a frequency-ordered view of the error landscape. For the per-character fix-rate analysis below we re-decompose multi-character spans into the individual character-level edits they contain, producing the ten inner subtypes shown in Tables 8.1 and 8.2. Two of these subtypes (Turkish diacritic and Punctuation) involve both substitution and omission operations and so receive a row in each table panel, totalling twelve table rows for ten unique subtypes. Inner counts for character omission exceed their outer counterpart in Section 4.5 (755 versus 450) because the inner re-decomposition recovers omission positions that the outer taxonomy assigned to multi-character span rows; the 438 positions inside spans whose inner alignment cannot decompose the edit remain undecomposed in the Unresolvable span row of Table 8.1.

Example 9 illustrates the central pattern that the analysis below quantifies: a circumflex correction that the model handles reliably alongside a vowel error that it leaves unchanged. The model restores the circumflex in *iktizâsına*—a correction in the subtype with the highest correction rate of 42%—but in the same sequence leaves the vowel error *eğerçe* (ground truth: *eğerçi*) untouched. Example 6 showed a complementary contrast: ByT5 achieves perfect correction on an isolated vowel-harmony error (line 156) but copies a complex word-boundary error unchanged and introduces a capitalization error (line 154).

Example 9 (Subtype-level correction fates on a test sequence from *Ususül Hikem*).¹

Row	Key Span	CER	Fate
<i>Circumflex substitution fixed, vowel substitution untouched</i>			
OCR	iktizâsına göre eğerçe ...	4.5%	
ByT5	iktizâsına göre eğerçe ...	1.5%	
Ref	iktizâsına göre eğerçi ...		
	a→â: circumflex substitution → FIXED		Improved
	e→i: vowel substitution → UNTOUCHED		

Because the two models’ subtype-level correction profiles are nearly identical (bootstrap testing confirms a statistically significant difference only for circumflex substitution), we present the unweighted model’s results throughout this section. Confidence-weighted differences are summarized at the end.

Table 8.1 Character-level correction outcomes for substitution errors by the unweighted ByT5 model (2,020 error characters). See text for confidence-weighted comparison.

Error subtype	N	Corr.	Untch.	Failed	Intr.	Net
<i>Turkish-specific substitutions</i>						
Circumflex substitution	245	103	136	6	32	+71
Dotted/dotless i substitution	74	4	67	3	2	+2
Turkish diacritic substitution	45	5	40	0	7*	−2
Vowel substitution	176	13	158	5	10	+3
<i>Other substitutions</i>						
Other character substitution	729	61	627	41	28	+33
Unresolvable span	438	16	407	15	0	+16
<i>Formatting substitutions</i>						
Case substitution	191	62	127	2	33	+29
Punctuation substitution	122	11	91	20	11*	0
All substitutions	2,020	275	1,653	92	123	+152

Net = Corrected − Introduced; untouched and failed-attempt characters do not contribute to the net count. *Introduced counts for Turkish diacritic substitution and Punctuation substitution include both substitution and omission operations by the model.

1. The full 66-character reference contains a third OCR error not visible in the table — a missing space (*vâki’olmuşdur* instead of *vâki’ olmuşdur*). ByT5 corrects this missing space alongside the circumflex; the reported 4.5% baseline and 1.5% corrected CER values include all three errors.

Table 8.2 Character-level correction outcomes for omission errors by the unweighted ByT5 model (1,072 error characters).

Error subtype	N	Corr.	Untch.	Failed	Intr.	Net
Character omission	755	28	719	8	39	−11
Turkish diacritic omission	93	3	89	1	0	+3
Spacing omission	100	22	77	1	20	+2
Punctuation omission	124	18	106	0	0	+18
All omissions	1,072	71	991	10	59	+12

Net = Corrected − Introduced.

8.1.1 Substitution And Omission Patterns

The clearest pattern is the model’s conservatism. Of the 3,092 error characters, the unweighted model copies 85.5% verbatim from the OCR input, corrects 11.2%, and actively changes but fails on only 3.3%. For every error character the model successfully corrects, roughly eight others pass through unchanged. Failed attempts account for only 102 of 3,092 characters, so the model’s changes are correct much more often than wrong. The problem is not inaccurate correction; it is that correction is rarely attempted.

Turkish-specific substitutions. Circumflex substitution is the subtype the model corrects most often, at 42% (Table 8.1), as illustrated in Example 9. The other Turkish-specific substitution subtypes are corrected at far lower rates: dotted/dotless *i* at 5.4% and vowel at 7.4%, comparable to generic single-character errors, while Turkish diacritic substitution is corrected at 11.1% on a small base of 45 characters, with a net of −2. The circumflex result carries a book-specific confound: *Arnavutlar Solyotlar* achieves 61% circumflex correction while *Tarihçe-i Erzurum* manages only 20%, a pattern consistent with *Arnavutlar*’s dominant share of training sequences. The 42% circumflex rate, together with a net gain of +71 characters, identifies this subtype as the primary candidate for selective correction; all other subtypes have net gains below +33.

Other substitutions. Other character substitution and Unresolvable spans together account for 1,167 error characters with a combined correction rate of only 6.6%. The model treats these largely as copy-through.

Formatting errors. The formatting subtypes (case substitution at 32.5% and

spacing omission at 22.0%) are corrected at higher rates than most Turkish-specific substitutions, despite being assigned the highest removal weight in the synthetic-data scheme (Section 6.2).

Book-level concentration. The aggregate net improvement concentrates in a single book. *Arnavutlar Solyotlar*, which supplies the majority of training sequences, accounts for 216 of the model’s 346 corrections and a net gain of +152 characters. The remaining three books collectively net only +12 characters, with *Tarihçe-i Erzurum* net negative due to a high rate of newly introduced errors. Training exposure is the primary determinant of correction ability.

Omission errors. We use “omission” to describe OCR errors where a ground-truth character is absent from the output, distinguishing this from “deletion” as an edit operation in the alignment algorithm. When OCR omits a character, nothing in the model’s input marks the gap; restoring it requires generating the missing character from context alone. The overall omission correction rate is 6.6% as shown in Table 8.2, with character omissions at just 3.7% and 95% of those errors passing through unchanged. Spacing omission at 22% and Punctuation omission at 14.5% fare better because the model can rely on local boundary cues rather than linguistic reconstruction. The omission panel nets only +12 characters, meaning attempts at character restoration are nearly as likely to introduce new problems as to fix existing ones.

Span-level patterns. Multi-character spans improve only when they are short, and the longest and most structurally complex error spans are entirely outside this model’s correction capability. Of the 595 multi-character error spans, 535 are unchanged, 38 improve, 15 achieve perfect correction, and 7 are degraded. Improvement concentrates in short spans of 2–3 characters, where 11.6% show some improvement. None of the 10 spans of length seven or more improve under the unweighted model, and only 9 of 190 multi-character omission spans show any improvement at all.

8.1.2 Introduced Errors And Confidence Effects

Newly introduced errors fall in the same subtypes the corrector most often edits. The unweighted model introduces 182 errors in total; the three largest contributors are character omissions (39), where the model deletes a previously correct character; case substitution events (33); and circumflex additions (32), where the model inserts a circumflex where none belongs. The remaining 78 are distributed across the other subtypes. This distribution mirrors the correction-rate ranking: circumflex and case

are also the two best-corrected subtypes. The subtypes where the model is most active in making changes are also where it makes the most mistakes, suggesting a precision/recall trade-off at the subtype level. For Turkish diacritics, this trade-off tips negative: the model introduces 7 errors against only 8 corrections, yielding essentially no net benefit despite Turkish diacritics being among the subtypes assigned the lowest removal weight in Section 6.2.

Correction rates increase monotonically with OCR confidence: 12.2% for sequences with minimum token confidence below 0.3, rising to 30.1% for those above 0.9. This pattern reflects data difficulty rather than a trainable parameter. Low-confidence sequences are harder to correct because their dense, overlapping errors create contexts where the model cannot reliably identify what needs to change; high-confidence sequences contain isolated errors in otherwise clean contexts, which are exactly the conditions under which a conservative corrector excels. Newly introduced errors show a significant negative correlation with confidence (Spearman $\rho = -0.149$, $p < 0.001$), so low-confidence sequences suffer both the lowest correction rates and the highest damage rates. The rank correlation between confidence and correction rate is positive but non-significant ($\rho = 0.046$, $p = 0.251$), indicating the relationship is better characterized as a threshold effect—very high-confidence sequences are substantially easier—than a continuous gradient.

Confidence-weighted comparison. The confidence-weighted model’s subtype-level correction profiles are nearly indistinguishable from the unweighted model’s. Bootstrap testing across all subtypes finds a statistically significant difference only for circumflex substitution (unweighted 42.0% vs. confidence-weighted 39.2%, $\Delta = 2.8$ pp, $p = 0.014$); four subtypes have identical correction counts. The confidence-weighted model introduces 176 new errors versus 182 for the unweighted model, and on *Tarihçe-i Erzurum* introduced errors fall from 58 to 45. The confidence-weighted correction rate pattern across confidence levels is identical. These results confirm that confidence weighting’s primary benefit is restraint on overcorrections rather than improved correction rates.

8.1.3 Implications

The analysis points to four limits on the practical scope of selective correction for this model.

1. The overwhelming conservatism (85–86% of error characters pass through

unchanged) means the model’s default behaviour is already highly selective, attempting changes conservatively rather than indiscriminately.

2. Only circumflex substitutions are corrected at a rate of 42% that makes targeted application clearly beneficial; all other Turkish-specific subtypes are corrected at 5–8% (dotted/dotless i substitution 5.4%, vowel substitution 7.4%, Turkish diacritic 5.8% combining the substitution and omission rows of Tables 8.1–8.2), which is too low for reliable correction.
3. Omission errors and long multi-character spans are harder to correct: the model cannot generate characters absent from its input, and spans of seven or more characters never improve.
4. Confidence weighting does not change which error families are viable correction candidates.

These findings align with the conclusion of the following section: detection is valuable for locating errors and potentially routing them to specialized correctors, but the general-purpose corrector is reliable only for a narrow subset of error types.

8.2 Quality Estimation For Selective Correction

The error-type analysis (Section 8.1) showed that the corrector overwhelmingly preserves the OCR input, intervening on only a small fraction of error characters. This conservatism limits aggregate improvement but also means that most of the corrector’s interventions are beneficial. The 10% sequence degradation rate (Table 6.2) nevertheless raises a practical question: can detection signals predict which sequences will be harmed, enabling selective suppression of overcorrections? The confidence-weighted training variant reduced degradations at training time (Section 6.4). Here we investigate whether comparable gains can be achieved by selectively applying ByT5 corrections at inference time using detection-derived features. All correction experiments in this section use the unweighted ByT5 model from Chapter 6.

We first establish the ceiling via two oracle analyses. The *sequence-level oracle* applies the optimal per-sequence decision: accept the ByT5 correction when it reduces CER, otherwise retain the original. Formally,

$$\mathbf{y}_{\text{oracle}}^* = \begin{cases} \hat{\mathbf{y}} & \text{if } \text{CER}(\hat{\mathbf{y}}, \mathbf{r}) \leq \text{CER}(\mathbf{x}, \mathbf{r}), \\ \mathbf{x} & \text{otherwise,} \end{cases} \quad (8.1)$$

where $\mathbf{x}$ is the OCR output, $\hat{\mathbf{y}}$ the ByT5 correction, and $\mathbf{r}$ the reference. On the 828-sequence test set, this yields $\text{CER} = 9.34\%$, a headroom of 0.27 pp over the unweighted baseline. The *span-level oracle* makes independent accept/reject decisions per edit span, achieving $\text{CER} = 9.20\%$ —an additional 0.14 pp that quantifies the cost of sequence-level granularity. Per-book analysis reveals heterogeneous headroom: Tarihçe-i Erzurum exhibits the highest degradation rate (26.7%) and the largest oracle gain (0.70 pp), while Ususül Hikem shows only 9.3% degradation and 0.17 pp headroom.

To attempt to realize this headroom, we formulate selective correction as a binary Quality Estimation (QE) problem. Given an OCR sequence $\mathbf{x}$ and its ByT5 correction $\hat{\mathbf{y}}$, we train a classifier f to predict whether applying $\hat{\mathbf{y}}$ will reduce sequence-level CER. The selection rule is

$$\mathbf{y}^* = \begin{cases} \hat{\mathbf{y}} & \text{if } f(\mathbf{z}) \geq \theta, \\ \mathbf{x} & \text{otherwise,} \end{cases} \quad (8.2)$$

where $\mathbf{z} \in \mathbb{R}^{17}$ is the feature vector and θ is a CER-optimal threshold fitted on the development set.

The 17 features fall into two families. **Detector features** (13) are aggregated from the token-level error probabilities $\{\hat{p}_i\}_{i=1}^n$ produced by Method 3 (confidence-fused BERT, canonical variant: $F_1 = 0.647$, $\text{ROC-AUC} = 0.844$; Table 7.6): four summary statistics ($\max_i \hat{p}_i$, $\text{mean}_i \hat{p}_i$, $\sum_i \hat{p}_i$, top- k mean with $k=3$); three count features ($|\{i : \hat{p}_i > t\}|$ for $t \in \{0.3, 0.5, 0.7\}$); three span-level features (number of contiguous flagged spans, mean span length, maximum span length); sequence length n ; and two normalized variants ($\sum \hat{p}_i / n$, $|\{i : \hat{p}_i > 0.5\}| / n$). **Edit features** (3) characterize the proposed correction without reference to the detector: character-level edit distance between the input $\mathbf{x}$ and the candidate correction $\hat{\mathbf{y}}$, word-level edit count, and the character-length ratio $|\hat{\mathbf{y}}|/|\mathbf{x}|$. The ‘all’ configuration additionally includes a single sequence-length meta-feature, totalling 17 features; the edit-only ablation excludes it so that no detector-side or meta signal contaminates the comparison.

We train two classifier families. *Logistic regression* (LR) with L_2 regularization searches over $C \in \{0.01, 0.1, 1.0, 10.0\}$. *LightGBM* (LGB) uses 100 estimators, maximum depth 3, and minimum 20 samples per leaf. Each family is evaluated on three feature subsets (edit-only, detector-only, all), yielding six configurations. Training uses the 852-sequence development set (367 produce non-identity corrections; positive rate 72.2%) with GroupKFold cross-validation ($k=4$, groups = book). Out-of-fold (OOF) predictions are concatenated to compute development-set AUC; the CER-

optimal threshold θ is selected on dev and applied to the 828-sequence test set (positive rate 65.1%). As a non-learned reference, we also evaluate a *rule baseline* that accepts a correction when a composite detection score—combining the character-level edit count, the summed token error probability $\sum_i \hat{p}_i$, and the count of high-probability tokens—exceeds a single CER-optimal threshold ($\theta_{\text{rule}}=1.15$) fitted on the development set.

8.2.1 Experiments & Results

All trained QE configurations achieve OOF AUC below 0.5: the values range from 0.383 (logistic regression with edit-only features) to 0.494 (LightGBM with detector-only features), and LightGBM consistently outperforms logistic regression at matched feature sets (Table 8.3). Every configuration selects a non-zero CER-optimal threshold on the development set, but no configuration reduces test CER below the unweighted baseline of 9.61%. The only configuration that exactly reproduces the unweighted baseline is logistic regression with detector-only features, whose development-optimal threshold filters no test sequences in practice; the remaining configurations all degrade test CER slightly (9.68% to 9.97%) while typically reducing degraded-sequence counts at the cost of lost improvements (Table 8.3).

No feature family provides a usable signal: edit-only, detector-only, and combined-feature configurations all sit near or below random, indicating that neither edit-distance descriptors nor detection $P(\text{error})$ aggregates discriminate beneficial corrections from harmful ones at the sequence level.

The rule baseline achieves test CER = 9.60% at its CER-optimal operating point ($\theta_{\text{rule}}=1.15$) with 78 degradations, essentially matching the unweighted baseline (9.61%, 79 degradations) rather than improving on it.

At the span level, the edit-mix oracle analysis evaluates whether available features can predict per-edit quality. The detection model’s maximum $P(\text{error})$ over each edit span, pooled across the WordPieces that overlap the span, achieves ROC-AUC of 0.533 (essentially random), while the ByT5 decoder’s 10th-percentile log-probability within the same span achieves ROC-AUC of 0.690.

Table 8.3 Quality estimation results for selective correction.

Model	Features	OOF AUC	Test CER (%)	Degraded
<i>Unweighted baseline</i>		—	9.61	79
<i>Confidence-weighted training</i>		—	9.66	64
LR	Edit (3)	0.383	9.71	79
LR	Detector (13)	0.386	9.61	79
LR	All (17)	0.396	9.73	77
LGB	Edit (3)	0.402	9.68	74
LGB	Detector (13)	0.494	9.97	55
LGB	All (17)	0.484	9.75	63
<i>Rule (CER-opt, $\theta_{rule}=1.15$)</i>		—	9.60	78
<i>Sequence oracle</i>		1.000	9.34	0
<i>Span oracle (greedy)</i>		—	9.20	—

8.2.2 Discussion

The usual motivation for two-stage post-OCR systems is that detection and correction are complementary: detection identifies *what* to fix, and correction determines *how*. Our results reveal a subtler picture. Detection accurately identifies erroneous tokens ($F_1 = 0.647$ at token level), but this information does not predict whether the downstream corrector will improve or worsen the sequence. The detector’s task (“is this token wrong?”) and the correction-selection task (“will this correction help?”) are different problems with different optimal features.

This finding reframes the role of detection in post-OCR processing: detection is valuable for *locating* errors and potentially for routing them to specialized correctors, but not for deciding whether to apply a general-purpose corrector. The corrector-side signal (logprob ROC-AUC = 0.69) suggests that future Quality Estimation efforts should focus on corrector-intrinsic features (decoding entropy, attention patterns, alternative hypotheses) rather than detector outputs. The low oracle headroom (0.27 pp) further confirms that the unweighted corrector is already operating near its quality ceiling on this dataset, leaving little room for inference-time refinement of any kind.

Several design choices could in principle bias the QE results toward the null. The training set is small (852 dev sequences, of which only 367 produce non-identity corrections), and GroupKFold with 4 book-level groups yields folds of unequal size.

Class balance differs between dev (positive rate 72.2%) and test (65.1%), so OOF-tuned thresholds may not transfer optimally. Despite these caveats, the uniformly near-chance AUC across all feature sets and model families (Table 8.3) suggests a genuine absence of signal rather than a methodological artifact.

The confidence-weighted training variant, which achieves the same degradation reduction without inference-time overhead (Section 6.4), validates the alternative strategy of embedding risk management in training rather than inference. Section 4.5.3 reported a structural alignment between high OCR confidence and the substitution subtypes a corrector could plausibly fix; this alignment is the empirical underpinning of the negative QE result reported here.

8.3 Qualitative Comparative Error Analysis

The preceding analyses quantified correction rates across error subtypes and demonstrated that detection scores cannot predict correction quality at the sequence level. To understand the mechanisms behind these aggregate findings, we trace detection and correction predictions on individual test-set lines, exposing complementary strengths and weaknesses (additional worked examples by category in Appendix C, Tables C.5–C.10).

Example 6 already illustrated the three correction-outcome categories on lines 154–156 from *Hayattan Sahifeler* (p. 40). Here we examine how different detectors classify tokens in the same lines. Example 10 traces the subword-level predictions of Method 1 (calibrated confidence thresholding) on line 156, where the word-initial false positive reveals a systematic limitation of confidence-only detection.

Example 10 (Detection method comparison on test line 156, *Hayattan Sahifeler*).

Token	Raw	$\hat{p}$	True	Result
ç	.444	.564	0	FP
##irk	1.00	.284	0	TN
##ef	.998	.353	1	TP
##ıyla	.609	.541	1	TP
hemen	1.00	.049	0	TN
koş	1.00	.202	0	TN
##tu	1.00	.132	0	TN
.	1.00	.049	0	TN
Confusion: TP = 2, FP = 1, FN = 0, TN = 5. P = 0.667, R = 1.0.				

The subword-level tokenization exposes a subtlety invisible at the word level: the word-initial subword “ç” receives low raw confidence (0.444) despite being a correctly recognized character, because the decoder is uncertain at the start of a rare word. Beta calibration maps this to $\hat{p}=0.564 > \theta$, producing a **false positive**. The erroneous subwords “##ef” and “##ıyla” are correctly flagged (TP), while all tokens of the correct words “hemen” and “koştu” remain below threshold (TN). The canonical Method 3 variant reaches F_1 of 0.647 against Method 1’s 0.577 on the test set (Table 7.6); much of the gap comes from suppressing false positives on contextually plausible WordPieces, the failure mode visible above for ‘ç’.

The more challenging case is line 154 (Example 6), where ByT5 *degraded* performance: the OCR output “tekmi Çifti” should be “tek mi çift mi” (word boundary and casing errors), but ByT5 only capitalized “t→T” without splitting the fused tokens. Here, detection correctly flagged the erroneous region, but the downstream corrector failed to apply the correct edit—demonstrating that accurate detection is necessary but not sufficient for successful correction, and illustrating why the quality-estimation experiment in Section 8.2 yields a negative result: the detector identifies the problem but has no information about whether the corrector can solve it.

8.4 Comparison With External Tools

Chapters 6 and 7 evaluated the corrector and the detectors against the baseline, the raw AKİS output. This section compares the corrector with external systems under the error-rate definitions of Section 5.2: two modern-Turkish toolkits, scored on the full 828-sequence test partition; and a second OCR recognizer and a large language model, each scored on a smaller four-page probe of one page per book. For the toolkit comparison we report raw CER throughout, alongside a format-normalized variant (Turkish-aware lower-casing, punctuation stripping, and whitespace collapse, applied identically to every system). In each case the external system does not lower CER below the uncorrected baseline on its own evaluation set; the two ByT5 models are the only ones that do.

8.4.1 Modern-Turkish Spell Checkers And Morphology

We evaluate the two most capable correction-style tools available for Turkish: *Zemberek*’s spell checker in its best-documented mode (context ranking with a 2-gram language model), and *TULAP*’s neural Deasciifier, a tool that restores Turkish diacritics (ç, ğ, ı, ö, ş, ü) to text written in plain ASCII and its only text-to-text corrector, run per line. Each toolkit uses the configuration its documentation recommends for correction, and positive controls confirm that both behave correctly on modern Turkish before we apply them to Ottoman. Both are scored with the character- and word-error routines and the test partition used for the ByT5 models. The remaining toolkit configurations appear in Appendix E. Table 8.4 reports the resulting error rates for every system on the full test set.

Table 8.4 Character- and word-error rates (raw and format-normalized) and sequence-level outcomes on the 828-line test set.

System	CER raw	CER norm	WER raw	WER norm	Imp / Deg / Unch
Baseline (raw OCR)	10.51	9.27	33.10	28.31	0 / 0 / 828
ByT5 unweighted	9.61	8.64	28.72	25.33	244 / 79 / 505
ByT5 confidence-weighted	9.66	8.71	29.00	25.59	238 / 64 / 526
TULAP Deasciifier (per-line)	10.55	9.32	33.33	28.53	4 / 22 / 802
Zemberek SpellChecker (LM-ranked)	12.73	10.12	42.48	31.23	53 / 472 / 303

A freely available, general-purpose Turkish toolkit does not substitute for an Ottoman-specific corrector. Every *Zemberek* configuration, including its strongest spelling mode, leaves the text worse than no correction at all; a paired McNemar test against the unweighted ByT5 puts the difference at $p \approx 9.5 \times 10^{-89}$. A per-character decomposition over the 3,092 test-set error characters shows why. *Zemberek* repairs

154 of them (5.0%), almost all circumflex substitutions, and introduces 928 new error characters where it rewrites Ottoman, Arabic, and Persian forms into their nearest modern-Turkish lexeme. This is how a modern-Turkish lexicon and language model behave on late-Ottoman text, and it is why the task needs a domain-adapted model.

TULAP provides no spell checker or normalizer. Its only correction tool is a character-level neural deasciifier that emits one output character per input character, and per line it barely acts: 95.4% of lines come back byte-identical and CER moves from 10.51% to 10.55%. The one-to-one constraint is structural. The deasciifier can change diacritics and case but cannot repair the insertions, deletions, and multi-character spans that make up most of the error inventory, so it corrects only 6 of the 3,092 error characters. Its documented workflow ASCII-folds the input before restoring diacritics; on text that already carries diacritics this strips correct characters and re-guesses them, raising CER to 14.44%, so we report it only as a diagnostic in Appendix E.

The morphological probe is a reference-free diagnostic, and we read it under two constraints. First, the two analyzers use different tokenization and unknown-token conventions, so their parse rates are not mutually comparable; within each analyzer a system is interpreted only relative to the ByT5 outputs and to the gold reference measured under that analyzer. Second, a higher parse rate does not imply a more accurate transliteration, because both analyzers are built for modern Turkish and classify correct Ottoman, Arabic, and Persian forms as unknown even in the gold reference. These two constraints are what make the result legible. *Zemberek*’s normalizer parses at 0.931, above the gold reference’s 0.855 under the same analyzer, while recording the highest character error rate in the study (15.27%); the second analyzer gives the same ordering (0.902 against 0.816). A system that exceeds the reference’s own parse rate is measuring conformity to modern Turkish, not transliteration fidelity. The reference’s own circumflex-bearing tokens, parsed at 56.3% and 10.1% against 87.9% and 87.5% for its plain tokens (Figure 8.1), show where the proxy stops tracking quality: it penalizes the Ottoman-specific forms it should accept and cannot stand in for transliteration fidelity.

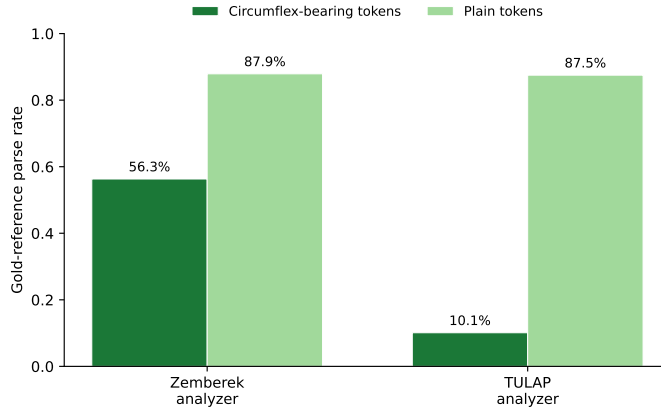


Figure 8.1 Gold-reference morphological parse rate for circumflex-bearing versus plain tokens, under both analyzers.

The full set of toolkit configurations, with raw and format-normalized CER and WER, sequence-level outcomes, per-book CER, and paired tests against the unweighted ByT5, is given in Appendix E.

8.4.2 Cross-Recognizer Generalization (*Transkribus*)

So far the corrector has only seen errors from AKIS, the recognizer used throughout this thesis. To check whether it helps on a different recognizer’s output, we first remove the confounds that a recognizer change introduces. We score a second recognizer’s output through the same cleaning and metric pipeline: a four-page probe, one page per book, recognized by *Transkribus*’s *PyLaia* model. The same text-cleaning step the in-domain output receives is applied before scoring against the reference, which removes any cleaning asymmetry. The four-page scope follows the second recognizer’s free-tier limit; taking one page per book lets all four publication eras (1732–1920) appear, which gives a first read on whether transfer depends on book or period. Because the second recognizer transcribes the full page, we first align its output to the reference’s line coverage: nine lines with no test-set counterpart are removed, namely four body lines that the in-domain pipeline transcribed but dropped during dataset construction, and five short marginal numerals (folio and page numbers) that the reference omits. Two further lines the in-domain segmenter never produced were checked and have no counterpart in the second recognizer’s output, so they need no adjustment.

Table 8.5 *Transkribus* probe, page-concatenation strategy,² per-page character error rate (%), after aligning the second recognizer’s output to the reference’s line coverage. AKİS-cleaned is the in-domain recognizer under identical cleaning.

Book (page)	AKİS-cleaned	Transkribus-cleaned	Transkribus + ByT5	Gap to AKİS
Hayattan Sahifeler	4.45	24.28	23.56	+19.11
Tarihçe-i Erzurum	11.25	26.51	25.49	+14.24
Ususül Hikem	18.44	41.17	41.07	+22.63
Arnavutlar Solyotlar	12.64	34.52	32.49	+19.85
Mean	—	31.62	30.65	—

Table 8.5 reports the per-page error rates. Each page is compared along three quantities: the in-domain recognizer after cleaning, the second recognizer after the identical cleaning, and that second recognizer’s output after correction, all scored on the lines the test-set reference covers. The second recognizer is much weaker on these pages, at 24.3–41.2% character error against 4.5–18.4% for the in-domain recognizer, and applying the same symmetric cleaning shifts its error by a mean of 0.06 percentage points, so the gap is a real recognition-quality difference and not an effect of cleaning. Correction, trained only on the in-domain recognizer’s errors, lowers character error on all four pages, by 0.10 to 2.03 points (mean 0.97). The reduction is largest on the Arnavutlar Solyotlar page (2.03 points), the book that dominates the training corpus, and falls to 0.10–1.02 points on the other three books, whose training shares are smaller and close to one another. The per-page improvement therefore follows training exposure more than publication era. These reductions are small against the 14–23 point gap that still separates the corrected second recognizer from the in-domain baseline: the corrector refines a recognizer’s output, but it does not turn a weak recognizer into a strong one.

The probe is small, and we report it descriptively. The free-tier page cap limited the study to four pages, one per book, so that the corpus’s four publication eras (1732–1920) each appear and we can see whether transfer depends on book or period; with a single page per book the figures are illustrative and we report no inferential test on the per-page CER means. That alignment lowers the second recognizer’s mean character error by about four points, and we state it because the adjustment works in that system’s favour.

2. An alternative per-line pairing by vertical position is far noisier (page CER 45–133%) because it is sensitive to how the two systems segment lines; we therefore report the page-concatenation strategy.

8.4.3 Large Language Model (LLM)

A large language model offers a different route to correction than a fine-tuned one, and a bounded pilot lets us check whether it would make the domain-adapted corrector unnecessary. We evaluate one such model, GPT-5.5 Thinking, prompted zero-shot with a fixed instruction: no fine-tuning, no few-shot examples, no chain-of-thought, and no per-line retries. The model receives only the cleaned recognizer output, never the reference, and the prompt names the forms it must preserve: circumflex letters, apostrophes, and hyphenated Ottoman compounds. We run it on the 85 lines of the four probe pages, one page per book, and score every output offline with the character- and word-error routines and the error-rate definitions of Section 5.2, on the same line frame used for the ByT5 models. The identical prompt is issued in two independent fresh sessions³ with memory, custom instructions, and tools disabled, so that run-to-run variation is visible rather than hidden in a single draw. Table 8.6 reports the per-line comparison.

Table 8.6 Comparison on the 85 test lines of the four probe pages (one page per book). Inputs and references are the cleaned AKIS-pipeline outputs; status threshold $|\Delta\text{CER}| < 0.001$. For the per-line mean of the two GPT-5.5 runs, the per-line CER is averaged across the two runs and each line re-classified against the baseline; the improved/degraded/unchanged counts are therefore a re-classification of the averaged outputs, not the arithmetic mean of the two runs’ counts.

System	CER	WER	Imp / Deg / Unch
Baseline (cleaned OCR)	11.80	37.33	0 / 0 / 85
ByT5 unweighted	11.09	33.47	29 / 13 / 43
ByT5 confidence-weighted	11.15	33.70	27 / 11 / 47
GPT-5.5 Thinking (run 1)	11.34	32.96	29 / 10 / 46
GPT-5.5 Thinking (run 2)	12.56	36.60	19 / 10 / 56
GPT-5.5 Thinking (per-line mean of two runs)	11.95	34.78	30 / 14 / 41

The pilot does not support replacing the domain-adapted corrector with a large language model. Averaged over two runs the model reaches 11.95% character error, above the 11.80% baseline, while the unweighted ByT5 corrector reaches 11.09% on the same lines: the corrector lowers error and the model does not. The two runs also differ from each other, by more than noise. They give 11.34% and 12.56%, and a single run would have supported opposite conclusions, since run 1 alone improves over the baseline ($p = 0.003$) while run 2 alone does not ($p = 0.14$). At $n = 85$ the

3. The two sessions used identical settings (memory off, no custom instructions, no tools) and the same locked prompt. They agreed byte-for-byte on 53 of the 85 lines (62.4%); a paired exact-binomial test on per-line improved-vs-baseline outcomes rejects equal rates between the runs at $p \approx 0.006$.

bootstrap intervals overlap across all corrected systems,⁴ so this slice cannot rank them numerically; what it establishes is that a zero-shot model is unstable across runs and, averaged over them, no better than leaving the recognizer output alone. We read the slice as a bounded sanity check rather than evidence that the model fails outright. This matches prior reports that zero-shot post-correction does not reliably help historical OCR and that fine-tuned models remain competitive on low-resource scripts (Boros et al., 2024; Bourne, 2024; Greif et al., 2025; Gunathilake et al., 2025; Kanerva et al., 2025).

The failures recur across runs rather than appearing at random. Two of the three modes reproduce in both runs: a substitution on an already-correct input (**uğraşarak** to **girerek**) and a formatting-rule violation that rewrites the dialogue dash to an asterisk. The third is run-specific, an antecedent that run 2 fabricates and prepends to a complete line. On the Arabic-quotation line of Table C.5 (Appendix C), neither run recovered the gold transliteration, on a span the local context gives no basis to reconstruct. The two reproduced modes are a hallucination on correct input and an instruction-following violation; the run-specific one mirrors, at the level of a single line, the cross-run variance seen in the aggregate.

4. These 85 lines are the four probe pages of Section 8.4.2, one page per book. The 95% bootstrap intervals for the corrected-system means span roughly 9–14% and overlap; we therefore report paired tests and the mean of two runs to separate stochastic noise from systematic effects. A corpus-scale, multi-model, prompt-engineered language-model evaluation is left to future work.

9. SUMMARY

The main findings of this thesis are the following. First, byte-level correction is effective, but only on some error types. Second, OCR confidence makes the corrector more cautious: it lowers the number of sequences the corrector makes worse, from 79 to 64, without improving overall CER. Third, detection and correction address different aspects of the post-OCR problem rather than composing into a sequential pipeline. The evidence for each follows.

Single-stage correction. This thesis evaluated fully automatic correction methods for transliterated Ottoman Turkish produced by AKİS, a frozen TrOCR-based recognizer, on a corpus of 8,294 sequences comprising 102,000 tokens from four printed books spanning 1732 to 1920. Baseline test-set error rates stood at 10.51% CER and 33.10% WER. The ByT5 corrector reduced test CER to 9.61% and WER to 28.72%. Most test sequences were unchanged under correction, with a smaller fraction degraded as expected for any general-purpose corrector; per-sequence counts are reported in Table 6.2. An error-type decomposition of the 3,092 test-set error characters revealed that the corrector copies 85.5% of errors verbatim and corrects 11.2%, with circumflex substitutions the sole subtype corrected at a rate of 42%, yielding clear net benefit. All other Turkish-specific subtypes were corrected at 5–8% (combined substitution+omission rates where both panels apply), and omission errors at 6.6%, confirming that the corrector helps on only a few error types. Corrections concentrated in the training-dominant book, Arnavutlar Solyotlar, which accounted for 216 of 346 total corrections, suggesting that training exposure is the primary determinant of correction ability. A comparison against external systems—two modern-Turkish toolkits on the full test set, and a second OCR recognizer and a zero-shot large language model on a four-page probe—found that none lowered CER below the uncorrected baseline on its evaluation set, whereas both ByT5 variants did (Section 8.4).

Confidence weighting. Confidence-weighted training did not improve corpus-level CER but reduced degraded sequences by 19%, from 79 to 64 sequences, a statistically significant reduction (McNemar’s exact test, $p = 0.017$). In this setting, confidence

was most useful when embedded in training rather than applied at inference time. This claim follows from two findings reported above: the training-time reduction in degraded sequences (Section 6.4, Table 6.2), and the quality-estimation result that detection-side scores cannot predict per-sequence correction outcomes well enough to improve CER over the unweighted corrector (Section 8.2, Table 8.3). The best out-of-fold AUC across all configurations is 0.494, at chance, and no configuration achieves statistically meaningful discrimination between corrections that improve sequences and those that degrade them.

Detection. Three detection methods were evaluated at the token level: Method 1 (calibrated confidence thresholding) with $F_1 = 0.577$, Method 2 (BERT-only contextual classifier) reaching $F_1 = 0.640$, and Method 3 (confidence-fused BERT, canonical concatenation variant with beta-calibrated confidence) at $F_1 = 0.647$. Most of the gain comes from adding contextual modelling to confidence-only detection; fusing confidence back into the contextual model adds little further, suggesting substantial overlap between the two signals. At the token level, a paired bootstrap on the canonical 10,393-token grid confirms that both Method 2 and Method 3 significantly outperform Method 1 ($\Delta F_1 \approx +0.06$ to $+0.07$, all 95% CIs exclude zero, $p < 0.001$); the additional gain from fusion over BERT-only context is not significant ($p = 0.151$). A sequence-level paired bootstrap, however, found no significant pairwise difference at the per-sequence level at which the corrector consumes detection (all 95% CIs on ΔF_1 include zero), reflecting an operating-point trade-off: methods that are more precise at token level flag fewer sequences and lose recall under OR-aggregation. A quality estimation experiment found that detection scores cannot predict whether a given correction will help or harm, with near-chance AUC across all configurations, confirming that there is little headroom for inference-time selective application of this corrector’s outputs (oracle gain ≤ 0.27 pp).

The scope of automatic correction is bounded by the model’s training exposure and the structural properties of the errors it encounters.

10. LIMITATIONS

This study uses one frozen AKIS recognizer checkpoint: epoch 483, with 4 encoder and 4 decoder layers. The recognizer is used only for inference. The post-processing methods operate on its Latin-script transliteration output, not on the original Ottoman-script page image. Image-level alternatives, such as retraining AKIS, changing the visual recognizer, or re-running OCR with a different model, are outside the experiment. The confidence scores used in calibration, confidence-weighted training, and confidence-based detection also come from this specific AKIS decoder and decoding setup. A different OCR model or checkpoint would require new calibration and could produce different error patterns.

The corpus is limited in domain. It contains four printed matbu/naskh books from the AKIS corpus. These books vary in year, genre, and OCR difficulty, but they do not cover manuscripts, administrative registers, correspondence, verse, or other Ottoman document types. The results should therefore be read as evidence for this four-book printed-text setting rather than as a general claim about all Ottoman Turkish OCR output.

The experiments do not implement a full detect-then-edit pipeline with a separate second-stage corrector. The thesis evaluates single-stage ByT5 correction, confidence-based detection, contextual BERT detection, and confidence-fused BERT detection. It then tests whether detection scores can help decide when to apply the ByT5 correction. This setup measures whether detection can guide the existing single-stage corrector at inference time, but it does not report CER for a separately trained two-stage correction system.

The confidence-fusion experiments cover two fusion architectures (additive and concatenation, both after the BERT encoder) crossed with two confidence-input formulations (raw aggregated confidence and beta-calibrated probability), for four variants total. Other designs may behave differently, including input-layer fusion, attention-layer fusion, auxiliary objectives, or separate confidence streams. The synthetic augmentation procedure has similar limitations. The three removal weights

(0.3 for Turkish-specific subtypes, 1.0 for generic subtypes, 3.0 for formatting subtypes) are hand-chosen and not ablated, and different weights could change which error subtypes are emphasized during training. The presence of synthetic augmentation is ablated directly: a parallel ByT5 model trained on the original correction pairs alone under otherwise-matched conditions reaches 9.84% test CER, against 9.61% for the augmented model, confirming that augmentation contributes to the reported CER reduction. The decision to augment training data is grounded in prior post-OCR work on under-resourced and historical languages, where related strategies have consistently yielded measurable gains (Chapter 2; further discussed in Section 6.1).

Our evaluation of large language models is limited to a bounded zero-shot pilot (Section 8.4.3): a single model, prompted without fine-tuning or few-shot examples, on the four probe pages rather than the full test set. The pilot is a sanity check against the question of whether a modern large language model would supersede a domain-adapted corrector, not a tuned LLM benchmark; a corpus-scale, multi-model, prompt-engineered LLM evaluation remains future work.

The evaluation uses automatic metrics. CER and WER are computed per sequence and macro-averaged, and CER uses the reference length as denominator. These choices are used consistently, but other post-OCR work may use different denominators or averaging schemes. A max-length denominator changes the exact CER values slightly, although the main direction of the findings remains the same. The final reported outcomes are CER, WER, detection metrics, calibration metrics, and sequence-level improved/degraded/unchanged counts.

The use of automatic metrics does not mean that the data were processed without human inspection. During preprocessing, prediction–reference pairs were manually checked and cleaned to remove page numbers, citation markers, formatting differences, and other artifacts that made alignment unreliable. The synthetic-data priorities were also shaped by the difference between surface formatting errors and linguistically meaningful Turkish-specific errors. What is missing is a separate formal human-evaluation protocol. The thesis does not report blinded human ratings of readability, semantic adequacy, or downstream usefulness.

The statistical estimates are tied to one canonical split and one seed. Bootstrap confidence intervals resample the fixed 828-sequence test set. They measure uncertainty over test sequences, not variation across random seeds, training runs, or alternative train/dev/test splits. Per-book analysis is especially limited because the book-level test sets are small and uneven. The per-book results are still useful for diagnosis, but they should be read as descriptive rather than equally powered statistical claims

for each book.

The thesis uses the expert-prepared ground-truth transliterations from the AKİS corpus as the reference standard. This work further cleans the prediction–reference pairs for alignment, but it does not repeat the original AKİS ground-truth construction or validation procedure. Any remaining systematic inconsistency in the reference would therefore be reflected in the evaluation and in the subtype analysis.

The split prevents same-page leakage, because lines from the same source page are assigned to the same train, development, or test partition. The modelling unit is narrower than the split unit. The detection and correction models receive individual OCR lines as input. They do not use neighbouring lines, full-page context, or document-level repetition. As a result, repeated names, repeated phrases, or consistent OCR confusions across a page or book cannot be used by the current models.

The subtype findings are affected by book composition. A large share of the successful corrections comes from *Arnavutlar Solyotlar*, which is also the training-dominant book. The strong result for circumflex substitutions should therefore be interpreted as a finding under this training distribution. A more balanced corpus could produce a different subtype-level correction profile.

The negative result for detection-guided selective correction is limited to the tested selection setup. The experiment uses detector-side features from the confidence-fused detector to decide whether to apply the unweighted ByT5 correction. Under this setup, detection scores do not improve correction selection over the unweighted baseline. This finding does not cover quality estimation based on corrector-side signals such as decoder log-probability, output entropy, or alternative generated hypotheses. It shows that detector-side error probabilities are not enough to decide whether to apply this corrector on this corpus.

The detection thresholds were tuned to maximize token-level F_1 on the development set, matching the headline detection metric reported in Chapter 7. Under this tuning grid, the sequence-level paired bootstrap in Section 7.5 finds no significant pairwise difference between the three methods at the per-sequence level. A separate set of thresholds tuned to maximize sequence-level F_1 on the development set could shift the per-sequence operating points and might change the comparison. We did not perform this re-tuning: doing so would replace the operating point used everywhere else in the thesis (in the per-token tables of Chapter 7, in the QE feature engineering of Section 8.2, and in the example traces of Section 8.3) with a different operating

point used only for the comparison. Holding the threshold fixed at the token-level F_1 -optimal value across both layers of evaluation is the more direct comparison of how the methods behave at the operating point the rest of the thesis already commits to.



11. CONCLUSION

This thesis set out to test how much of the post-OCR correction problem on transliterated Ottoman Turkish can be addressed with confidence-aware byte-level methods on a frozen recognizer. The questions that motivated this work were practical: how far can fully automatic post-OCR correction be pushed in a low-resource, non-Latin setting where retraining the recognizer is not feasible, and what is the most useful role of OCR confidence within that pipeline.

This thesis makes three contributions beyond the specific results. Methodologically, it treats OCR confidence as a training-time risk-control signal rather than as a ranking input or an inference-time selection signal; this single framing accounts for both the degradation reduction observed under confidence-weighted training and the absence of any inference-time selective-correction gain. Empirically, the per-subtype decomposition of correction outcomes in Section 8.1 shows that corpus-level CER can hide near-zero correction rates on most error families, indicating that subtype-decomposed reporting gives a more accurate picture of correction behaviour than corpus-level CER alone. The negative result on detection-guided selective correction in Section 8.2 provides a concrete bound on how far token-level detection scores can carry an existing corrector. The external comparison in Section 8.4 reinforces the value of domain adaptation: neither off-the-shelf modern-Turkish toolkits, a second OCR recognizer, nor a zero-shot large language model surpassed the domain-adapted corrector.

The results point to four directions for future work. On the architecture side, we chose byte-level processing because it handles any character and aligns with character-level OCR errors. This choice does not address long-span errors well: the model improves none of the spans of seven or more characters reported in Section 8.1. Correction units above the byte level, such as word-aware or morpheme-aware decoders or routing long spans to a separate module, are the natural direction. On the data side, the asymmetry between circumflex correction at 42 percent and the other Turkish-specific subtypes at 5 to 8 percent (Section 8.1), together with the per-book correction concentration, identifies controlled augmentation rather than naive per-

subtype specialization as the path to broader coverage. On the quality-estimation side, the detector-versus-corrector ROC-AUC asymmetry in Section 8.2 (0.53 vs. 0.69 over edit spans) makes corrector-intrinsic features the more promising basis for selective correction; the QE Discussion in that section already names this direction, and a thesis-level commitment to it is the natural next step. Finally, on the recognizer side, the calibration map and the confidence-weighted training are both anchored to a single AKIS checkpoint (epoch 483, four encoder and four decoder layers) and a single beam-search configuration; varying the checkpoint or the decoding parameters — alternative early-stopping epochs, different beam widths — would quantify how much of the observed calibration behaviour and degradation-reduction effect is checkpoint-specific.

The pipeline-design lesson behind these results, that uncertainty signals tend to be more useful in shaping what a model learns than in deciding what it should output, may apply to other post-editing tasks under similar data constraints. Confirming that would require testing such tasks directly before treating inference-time selective correction as the default integration point for confidence. For low-resource non-Latin OCR settings where retraining the recognizer is not feasible, this thesis argues that confidence-aware byte-level correction is a defensible pipeline, but only with realistic expectations: aggregate gains are modest and concentrated in particular error families, and they are best obtained where confidence shapes training rather than where it selects at inference.

BIBLIOGRAPHY

- Akcan, A., Bilgin Taşdemir, E. F., Kızılırmak, F., Kuru, M., Öncel, F., & Yanıkoğlu, B. (2022). Akis: Transcription software of Ottoman-Turkish texts using deep learning. *Journal of the Ottoman and Turkish Studies Association*, 9(2), 43–48.
- Berg-Kirkpatrick, T., Burkett, D., & Klein, D. (2012). An empirical investigation of statistical significance in NLP. *Proceedings of the 2012 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL)*, 995–1005. <https://aclanthology.org/D12-1091/>
- Boros, E., Ehrmann, M., Romanello, M., Najem-Meyer, S., & Kaplan, F. (2024). Post-correction of historical text transcripts with large language models: An exploratory study. *Proceedings of the 8th Joint SIGHUM Workshop on Computational Linguistics for Cultural Heritage, Social Sciences, Humanities and Literature (LaTeCH-CLfL 2024)*, 133–159. <https://aclanthology.org/2024.latechclfl-1.14/>
- Bourne, J. (2024). *CLOCR-C: Context leveraging OCR correction with pre-trained language models*. arXiv: 2408.17428. <https://arxiv.org/abs/2408.17428>
- Chiron, G., Doucet, A., Coustaty, M., & Moreux, J.-P. (2017). ICDAR2017 competition on post-OCR text correction. *Proceedings of the 14th IAPR International Conference on Document Analysis and Recognition (ICDAR)*, 1, 1423–1428.
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 4171–4186. <https://doi.org/10.18653/v1/N19-1423>
- Dietterich, T. G. (1998). Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7), 1895–1923. <https://doi.org/10.1162/089976698300017197>
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. *Proceedings of the International Conference on Learning Representations (ICLR)*.
- Efron, B. (1979). Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1), 1–26.
- Greif, G., Griesshaber, N., & Greif, R. (2025). *Multimodal LLMs for OCR, OCR post-correction, and named entity recognition in historical documents*. arXiv: 2504.00414. <https://arxiv.org/abs/2504.00414>

- Guan, S., & Greene, D. (2024). Advancing post-OCR correction: A comparative study of synthetic data. *Findings of the Association for Computational Linguistics: ACL 2024*, 6036–6047. <https://doi.org/10.18653/v1/2024.findings-acl.361>
- Guan, S., Xu, C., Lin, M., & Greene, D. (2024). Effective synthetic data and test-time adaptation for OCR correction. *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 15412–15425. <https://doi.org/10.18653/v1/2024.emnlp-main.862>
- Gunathilake, A., Karunaratna, N., Bandaranayake, T., de Silva, N., & Ranathunga, S. (2025). *LMSpell: Neural spell checking for low-resource languages*. arXiv: 2512.05414. <https://arxiv.org/abs/2512.05414>
- Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 1321–1330.
- Hemmer, A., Coustaty, M., Bartolo, N., & Ogier, J.-M. (2024). ConfBERT: Confidence-enhanced BERT for improved OCR error detection. *Document Analysis Systems (DAS 2024)*, 14994, 213–228.
- Kanerva, J., Ledins, C., Käpyaho, S., & Ginter, F. (2025). OCR error post-correction with LLMs in historical documents: No free lunches. *Proceedings of the Third Workshop on Resources and Representations for Under-Resourced Languages and Domains (RESOURCEFUL-2025)*, 38–47. <https://aclanthology.org/2025.resourceful-1.8/>
- Koehn, P. (2004). Statistical significance tests for machine translation evaluation. *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 388–395. <https://aclanthology.org/W04-3250/>
- Kull, M., Silva Filho, T., & Flach, P. (2017). Beta calibration: A well-founded and easily implemented improvement on logistic calibration for binary classifiers. *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 54, 623–631.
- Li, M., Lv, T., Chen, J., Cui, L., Lu, Y., Florencio, D., Zhang, C., Li, Z., & Wei, F. (2023). TrOCR: Transformer-based optical character recognition with pre-trained models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(11), 13094–13102. <https://doi.org/10.1609/aaai.v37i11.26538>
- Lin, T.-Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). Focal loss for dense object detection. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2980–2988.
- Löfgren, V., & Dannélls, D. (2024). Post-OCR correction of digitized Swedish newspapers with ByT5. *Proceedings of the 8th Joint SIGHUM Workshop on Computational Linguistics for Cultural Heritage, Social Sciences, Humanities and Literature (LaTeCH-CLfL 2024)*, 237–242. <https://aclanthology.org/2024.latechclfl-1.23/>

- Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. *Proceedings of the International Conference on Learning Representations (ICLR)*.
- Nguyen, T. T. H., Jatowt, A., Coustaty, M., & Doucet, A. (2021). Survey of post-OCR processing approaches. *ACM Computing Surveys*, 54(6), 124:1–124:37. <https://doi.org/10.1145/3453476>
- Nguyen, T. T. H., Jatowt, A., Nguyen, N. V., Coustaty, M., & Doucet, A. (2020). Neural machine translation with BERT for post-OCR error detection and correction. *Proceedings of the ACM/IEEE Joint Conference on Digital Libraries (JCDL)*, 333–336.
- Platanios, E. A., Stretcu, O., Neubig, G., Póczos, B., & Mitchell, T. M. (2019). Competence-based curriculum learning for neural machine translation. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 1162–1172.
- Ramírez-Orta, J., Xamena, E., Maguitman, A., Milios, E., & Soto, A. J. (2022). Post-OCR document correction with large ensembles of character sequence-to-sequence models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 36, 11192–11199.
- Rigaud, C., Doucet, A., Coustaty, M., & Moreux, J.-P. (2019). ICDAR 2019 competition on post-OCR text correction. *Proceedings of the 15th International Conference on Document Analysis and Recognition (ICDAR)*, 1588–1593.
- Rijhwani, S., Anastasopoulos, A., & Neubig, G. (2020). OCR post-correction for endangered language texts. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 5931–5942. <https://doi.org/10.18653/v1/2020.emnlp-main.478>
- Schweter, S. (2020). BERTurk – BERT models for Turkish. <https://doi.org/10.5281/zenodo.3770924>
- Şen, U., Bilecen, B., Bilgin Taşdemir, E. F., & Yanıkoğlu, B. (2025). Transcription of Ottoman documents using transformer based models. *Proceedings of the 33rd Signal Processing and Communications Applications Conference (SIU)*, 1–4. <https://doi.org/10.1109/SIU66497.2025.11112382>
- Specia, L., Blain, F., Logacheva, V., Astudillo, R. F., & Martins, A. F. T. (2018). Findings of the WMT 2018 shared task on quality estimation. *Proceedings of the Third Conference on Machine Translation (WMT)*, 689–709. <https://aclanthology.org/W18-6451/>
- Stankevičius, L., & Lukoševičius, M. (2022). Correcting diacritics and typos with a ByT5 transformer model. *Applied Sciences*, 12(5), 2636. <https://doi.org/10.3390/app12052636>
- Xue, L., Barua, A., Constant, N., Al-Rfou, R., Narang, S., Kale, M., Roberts, A., & Raffel, C. (2022). ByT5: Towards a token-free future with pre-trained byte-to-

byte models. *Transactions of the Association for Computational Linguistics*,
10, 291–306. https://doi.org/10.1162/tacl_a_00461



APPENDIX A - SINGLE-STAGE CORRECTION DETAILS

Per-Book Correction Outcomes

Table A.1 reports test-set correction outcomes for both single-stage variants on the four books separately. Aggregate values match the main text (Table 6.1, Section 6.4); the per-book breakdown shows that the small aggregate gain is concentrated in two books and that Tarihçe-i Erzurum is the only book on which unweighted ByT5 has negative mean CER reduction.

Table A.1 Per-book test-set correction outcomes for the unweighted (UW) and confidence-weighted (CW) ByT5 variants. Baseline is the AKİS OCR output before correction. Δ is mean CER reduction (positive = improvement). Imp/Deg columns give percentages of improved and degraded sequences.

Book	n	Base CER	UW CER	UW Δ	CW CER	CW Δ	UW Imp/Deg	CW Imp/Deg
Arnavutlar Solyotlar	555	0.0953	0.0837	0.0116	0.0845	0.0109	28.8/6.7	27.4/5.4
Hayattan Sahifeler	112	0.0606	0.0585	0.0021	0.0601	0.0005	17.0/12.5	17.0/12.5
Tarihçe-i Erzurum	75	0.1351	0.1359	-0.0008	0.1331	0.0021	29.3/26.7	34.7/18.7
Ususül Hikem	86	0.1995	0.1898	0.0096	0.1905	0.0090	50.0/9.3	47.7/7.0
Overall	828	0.1051	0.0961	0.0090	0.0966	0.0085	29.5/9.5	28.7/7.7

The pattern most relevant to Section 6.4 is on Tarihçe-i Erzurum (n=75): unweighted ByT5 has slightly negative mean CER reduction here, while confidence-weighted ByT5 attains a positive mean and cuts the per-sequence degradation rate by about 30% (26.7% to 18.7%). On the lower-baseline-error Arnavutlar Solyotlar and Hayattan Sahifeler the two variants are nearly indistinguishable; on Ususül Hikem both achieve the largest improved-sequence rate, close to 50%.

Identity-Pair Statistics By Book

Table A.2 reports the per-book breakdown of identity training pairs. As described in Section 6.3, an identity pair is created from each non-synthetic training row whose OCR-vs-ground-truth Character Error Rate falls below the inclusion threshold of 0.05; the input and target of an identity pair are both the ground-truth reference. The aggregate identity count is 2,791 of the 6,614 original training rows (42.2%),

matching the value reported in Section 6.3.

Table A.2 Identity-pair counts per book on the training partition. n is the number of original (non-synthetic) training rows in that book; % is the fraction of those rows that produced an identity pair.

Book	n (orig. train)	n (identity)	%
Arnavutlar Solyotlar	4,550	2,196	48.3
Hayattan Sahifeler	792	409	51.6
Tarihçe-i Erzurum	612	151	24.7
Ususül Hikem	660	35	5.3
Overall	6,614	2,791	42.2

The per-book distribution is highly skewed. Hayattan Sahifeler, the cleanest book, supplies an identity-pair rate above 50%; Ususül Hikem, the noisiest, supplies one below 6%. The corrector therefore sees ‘do nothing’ supervision overwhelmingly from low-error books and almost never from Ususül Hikem. This asymmetry is consistent with the per-book degradation pattern in Section 6.4: the books with the most identity-pair exposure during training are also the books on which the trained models are most conservative at inference time.

Synthetic Data Generation Algorithm

The procedure below specifies the synthetic correction-pair generator. It is a deterministic-seeded offline procedure: it is run once to produce a fixed table of synthetic pairs, and that table is used by every downstream training run.

Procedure: synthetic correction-pair generation (per source row).

Require: Source row with OCR text x and ground-truth text y ; per-subtype weights $w_{\text{Turkish-specific}}=0.3$, $w_{\text{generic}}=1.0$, $w_{\text{formatting}}=3.0$; max-variants cap $V_{\text{max}}=2$; random seed 42

- 1: $\mathcal{D} \leftarrow$ all non-equal opcodes from $\text{ALIGN}(x, y)$ (replace, insert, delete) at character level
- 2: Drop punctuation-only diffs from $\mathcal{D}$
- 3: **if** $|\mathcal{D}| < 2$ **then**
- 4: **return** $\{(x, y)\}$ *// insufficient errors; emit only the original pair*
- 5: **end if**
- 6: **for** $v \in \{1, \dots, V_{\text{max}}\}$ **do**
- 7: $k \leftarrow \text{UNIFORM}\{1, \dots, |\mathcal{D}| - 1\}$ *// number of errors to fix in this variant*
- 8: Sample $S \subseteq \mathcal{D}$, $|S|=k$, with each diff $d \in \mathcal{D}$ weighted by $w_{\text{subtype}(d)}$
- 9: $x_v \leftarrow$ apply each $d \in S$ to x by substituting the OCR span with the ground-truth span
- 10: Emit pair (x_v, y) *// target equals the original ground-truth y*
- 11: **end for**
- 12: Emit the original pair (x, y) alongside the variants

The procedure runs once over the 6,614 training-partition source rows to produce 5,463 synthetic correction pairs in addition to the 6,614 original correction pairs and 2,791 identity pairs (Section 6.3, Appendix A); the totals match the corpus statistics reported throughout the thesis. The subtype-weight assignment behind w is described in Section 6.2 together with the subtype taxonomy.

APPENDIX B - ERROR DETECTION DETAILS

Token-Level Labelling Rule

The True column for the per-token detection tables in Chapter 7 (Methods 1, 2, and 3) is generated by a token-level Levenshtein alignment between the OCR-side and reference-side WordPiece sequences, with every **replace**, **insert**, or **delete** position in the alignment receiving label 1 (the canonical labelling rule is implemented in the detection scripts shared by all three methods). The labelling therefore follows the WordPiece alignment rather than character-level errors. Example 10 illustrates the consequence: under BERTurk (dbmdz/bert-base-turkish-cased), the reference word *çirkefiyle* splits as [ç, ##irk, ##efi, ##yle] while the OCR word *çirkefiyla* splits as [ç, ##irk, ##ef, ##ıyla], so the alignment marks position 2 as ##ef ↔ ##efi and position 3 as ##ıyla ↔ ##yle; both positions receive True = 1 even though the constituent characters of ##ef are themselves correct. Consequently, the token-level F₁ values reported for the three detection methods should be read as a measure of each detector’s ability to flag the WordPiece-alignment positions where errors are located, not as a measure of which exact characters are wrong — a granularity choice that propagates through all three detection methods, which share the same labelling rule.

Per-Book Detection Results

Tables B.1–B.6 report per-book detection metrics on the test set for each detector configuration evaluated in Chapter 7. Aggregate values match the main text. AUC denotes ROC-AUC. The *Err* column is the fraction of evaluation units (tokens or characters, depending on the configuration) tagged as erroneous in each book’s test partition.

Table B.1 Method 1 (token-level beta calibration): per-book detection breakdown on the test set.

Book	Seq	Tok	Err (%)	F₁	P	R	AUC
Arnavutlar Solyotlar	555	5,537	26.2	0.560	0.509	0.623	0.747
Hayattan Sahifeler	112	1,352	14.9	0.526	0.419	0.706	0.818
Tarihçe-i Erzurum	75	1,518	34.1	0.568	0.552	0.584	0.715
Ususül Hikem	86	1,986	34.4	0.637	0.566	0.728	0.775
Overall	828	10,393	27.4	0.577	0.521	0.647	0.759

Table B.2 Method 1 (character-level beta calibration): per-book detection breakdown on the test set.

Book	Seq	Char	Err (%)	F₁	P	R	AUC
Arnavutlar Solyotlar	555	18,854	8.4	0.335	0.251	0.501	0.766
Hayattan Sahifeler	112	4,417	4.8	0.286	0.197	0.524	0.821
Tarihçe-i Erzurum	75	4,764	10.8	0.320	0.263	0.409	0.723
Ususül Hikem	86	4,620	16.6	0.420	0.352	0.520	0.748
Overall	828	32,655	9.4	0.347	0.268	0.492	0.766

Table B.3 Method 1 multi-feature logistic regression: per-book detection breakdown on the test set.

Book	Seq	Tok	Err (%)	F₁	P	R	AUC
Arnavutlar Solyotlar	555	5,537	26.2	0.585	0.537	0.641	0.797
Hayattan Sahifeler	112	1,352	14.9	0.500	0.402	0.662	0.819
Tarihçe-i Erzurum	75	1,518	34.1	0.575	0.574	0.576	0.766
Ususül Hikem	86	1,986	34.4	0.629	0.529	0.776	0.788
Overall	828	10,393	27.4	0.588	0.528	0.663	0.796

Table B.4 Method 2 (BERT-only contextual classifier): per-book detection breakdown on the test set.

Book	Seq	Tok	Err (%)	F ₁	P	R	AUC
Arnavutlar Solyotlar	555	5,537	26.2	0.670	0.653	0.688	0.855
Hayattan Sahifeler	112	1,352	14.9	0.463	0.398	0.552	0.814
Tarihçe-i Erzurum	75	1,518	34.1	0.648	0.595	0.712	0.809
Ususül Hikem	86	1,986	34.4	0.631	0.547	0.747	0.772
Overall	828	10,393	27.4	0.640	0.591	0.697	0.834

Table B.5 Method 3 (additive-fusion BERT, init $\lambda = 0.1$ following (Hemmer et al., 2024)): per-book detection breakdown on the test set. “Raw” and “ β -cal” refer to the confidence input fed to the fusion module.

Book	Seq	Tok	Err (%)	F ₁		AUC	
				Raw	β -cal	Raw	β -cal
Arnavutlar Solyotlar	555	5,537	26.2	0.680	0.679	0.860	0.859
Hayattan Sahifeler	112	1,352	14.9	0.443	0.460	0.808	0.807
Tarihçe-i Erzurum	75	1,518	34.1	0.649	0.653	0.806	0.804
Ususül Hikem	86	1,986	34.4	0.616	0.619	0.760	0.760
Overall	828	10,393	27.4	0.638	0.641	0.833	0.833

Table B.6 Method 3 (concatenation-fusion BERT): per-book detection breakdown on the test set. “Raw” and “ β -cal” refer to the confidence input fed to the fusion module.

Book	Seq	Tok	Err (%)	F ₁		AUC	
				Raw	β -cal	Raw	β -cal
Arnavutlar Solyotlar	555	5,537	26.2	0.682	0.684	0.867	0.869
Hayattan Sahifeler	112	1,352	14.9	0.513	0.508	0.840	0.842
Tarihçe-i Erzurum	75	1,518	34.1	0.653	0.657	0.813	0.813
Ususül Hikem	86	1,986	34.4	0.598	0.612	0.761	0.767
Overall	828	10,393	27.4	0.643	0.647	0.842	0.844

The high-base-rate books, Tarihçe-i Erzurum and Ususül Hikem, contribute most of the absolute predictive content for each detector configuration; on Hayattan Sahifeler, where only 4–18% of evaluation units are erroneous, every detector’s precision drops

sharply while recall holds, producing the F_1 minima visible in each table. The character-level Method 1 in Table B.2 reports the largest base-rate-driven precision drop, because aggregating along characters rather than tokens lowers the global error rate from 27.4% to 9.4%.

Calibration Bin Tables

Tables B.7–B.9 report the per-bin observed error rates that underlie the calibration curves of Section 4.3 and Fig. 4.1. Bins are equal-width over the minimum-token-confidence range $[0, 1]$. *Observed* is the empirical sequence-level error rate within the bin; *Naïve* is $1 - s$ at the bin center; *Beta-cal.* is the value produced by the fitted restricted beta calibrator with $a=0.1387$, $b=0.2276$ (Section 4.3); *Mean CER* is the mean character-level error rate of sequences in the bin. The training-partition table is the data behind Fig. 4.1. A small number of sequences (6 of 6,614 training, 2 of 852 development, and 2 of 828 test) carry no scoreable token and therefore have no defined minimum-token confidence; these are excluded from the bins, so each table’s n column sums slightly below its partition size.

Table B.7 Calibration bins on the training partition. n is sequence count per bin.

Bin	Center	n	Observed	Naïve	Beta-cal.	Mean CER
[0.0, 0.1)	0.05	102	0.912	0.95	0.654	0.188
[0.1, 0.2)	0.15	206	0.961	0.85	0.615	0.210
[0.2, 0.3)	0.25	422	0.943	0.75	0.594	0.189
[0.3, 0.4)	0.35	614	0.936	0.65	0.578	0.169
[0.4, 0.5)	0.45	743	0.904	0.55	0.564	0.136
[0.5, 0.6)	0.55	865	0.868	0.45	0.550	0.125
[0.6, 0.7)	0.65	626	0.812	0.35	0.535	0.095
[0.7, 0.8)	0.75	592	0.720	0.25	0.519	0.078
[0.8, 0.9)	0.85	634	0.683	0.15	0.497	0.067
[0.9, 1.0]	0.95	1,804	0.400	0.05	0.455	0.032

Table B.8 Calibration bins on the development partition. Beta calibrator parameters are fitted on training only; values shown here are out-of-fit predictions.

Bin	Center	n	Observed	Naïve	Beta-cal.	Mean CER
[0.0, 0.1)	0.05	13	1.000	0.95	0.654	0.361
[0.1, 0.2)	0.15	30	1.000	0.85	0.615	0.198
[0.2, 0.3)	0.25	61	0.902	0.75	0.594	0.180
[0.3, 0.4)	0.35	83	0.928	0.65	0.578	0.157
[0.4, 0.5)	0.45	92	0.870	0.55	0.564	0.112
[0.5, 0.6)	0.55	85	0.882	0.45	0.550	0.103
[0.6, 0.7)	0.65	85	0.824	0.35	0.535	0.085
[0.7, 0.8)	0.75	81	0.728	0.25	0.519	0.141
[0.8, 0.9)	0.85	72	0.708	0.15	0.497	0.066
[0.9, 1.0]	0.95	248	0.448	0.05	0.455	0.038

Table B.9 Calibration bins on the test partition. Beta calibrator parameters are fitted on training only; values shown here are out-of-fit predictions.

Bin	Center	n	Observed	Naïve	Beta-cal.	Mean CER
[0.0, 0.1)	0.05	14	0.929	0.95	0.654	0.243
[0.1, 0.2)	0.15	39	0.974	0.85	0.615	0.190
[0.2, 0.3)	0.25	51	0.961	0.75	0.594	0.191
[0.3, 0.4)	0.35	91	0.956	0.65	0.578	0.164
[0.4, 0.5)	0.45	103	0.951	0.55	0.564	0.148
[0.5, 0.6)	0.55	98	0.908	0.45	0.550	0.103
[0.6, 0.7)	0.65	71	0.803	0.35	0.535	0.102
[0.7, 0.8)	0.75	81	0.753	0.25	0.519	0.073
[0.8, 0.9)	0.85	69	0.667	0.15	0.497	0.076
[0.9, 1.0]	0.95	209	0.445	0.05	0.455	0.035

The training, development, and test bin tables show the same structure: the naïve $1 - s$ predictor over-states error rates in low-confidence bins and under-states them in high-confidence bins, while the fitted beta calibrator compresses both ends toward the marginal base rate. Across the test bins, the observed error rate exceeds the calibrated prediction in nine of ten bins; the highest-confidence bin is the only one where the calibrated prediction slightly over-estimates. This residual mis-calibration in the highest-confidence bin is small (0.445 vs. 0.455) and matches the pattern visible in Fig. 4.1.

APPENDIX C - CORRECTION ANALYSIS DETAILS

Subtype-By-Book Error Character Distribution

Tables C.1 and C.2 give the numerical companion to the subtype $\times$ book heatmap of Figure 4.2 (Section 4.5.2), reporting per-book error-character counts for each of the thirteen subtypes on the full corpus and on the test partition respectively.

Table C.1 Subtype $\times$ book distribution of error characters (corpus scope, $n = 28,198$). Companion to Figure 4.2. Book columns: Arnavutlar (Arnavutlar Solyotlar), Hayattan (Hayattan Sahifeler), Tarihçe (Tarihçe-i Erzurum), Ususül (Ususül Hikem).

Subtype	Arnavutlar	Hayattan	Tarihçe	Ususül	Total
Circumflex substitution	769	74	538	313	1,694
Dotted/dotless i substitution	320	56	86	85	547
Turkish diacritic	803	194	618	871	2,486
Vowel substitution	642	105	178	380	1,305
Other character substitution	1,527	281	535	773	3,116
Short multi-character substitution (2–3)	3,041	513	993	1,791	6,338
Medium multi-character substitution (4–6)	661	101	243	557	1,562
Long multi-character substitution (7+)	241	39	103	160	543
Character omission	1,848	330	612	1,202	3,992
Case substitution	900	277	326	79	1,582
Punctuation	377	161	209	207	954
Spacing omission	99	21	46	195	361
Unresolvable span	1,711	308	657	1,042	3,718
Total	12,939	2,460	5,144	7,655	28,198

Table C.2 Subtype $\times$ book distribution of error characters (test partition scope, $n = 3,092$).

Subtype	Arnavutlar	Hayattan	Tarihçe	Ususül	Total
Circumflex substitution	104	8	67	39	218
Dotted/dotless i substitution	38	2	9	11	60
Turkish diacritic	69	16	84	57	226
Vowel substitution	61	15	14	28	118
Other character substitution	166	28	63	92	349
Short multi-character substitution (2–3)	318	49	89	189	645
Medium multi-character substitution (4–6)	89	23	31	89	232
Long multi-character substitution (7+)	9	0	12	38	59
Character omission	240	25	54	131	450
Case substitution	90	30	38	8	166
Punctuation	46	19	15	17	97
Spacing omission	10	2	4	18	34
Unresolvable span	210	24	63	141	438
Total	1,450	241	543	858	3,092

Unresolvable-Spans Inventory

The unresolvable-spans construction in Section 8.1 excludes 438 GT-anchored character positions across 135 outer replace spans. Tables C.3–C.4 report the per-book distribution of these positions and the joint outcome of both ByT5 variants on the corresponding outer spans.

Table C.3 Per-book distribution of unresolvable spans on the test set. *Pct* is the percentage share of the 438 unresolvable positions.

Book	n (seq)	n (pos)	n (spans)	Pct (%)
Arnavutlar Solyotlar	61	210	66	47.95
Hayattan Sahifeler	8	24	8	5.48
Tarihçe-i Erzurum	17	63	18	14.38
Ususül Hikem	31	141	43	32.19
Overall	117	438	135	100.00

Table C.4 Joint per-span fate of the unweighted and confidence-weighted ByT5 variants on the 135 unresolvable outer spans. Each row is one combination of unweighted-span outcome $\times$ confidence-weighted-span outcome.

Unweighted on span	Confidence-weighted on span	n (spans)
corrected	corrected	2
failed_or_altered	failed_or_altered	18
failed_or_altered	untouched	4
untouched	corrected	1
untouched	failed_or_altered	1
untouched	untouched	109
Total		135

The dominant outcome on unresolvable spans is ‘untouched–untouched’ (109 of 135, 80.7%), confirming the conservatism observation in Section 8.1: the corrector copies the OCR span verbatim. Of the 26 spans where at least one variant edits the span, 18 are double-failures (both variants alter the span without recovering the gold reference); only 3 spans are corrected by one or both variants. The two ‘corrected–corrected’ spans are the only cases where the alignment around the outer span cooperates with both models’ edits to produce gold output despite the inner GT/OCR length mismatch.

Additional Failed-Correction Examples

Tables C.5–C.10 present worked examples that illustrate failure modes of the single-stage corrector. Each table corresponds to one category. All examples are drawn from the canonical 828-sequence test set with selection seed 42; sequences used as running examples in Chapter 4 are excluded from the candidate pool. CER values quoted in each row recompute the canonical predictions to numerical precision.

Category A: Tarihçe-i Erzurum Degradation

Sequences from Tarihçe-i Erzurum where both variants worsen the OCR baseline.

Table C.5 Worked example for Category A. CER values are computed by the canonical metric (Section 5.2). UW: unweighted; CW: confidence-weighted.

Example A.1 (<i>Tarihçe-i Erzurum</i> , row 8155)	
OCR	" ve müzükenâhım gül mümeyyizuk " tarrak-ı mümâlâkın her tarafını sarsıyordu. zincirler
Gold	" ve mezaknahum külle mümezzek " tarakası memalikin her tarafını sarsıyordu. Zincirler
UW pred	"Ve Müzükenâhım Gül Mümeyyizuk" tarrak-ı mümalakın her tarafını sarsıyordu. Zincirler
CW pred	"Ve Müzükenâhım Gül Mümeyyizuk" tarrak-ı mümâlâkın her tarafını sarsıyordu. Zincirler
CER	baseline 0.233; UW 0.267; CW 0.279
Status	UW degraded; CW degraded

In this Tarihçe-i Erzurum sequence the OCR baseline is already noisy at the surface level (capitalisation, sparse circumflex placement); both variants over-edit the surface spans, and the confidence-weighted variant introduces an additional substitution that pushes CER higher than the unweighted model and even higher than leaving the OCR untouched. The opening quoted span is a transliterated Arabic phrase that the OCR has heavily corrupted; both variants edit individual surface characters inside it without recovering the underlying quotation, which the local context and the training distribution give them no basis to reconstruct.

Category B: Long Unresolved Span

A sequence with an outer replace span of length 12 (OCR side) versus length 3 (gold side), both variants emitting the OCR span unchanged.

Table C.6 Worked example for Category B.

Example B.1 (<i>Ususül Hikem</i> , row 7374)	
OCR	ve leğen-i savletâdın muğteninden bu mikdâr kasaba derhal keşiçün
Gold	velâkin soltadın tûfenginden bir mikdâr kısadır anınçün
UW pred	ve leğen-i savletâdın muğteninden bu mikdâr kasaba derhal keşiçin
CW pred	ve leğen-i savletadın muğteninden bu mikdâr kasaba derhal keşiçin
CER	baseline 0.527; UW 0.545; CW 0.527
Status	UW degraded; CW unchanged
Outer span: OCR ‘ derhal keşi ’ (12 ch) → GT ‘ nın ’ (3 ch); both variants leave the span untouched.	

This is the failure mode behind the bulk of Table C.4’s ‘untouched–untouched’ outcomes: when the OCR span and the gold span have very different lengths and share few surface characters, both ByT5 variants emit the OCR span verbatim — a hard limit of the local-context post-correction setup.

Category C: Model-Introduced Circumflex Events

One sequence where the unweighted variant introduces a new circumflex at a gold non-circumflex position; one where it removes a gold circumflex.

Table C.7 Worked examples for Category C.

Example C.1 (introduced; <i>Ususül Hikem</i> , row 6965)	
OCR	ve galebeye muvaffak olmak bi'l-müşâhede ve el-ma'âtinati sâbit olan
Gold	ve galebeye muvaffak olmak bi'l-müşâhede ve'l-mu'âyene sâbit olan
UW pred	ve galebeye muvâffak olmak bi'l-müşâhede ve el-ma'âtinâtı sâbit olan
CW pred	ve galebeye muvaffak olmak bi'l-müşâhede ve el-ma'âtinâtı sâbit olan
CER	baseline 0.123; UW 0.138; CW 0.123
Status	UW degraded; CW unchanged
Example C.2 (removed; <i>Tarihçe-i Erzurum</i> , row 8143)	
OCR	esami-i mevkufîn: Hacı'Âkif Ağa - Torak Bey - Fârûk Bey-
Gold	Esami-i mevkufîn: Hacı Âkif Ağa - Durak Bey - Fârûk Bey-
UW pred	Esami-i mevkufin: Hacı Âkif Ağa - Torak Bey - Fâruk Bey-
CW pred	Esami-i mevkufin: Hacı Âkif Ağa - Torak Bey - Fâruk Bey-
CER	baseline 0.071; UW 0.071; CW 0.071
Status	UW unchanged; CW unchanged

In Example C.1 the unweighted variant writes a circumflex at a position where the gold reference has a plain vowel; in Example C.2 it drops a gold circumflex from a vowel that the OCR had encoded correctly. These two events together exemplify the pattern reported in the per-subtype analysis of Section 8.1: the model treats high-confidence vowels as candidates for both adding and removing diacritical marks.

Category D: Inter-Variant Disagreement

One sequence on which confidence weighting helps; one on which it does not. The strict pattern *unweighted improved + confidence-weighted degraded* returns zero matches in the 828-sequence test set; the example on which the unweighted variant was strictly better than confidence-weighted therefore shows the relaxed pattern *unweighted improved + confidence-weighted unchanged*, of which 17 cases exist (Section 6.4 discusses the broader conservatism asymmetry).

Table C.8 Worked examples for Category D.

Example D.1 (CW better; <i>Arnavutlar Solyotlar</i> , row 1191)	
OCR	rilan on iki küçük giden mürekkep bir
Gold	rilen on iki küçük gemiden mürekkep bir
UW pred	rilan on iki küçük giden mürekkep bir
CW pred	rilen on iki küçük giden mürekkep bir
CER	baseline 0.077; UW 0.128; CW 0.051
Status	UW degraded; CW improved
Example D.2 (UW improved, CW unchanged; <i>Arnavutlar Solyotlar</i> , row 2172)	
OCR	öldürmek hiç de şan-ı sayılamayacağını
Gold	öldürmek hiç de şan sayılamayacağını icap
UW pred	öldürmek hiç de şan sayılamayacağını
CW pred	öldürmek hiç de şan-ı sayılamayacağını
CER	baseline 0.171; UW 0.122; CW 0.171
Status	UW improved; CW unchanged

In Example D.1 the confidence-weighted variant resolves a low-confidence span correctly while the unweighted variant introduces a vowel-flip and a $p \rightarrow b$ substitution. In Example D.2 the unweighted variant edits the OCR baseline successfully while the confidence-weighted variant leaves it unchanged. Across the 828 test sequences, three sequences fall in the strict pattern where confidence weighting is better than the unweighted variant (unweighted degraded, confidence-weighted improved), zero sequences fall in the strict reverse direction, and seventeen fall in the relaxed reverse direction (unweighted improved, confidence-weighted unchanged). The asymmetry is consistent with the per-book conservatism pattern reported in Section 6.4: confidence weighting reduces overcorrections more often than it adds correct ones it would otherwise miss.

Category E: Dotted/dotless i Substitution

Two sequences exhibiting the model-introduced flip in each direction.

Table C.9 Worked example for Category E (i $\rightarrow$ ı direction).

Example E.1 (<i>Arnavutlar Solyotlar</i> , row 2196)	
OCR	âdeta bir hımbırî bir birader gibi hüsn-i telakki
Gold	âdeta bir hemşehri bir birader gibi hüsn-i telakki
UW pred	âdeta bir hımbır1 bir birader gibi hüsn-i telakki
CW pred	âdeta bir hımbır1 bir birader gibi hüsn-i telakki
CER	baseline 0.100; UW 0.100; CW 0.100
Status	UW unchanged; CW unchanged

Table C.10 Worked example for Category E (ı $\rightarrow$ i direction).

Example E.2 (<i>Arnavutlar Solyotlar</i> , row 2528)	
OCR	bî mucip olmadı. Hatta Maryola'nın
Gold	bı mucip olmadı. Hatta Maryola'nın
UW pred	bi mucip olmadı. Hatta Maryola'nın
CW pred	bi mucip olmadı. Hatta Maryola'nın
CER	baseline 0.029; UW 0.029; CW 0.029
Status	UW unchanged; CW unchanged

In both examples the variants substitute the wrong member of the dotted/dotless ‘i’ pair: in Example E.1 the model writes ‘ı’ where the gold form has a dotted ‘i’, and in Example E.2 it writes ‘i’ where the gold form has a dotless ‘ı’. The CERs are unchanged at the sequence level only because the substituted character is itself an error in the OCR baseline; at the token level both events count as model-introduced errors at the gold-anchored position.

APPENDIX D - REPRODUCIBILITY

Hyperparameter Documentation

Table D.1 consolidates the chosen hyperparameter values for the five trained canonical models alongside a brief documentation status. The audit underlying the status column was performed by reading every training script, every training log, every per-model training-config record, and the project-level pipeline notes. No project-wide hyperparameter sweep was found, so a documented sweep grid is not available for any model. Status codes used in the table: *matched* = the value was explicitly chosen to match another model in this thesis; *constraint* = a comment in the training script cites a non-quality constraint (typically GPU memory); *paper-ref* = the value is taken from a cited paper’s reported configuration; *derived* = the value is loaded from another fitted artifact in this thesis; *searched* = an explicit grid search is recorded in the training log; *none* = the chosen value is present without documented justification.

Table D.1 Chosen hyperparameter values per model with documentation status. UW = ByT5 unweighted; CW = ByT5 confidence-weighted; M2 = BERT-only; M3_a = additive fusion; M3_c = concatenation fusion.

Model	Hyperparameter (chosen value)	Status
UW	google/byt5-small (~300M params)	constraint
UW	epochs = 10	none
UW	per-device batch size = 2, grad. accumulation = 4 (effective 8)	constraint
UW	learning rate = 5×10^{-5} , warmup ratio = 0.1, weight decay = 0.01	none
UW	early stopping patience = 3, threshold = 0.001	none
UW	max input length = 128, max target length = 128	none
UW	beam search width = 4	none
UW	precision: bf16, max grad norm = 0.5	none
UW, CW	random seed = 42	matched
CW	batch size, grad. accumulation, lr, warmup, weight decay, early stop, max length	matched (UW)
CW	beta calibration $a = 0.1387$, $b = 0.2276$	derived (M1 token-level fit)
CW	weight floor = 0.2, weight ceiling = 1.0	none
CW	synthetic-pair fixed weight = 0.5, identity-pair fixed weight = 1.0	none
M2, M3 _a , M3 _c	dbmdz/bert-base-turkish-cased (110M params)	matched
M2, M3 _a , M3 _c	supervision: all subwords inherit the parent-token label	none (M2); matched (M3 _a , M3 _c)
M2, M3 _a , M3 _c	lr = 2×10^{-5} , batch size = 16, epochs = 10, weight decay = 0.01, warmup = 0.1	none
M2, M3 _a , M3 _c	focal loss with $\gamma = 2.0$, classifier dropout = 0.3, grad clip = 1.0	none
M2, M3 _a , M3 _c	post-hoc temperature scaling on dev logits, then F1-optimal θ on dev probs	none
M3 _a	additive-fusion init $\lambda = 0.1$	paper-ref (Hemmer et al., 2024)
M3 _c	concat fusion: confidence MLP hidden = 64, dropout = 0.3	none

The strongest documented choices are the byte-level model size (selected for the small-data regime) and the additive-fusion initial coefficient for Method 3 (taken from the ConfBERT paper (Hemmer et al., 2024)). The remaining values are typical defaults for HuggingFace fine-tuning of ByT5-small and BERT-base; we report this

honestly rather than reconstruct a search post-hoc. The single fitted hyperparameter that does carry a documented selection procedure is the beta calibration regularizer λ , whose six-value grid and dev-set selection rule are recorded in Section 7.2 of the main text.

Compute And Runtime

Table D.2 reports compute and wall-clock figures for every trained model and for the three fit-only Method 1 detectors. All values are extracted from on-disk training logs via timestamp differences, not estimated.

Table D.2 Compute and runtime per model, extracted from on-disk training logs. Method 1 character-level and multi-feature variants are CPU-only fits.

Model	Params	Epochs	Wall-clock (s)	Train-only (s)	Mean epoch (s)	Test infer. (s)	GPU
ByT5 unweighted	299.6M	9*	9,093	5,591	621.3	201.1	RTX 4090
ByT5 confidence-w.	299.6M	9*	5,712	5,501	611.2	201.8	cuda [†]
Method 2 (BERT-only)	110.9M	10	167	144	14.4	0.5	cuda [†]
Method 3 additive	111.6M	10	~190	~145	~14.5	~0.5	cuda [†]
Method 3 concat	111.6M	10	191	145	14.5	0.5	cuda [†]
Method 1 token (fit)	—	—	3	—	—	—	cpu
Method 1 char (fit)	—	—	3	—	—	—	cpu
Method 1 multi-feat. (LR)	9 wts	—	14	—	—	—	cpu

* Both ByT5 variants were configured for 10 epochs but converged at epoch 9 under the early-stopping criterion (patience = 3, threshold = 0.001).

[†] The training script logs only that the device is a CUDA GPU for these four models without recording the specific hardware. The unweighted ByT5 log is the only one that records the SKU explicitly; that run was on an NVIDIA GeForce RTX 4090.

Three quantities that an examiner might reasonably expect are absent from this table: peak GPU memory, CUDA version, and PyTorch version. None of the canonical training scripts emit these to the log, and we report this gap honestly rather than reconstruct values post-hoc. The training scripts also do not log per-step timing, so the mean-epoch column is computed by dividing the train-only time by the number of epochs and reflects the full epoch including both forward/backward passes and per-epoch development evaluation.

Statistical Methods Notes

The thesis reports three families of inference: (a) per-sequence change tests on paired binary outcomes, (b) per-sequence resampling for ΔF_1 between detector configurations, and (c) parameter fitting for calibration. The methodology choices are summarized below.

Paired Binary Outcomes (Degradation Flag, Improvement Flag)

The single-stage corrector results in Section 6.4 report McNemar’s exact test on the contingency tables of degraded-or-not and improved-or-not classifications across the unweighted and confidence-weighted variants, both evaluated on the 828-sequence test set. McNemar’s test is the standard paired-binary-outcome procedure for comparing two classifiers on the same examples (Dietterich, 1998); the exact form is used in preference to the asymptotic χ^2 approximation because some off-diagonal cells in the contingency tables have small counts (e.g., 10 sequences for the degradation crosstab’s smaller off-diagonal cell).

Sequence-Level Paired Bootstrap On ΔF_1

The detection-method comparison in Section 7.5 reports paired-bootstrap confidence intervals and centered two-sided p values for ΔF_1 between each pair of methods at the per-sequence level. The procedure is the classical paired-resamples-of-the-test-set bootstrap of Koehn (2004) and Berg-Kirkpatrick et al. (2012), with $B = 10,000$ replicates. A single set of resampled sequence indices is applied to all three methods at every replicate, so that within-replicate dependence between methods is preserved (this is the operative form of ‘paired’). Confidence intervals are reported as percentile intervals on ΔF_1 at the $\alpha = 0.05$ level, and centered two-sided p values are computed by shifting the bootstrap distribution to a mean of zero and counting replicates with $|\Delta F_1^{(b)} - \Delta F_1^{\text{obs}}| \geq |\Delta F_1^{\text{obs}}|$. Random seed is fixed at the bootstrap level (one seed per call) so that the pairwise comparisons are reproducible. The bootstrap implementation follows the resampling formulation of Efron (1979); the test choice for paired classifier comparison follows the recommendations of Koehn (2004) on this class of evaluation.

Calibration Parameter Fitting

The beta-calibration parameters a and b in Section 4.3 are fitted by minimizing negative log-likelihood with L-BFGS-B as recommended in Kull et al. (2017), using five random initial points per fit and selecting the converged solution with the lowest training loss. The five initial points all converge to the same solution $(a, b) = (0.1387, 0.2276)$ in our experiments, indicating a well-behaved likelihood surface; the multi-start procedure is therefore retained as a precaution rather than as an active tie-breaking step. Temperature scaling fits use the same L-BFGS-B procedure with bounded $T \in [0.01, 10.0]$, again following Guo et al. (2017).

Multiple Testing

The thesis performs several groups of paired comparisons (per-book correction outcomes; pairwise detection-method comparisons). We do not apply a multiple-testing correction to these comparisons. The reasons are (i) the per-book comparisons are reported as descriptive subgroup breakdowns rather than as confirmatory tests, and (ii) the detection-method comparisons are pre-specified as the three pairwise contrasts on the same fixed test set, which is a small enough family that an alpha-level correction would have negligible practical effect. We report this choice explicitly rather than apply a correction silently.

Indexing And Reproducibility

Every table, figure, and example reported in this appendix derives from one of the canonical pipeline outputs. The selection seed used for the curated examples in Appendix C is 42; the bootstrap seed used for the detection comparison in Section 7.5 is 20,260,430.

APPENDIX E - EXTERNAL-TOOL COMPARISON DETAILS

This appendix gives the full set of external-tool configurations compared against the ByT5 models in Section 8.4, with raw and format-normalized error rates, sequence-level outcomes, and paired tests against the unweighted ByT5 model.

Table E.1 External-tool configurations on the 828-line test set: raw and format-normalized CER and WER, sequence-level outcomes, and paired McNemar p -values against the unweighted ByT5. Headline modes are the first rows of each toolkit; the remaining rows are diagnostics referenced in Section 8.4.1.

System configuration	CER raw	CER norm	WER raw	WER norm	Imp / Deg / Unch	McNemar p vs ByT5-UW
Baseline (raw OCR)	10.51	9.27	33.10	28.31	0 / 0 / 828	3.3×10^{-24}
ByT5 unweighted	9.61	8.64	28.72	25.33	244 / 79 / 505	—
ByT5 confidence-weighted	9.66	8.71	29.00	25.59	238 / 64 / 526	1.7×10^{-2}
Zemberek SpellChecker (basic)	12.80	10.15	42.52	31.14	50 / 480 / 298	6.6×10^{-91}
Zemberek SpellChecker (LM-ranked)	12.73	10.12	42.48	31.23	53 / 472 / 303	9.5×10^{-89}
Zemberek Normalizer (per-line)	15.27	10.56	66.25	32.11	49 / 628 / 151	1.7×10^{-147}
Zemberek Normalizer (sentence-split)	15.28	10.56	66.25	32.09	49 / 629 / 150	7.9×10^{-149}
TULAP Deasciifier (per-line)	10.55	9.32	33.33	28.53	4 / 22 / 802	3.0×10^{-9}
TULAP Deasciifier (sentence-split)	10.88	9.32	33.33	28.53	4 / 146 / 678	6.1×10^{-7}
TULAP Deasciifier (ASCII-fold)	14.44	13.39	50.26	46.49	51 / 599 / 178	4.0×10^{-136}

The unweighted ByT5 row is the comparison anchor, so its McNemar cell is left empty. The Baseline p -value is the no-correction control against the unweighted ByT5. The headline rows (Baseline, ByT5 unweighted, ByT5 confidence-weighted, TULAP Deasciifier per-line, and Zemberek SpellChecker LM-ranked) reproduce the values of Table 8.4.